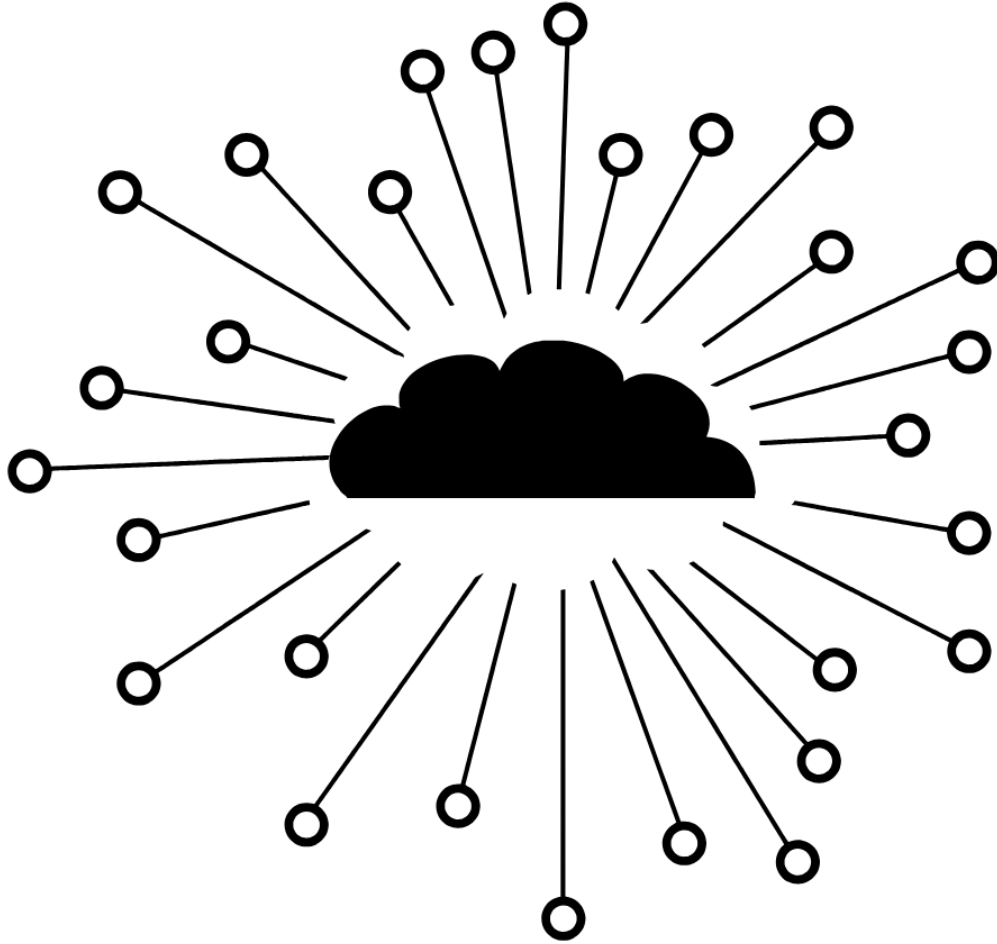




ANJAN KHADKA

THE CLOUD HANDBOOK

Personal Notes & Reference

Acknowledgment

This document is a compilation of personal study notes on Cloud Computing, prepared for learning and revision purposes. The content is not entirely original and draws upon multiple external sources.

Written explanations and concepts have been referenced from platforms such as IBM, GeeksforGeeks, and various Medium articles, along with the book *Mastering Cloud Computing*. Certain sections are based on notes converted from video/lecture content by Gate Smasher and ByteMonk.

AI tools including ChatGPT, Gemini, and a few other language models were used to help structure and organize the content. Most diagrams were created independently using Eraser, some were generated using AI prompts, and a few images have been sourced from the internet.

Detailed references and source links are provided at the end of this document.

Email: anjankhadka199@gmail.com

Github: <https://github.com/Anjan012>

Table of Contents

Acknowledgment	2
Introduction To Cloud Computing.....	9
What is Cloud Computing?.....	9
Need of Cloud Computing:.....	9
Characteristics of Cloud Computing.....	11
Evolution of Cloud computing	14
Cloud Reference Model	24
Benefits of a Cloud Reference Model	26
Cloud adoption.....	28
Cloud adoption vs. cloud migration	34
Cloud Computing Architecture	38
Why a Hierarchical Architecture?.....	40
CLOUD SERVICE & DEPLOYMENT MODELS	44
Cloud Service Models	44
SaaS – Software as a Service	46
PaaS – Platform as a Service.....	46
IaaS – Infrastructure as a Service	47
CLOUD DEPLOYMENT MODELS	50
Public Cloud	50
The Starting Point: Why Public Cloud Exists.....	51
Public Cloud Infrastructure: Who Actually Owns the Hardware?	53
Public Cloud and Multitenancy	54
Public Cloud as a Distributed System.....	56
Shared Responsibility:	57

Advantages of Public Cloud.....	58
Limitations and Challenges.....	59
Private Cloud	62
Need Private Cloud	63
Public Cloud vs Private Cloud	66
Hybrid Cloud.....	68
Cloud bursting	69
Dynamic Provisioning.....	70
Hybrid Cloud Is More Than “Using AWS and a Private Server”.....	71
Community Cloud	74
Cloud Design and Implementation Using SOA.....	77
What is SOA?.....	78
Principles of SOA.....	78
SOA in Cloud Computing.....	79
Cloud Design Using SOA	79
Implementation of SOA in the Cloud	80
Communication and Integration	81
Benefits of SOA in Cloud Computing.....	81
Challenges of SOA in Cloud Computing	82
SOA and Microservices	82
Cloud Storage Architecture.....	84
Step 1: Classifying Data by Type.....	84
Step 2: Redundancy — Never Trust a Single Copy.....	85
Step 3: Tiering — Not All Data Is Equally "Hot"	87
Virtualization in Cloud Computing and Types	90
Virtualization – Operating System Virtualization.....	100

What Is Operating System Virtualization?	100
The Role of the Hypervisor	101
Advantages of OS Virtualization	102
Types of Hypervisor (Type 1 vs Type 2)	104
Type 1 Hypervisor – Bare Metal	104
Type 2 Hypervisor – Hosted	104
Hypervisor Ecosystem	108
Virtual Machines vs Containers	114
Virtual Machines (VM) Architecture	114
Containers	116
AWS – Major Components	120
1. Compute Services	120
2. Storage Services	122
3. Database Services	123
4. Networking & Content Delivery	124
5. Security & Identity Management.....	125
6. Management & Monitoring.....	126
7. Developer Tools (DevOps).....	126
8. Machine Learning & AI	127
Cloud Security – Standards & Frameworks	129
ISO Standards for Cloud Security.....	129
Government Data Protection Regulations.....	131
Security, Trust, and Privacy in Public Cloud	134
Why Security Becomes More Difficult in the Cloud	135
The Three States of Data	136
Virtualization and the Data-in-Use Problem	137

Confidentiality, Integrity, Availability, and Privacy	137
Public Cloud and Multitenancy	139
The Noisy-Neighbor Problem	140
Network Security in the Cloud	140
Availability and DoS/DDoS Attacks	141
Malware and Shared Environments	142
Data Storage and Data Location	142
Data Deletion and Data Remanence	143
Access Control: Who Can Do What?	143
Authentication vs Authorization	144
Insider Threats	144
Usage Creep and Privacy.....	145
Identity Protection and Re-identification	146
The Cryptography Problem	147
Homomorphic Encryption	147
Privacy-Preserving Data Mining.....	151
Fine-Grained Access Control.....	154
Cryptographic Access Control	155
Centralized vs Distributed Access Control.....	156
Identity Management and Federated Identity	156
Trust in Cloud Computing.....	156
Chain of Trust and Accountability	157
Legal and Regulatory Issues	158
Shared Security Responsibility	158
A Complete Cloud Security Model	159
Major Security Challenges and Their Solutions	160

Security Trade-Offs	161
Identity and Access Management (IAM) in Cloud Computing	163
IAM in Cloud Computing	164
Why IAM Is Important in Cloud Computing	165
The Four Pillars of IAM	166
IAM Components and Technologies	168
Access Control Models	170
IAM Challenges in Cloud Computing	171
Best Practices for IAM in Cloud Computing	172
Future Trends in IAM for Cloud Computing	174
Introduction to Parallel Computing	177
Parallelism and Concurrency	178
Types and Concepts of Parallel Computing	179
Data Locality	182
Parallel Performance	182
Challenges of Parallel Computing	183
Scalability	184
How Parallel Computing Leads to MapReduce	186
MapReduce	187
Applications of MapReduce	197
Parallel Efficiency of MapReduce	200
Cloud Platforms and Applications	204
Web Services	204
AppEngine	205
Azure Platform	206
Aneka	206

Open Challenges	207
Scientific Applications	208
Business and Consumer Applications	208
References	210



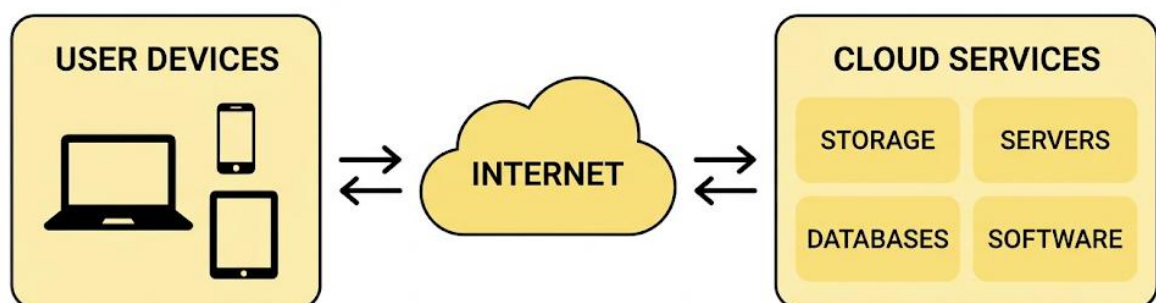
Introduction To Cloud Computing

What is Cloud Computing?

Cloud computing means delivering computing services — such as storage, servers, databases, networking, software, runtime, and virtualization — over the internet ("the cloud") instead of owning and managing physical infrastructure.

In simple words: Instead of buying and maintaining your own hardware/software, you access these services remotely over the internet, on-demand. Examples:

- Google Drive / Dropbox – you get free storage (e.g., 15 GB), can upload photos/documents, and access them from any device, anywhere, anytime.
- Netflix / Spotify – movies and songs are not downloaded/stored on your device; they stream directly from cloud servers whenever you have internet connectivity.



Need of Cloud Computing:

Imagine a small e-bike manufacturing business based in one city, doing well locally but wanting to expand nationally/globally.

Problem with traditional (on-premises) setup:

- Need to buy own servers, storage, and network hardware
- Need to hire an IT team (for servers, database, front-end, back-end, networking)
- Very high cost of infrastructure, staff, and maintenance
- Outsourcing everything to a third party (e.g., a large IT company) is also expensive

Solution: Cloud Computing (e.g., Software as a Service)

- No need to set up your own infrastructure
- Just get an admin panel/dashboard, upload your data, and manage your business
- Even a single person can manage operations
- Services become instantly accessible anywhere in the world (India, South Africa, any continent)

Cloud computing removes the need for heavy upfront investment in hardware/team and allows global reach with minimal setup.

Table: Major Cloud Providers

Cloud Provider	Approx. Market Share
AWS (Amazon Web Services)	~30–34% (Market Leader)
Microsoft Azure	~22%
Google Cloud Platform (GCP)	~10%
Alibaba Cloud	~6%
Others (IBM, Oracle, Salesforce, etc.)	Smaller share, but provide private/public cloud services

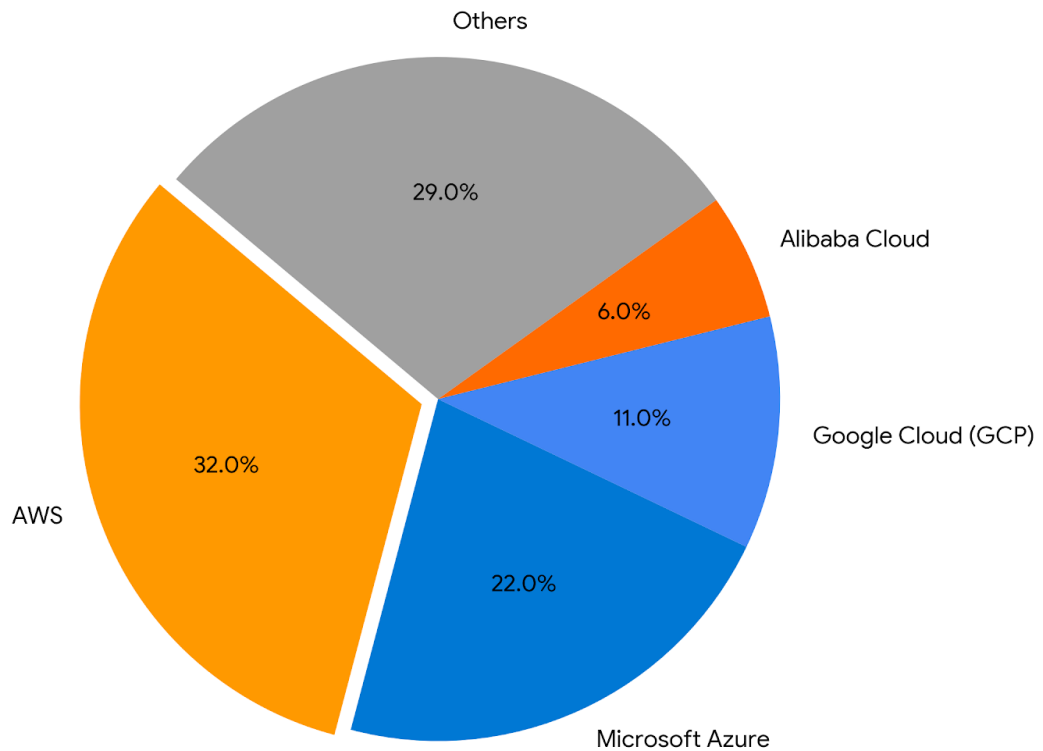


Figure: Cloud Infrastructure market share

Note: These companies (Amazon, Microsoft, Google) have massive computation power — huge numbers of CPUs, servers, and networking resources — which lets them offer reliable, scalable cloud services.

Characteristics of Cloud Computing

a) On-Demand Self-Service

- User can request resources (storage, computation, etc.) whenever needed, and get them immediately without manual intervention from the provider.
- Example: When mobile storage is full, you instantly buy more iCloud storage (monthly/yearly subscription) and get extra space right away.

b) Pay-As-You-Go (Utility/Metered Service)

- You pay only for what you use, similar to a utility bill (like electricity).

- Example: Uber/Ola – you don't buy/maintain your own car; you pay per kilometer for the ride. Stop using the service, and payment stops too.

c) Broad Network Access

- Services can be accessed from anywhere in the world via the internet, on various devices.

d) Resource Pooling

- Cloud provider's resources (servers, storage, computation) are shared among multiple users using virtualization, but appear dedicated to each user (logically separated).
- Example: A hotel swimming pool — it isn't "owned" by your room, but shared by all guests. You get access during your stay, and after checkout, another guest can use it.

e) Scalability and Elasticity

- Resources can be scaled up or down instantly based on demand.
- Example: If demand for e-bikes suddenly jumps from 1,000 to 1 lakh (or more) users (e.g., due to a government ban on petrol/diesel vehicles), the cloud can scale resources accordingly — you just pay more as you scale.
- This works because providers maintain enormous backend computing resources.

Concept	Explanation	Real-life Example
Cloud Computing	Delivery of computing services over internet	Google Drive, Netflix
On-Demand Self-Service	Instant resource provisioning	Buying more iCloud storage
Pay-As-You-Go	Pay only for usage	Uber/Ola per km billing

Broad Network Access	Access from anywhere, any device	Streaming Netflix on phone/laptop/TV
Resource Pooling	Shared resources, logically separated	Hotel swimming pool for all guests
Scalability/Elasticity	Resources scale up/down as per demand	E-bike company scaling from 1,000 to 1 lakh users

Evolution of Cloud computing

Before understanding cloud computing, let's understand the problem we were trying to solve.

At the most basic level, computing is about one thing:

We have a problem → we need computing resources → we want those resources to process data and give us results.

The question has always been:

How do we get more computing power, more storage, better reliability, and easier access without making the system unnecessarily expensive or difficult to manage?

Cloud computing did not appear suddenly. It is the result of solving this problem again and again.

Every generation solved one limitation of the previous generation — and usually created a new limitation.

The journey looks roughly like this:

*Mainframe → Distributed Systems → Cluster Computing → Grid Computing → Utility Computing
+ Virtualization → Web 2.0 + Service Orientation → Cloud Computing*

Let's understand why each step happened.

1. Mainframe Computing (1950–1970)

Imagine it is the 1950s. Computers are extremely expensive, huge machines. You cannot give every employee or organization their own computer. So, what do you do? You build one extremely powerful machine and allow many people to use it. That machine is the mainframe. Mainframes first came into existence around 1951 and were designed to handle huge amounts of data and massive input/output (I/O) operations.

The basic idea was:

Instead of everyone having a computer, let's have one very powerful computer that everyone can share.

This worked surprisingly well.

Mainframes were:

- Very powerful
- Highly reliable
- Designed for large-scale data processing
- Capable of handling massive I/O operations
- Highly fault-tolerant
- Able to operate with almost no downtime

Even today, mainframes are used for bulk processing tasks, such as large-scale online transactions. For example, imagine a bank processing millions of transactions. You don't want the machine handling those transactions to randomly shut down. So, mainframes were built with extremely high reliability.

But there was a problem.

They were extremely expensive.

You had one giant machine doing everything. If you needed more computing power, you generally had to buy more expensive mainframe capacity. *So, we had solved: How do we get powerful and reliable computing? But we had not solved: How do we get that computing power more cheaply and flexibly?*

That led us toward **distributed computing**.

2. Distributed Systems (1970–1980)

Now think from first principles. Why must one machine do all the work? If one machine can process 1,000 tasks, perhaps we can use 10 smaller machines, each processing 100 tasks. Instead of one giant computer:

Machine A + Machine B + Machine C + Machine D...

Now we have multiple independent computers working together. This is the basic idea behind a distributed system. **A distributed system is made up of multiple independent systems, but to the user they can appear as one system.**

The machines can communicate over a network and share resources. They don't even have to be in the same geographical location.

This gives us several important properties:

- **Scalability** — we can add more machines as needed.
- **Concurrency** — multiple machines can work at the same time.
- **Continuous availability** — the system can continue working even if some machines fail.
- **Resource sharing** — machines can share resources and workloads.
- **Heterogeneity** — different types of machines can participate.
- **Independence of failures** — failure of one machine does not necessarily mean the entire system fails.

So instead of:

One giant machine

we move toward:

Many machines working together

This was a major improvement. But now we have a new problem. **If I have many machines, how should I organize them? If they are close together, perhaps we can connect them with a fast network.**

That leads to **cluster computing**.

3. Cluster Computing (1980–1990)

Imagine we take several ordinary computers and put them in the same location. Then we connect them using a high-bandwidth network. Now these machines can work together almost like one powerful system. This is a cluster.

The basic idea is:

Instead of buying one expensive mainframe, connect many cheaper machines together.

This solved a major problem. Cost Clusters were much cheaper than traditional mainframes while still providing significant computing power. They were also easier to expand.

Suppose you have 10 machines and later need more computing power. You can add another machine — a node — to the cluster.

Need more power → add more nodes.

This gives us better scalability. But now look at the architecture. All these machines are usually located relatively close to one another because they need fast communication.

If Machine A sends something to Machine B, we don't want that communication to travel across a slow network. So, cluster computing solved:

High cost of centralized mainframes

But created/left another problem: Geographical restriction. What if I have useful computing resources in different cities or countries? Why should all machines have to sit in the same building?

That question leads us to **grid computing**.

4. Grid Computing (1990–2000)

Now imagine something bigger. Instead of putting all our machines in one building, let's connect machines that are geographically distributed. Maybe:

Machine A → New York

Machine B → London

Machine C → Tokyo

Machine D → Delhi

And connect them through the Internet. This is the basic idea of grid computing. A grid consists of computing resources from different locations, often belonging to different organizations.

Unlike a cluster, the machines don't have to be identical or physically close. Therefore, grid computing can contain heterogeneous nodes — different types of computers belonging to different organizations. This gives us enormous resource-sharing possibilities.

For example, an organization may have unused computing resources while another organization needs more computing power.

Why not share them? This is a natural extension of distributed computing. But now we hit another problem. Remember why clusters were located close together? Network speed.

Now our machines might be thousands of kilometers apart. Communication becomes harder. We may not have enough high-bandwidth connectivity. We also introduce more network-related problems:

- Higher communication delays
- Network failures
- Connectivity problems
- More difficulty coordinating machines

So, grid computing solved: The geographical limitation of clusters

But created a new challenge:

How do we reliably provide computing resources over networks when users need them?

This is one reason cloud computing is often described as a successor to grid computing.

But there is still another important idea missing.

5. Utility Computing (Late 1990s–2000)

Now think about electricity. You don't build your own power plant just because you need electricity. You simply use electricity and pay for what you consume. This gives us a powerful idea:

Computing should work like a utility.

Instead of buying and owning all the computing infrastructure ourselves, we should be able to consume computing resources as a service.

For example:

Need 10 servers → use them

Need 100 servers → use more

Need only 5 tomorrow → reduce usage

Pay based on usage This is utility computing.

It defines ways to provide computing services such as:

- Compute
- Storage
- Infrastructure

on a pay-per-use basis. This is a very important idea because it changes the question from:

"How much hardware do I need to buy?"

to:

"How much computing do I need right now?"

But there is still a physical problem. If one physical server is dedicated to one customer, we may still waste resources. Imagine a physical server has 100% capacity. A customer uses only 20%. What happens to the other 80%? It sits unused.

So, we need a way to share one physical machine efficiently among multiple users. That brings us to one of the most important technologies behind cloud computing: **Virtualization**.

6. Virtualization (1980–Present)

Let's start with the physical machine. Suppose we have: One physical server. Traditionally, one operating system and application workload might use that server. But what if we could divide the physical server into several virtual machines (VMs)

Now multiple workloads can run on the same physical hardware. This is virtualization. Virtualization creates a virtual layer over physical hardware, allowing multiple instances to run on the same machine. This dramatically improves resource utilization.

Instead of:

Server → One workload

we can have:

Server → Multiple virtual machines → Multiple workloads

This is a **foundational technology for cloud computing**. Technologies and services such as VMware vCloud and Amazon EC2 are based on virtualization concepts. Hardware virtualization remains one of the most common forms of virtualization.

Why is this so important for cloud?

Because now a cloud provider doesn't need to give you an entire physical server. They can take a large physical server and divide its resources into virtual machines. You can rent just the amount of computing capacity you need. And when you no longer need it, the provider can give those resources to someone else. This makes the utility-computing idea much more practical.

But we still need a good way for humans to interact with these services.

That brings us to the **Web**.

7. Web 2.0

Think about the early web. The web was initially much more like reading documents. You opened a page. You read it. But modern applications are different. You can:

- Interact with maps
- Post content
- Like and comment

- Upload files
- Communicate with others
- Use applications directly through a browser

This evolution is commonly called Web 2.0. Web 2.0 became especially popular around 2004. Examples include: Google Maps, Facebook, Twitter.

Web 2.0 gave us interactive and dynamic web pages and increased flexibility in how users interact with web applications. This became an important interface for cloud computing. Think about it:

The cloud may contain enormous amounts of computing power and storage somewhere else. But how does the user access it? Through the Internet and web-based interfaces. So, Web 2.0 helped make cloud services accessible and interactive for users.

But now we have another question:

If everything is becoming a service, how should those services be designed and managed?

That brings us to **service orientation**.

8. Service Orientation

Imagine we are building a large application. Instead of building one giant piece of software that does everything, we can think of the application as a collection of services. Each service performs a particular function. This makes applications more:

- Flexible
- Reusable
- Easier to change
- Easier to evolve

This idea of service orientation became an important reference model for cloud computing. It supports low-cost, flexible and evolvable applications. Two important concepts became associated with this model:

Quality of Service (QoS)

QoS is about defining and measuring the expected quality of a service. For example:

How fast should it respond?

How available should it be?

What level of performance should we expect?

This also includes the idea of an SLA (Service Level Agreement). An SLA defines the level of service that the provider promises to deliver.

Software as a Service (SaaS)

Instead of buying and installing software yourself, you can use the software as a service over the Internet. For example: Instead of installing and maintaining everything yourself → access the software through the cloud. This is the direction in which modern cloud computing develops.

So, What Is Cloud Computing? Now the pieces finally come together. We have:

Mainframes → powerful centralized computing

Distributed systems → multiple machines working together

Clusters → cheaper machines connected with high-speed networks

Grid computing → geographically distributed resources

Utility computing → computing as a pay-per-use service

Virtualization → efficiently share physical hardware

Web 2.0 → interactive web-based access

Service orientation → flexible services with defined quality and delivery

Cloud Computing

Cloud computing means storing and accessing data and programs on remote servers hosted on the Internet instead of relying only on your computer's hard drive or local server. In simple terms:

Computing resources are provided to users as services over the Internet.

Those resources can include:

- Computing power
- Data storage
- Applications
- Servers
- Development tools
- Networking services

This is why the cloud is not simply "someone else's computer." The cloud is the result of combining many ideas:

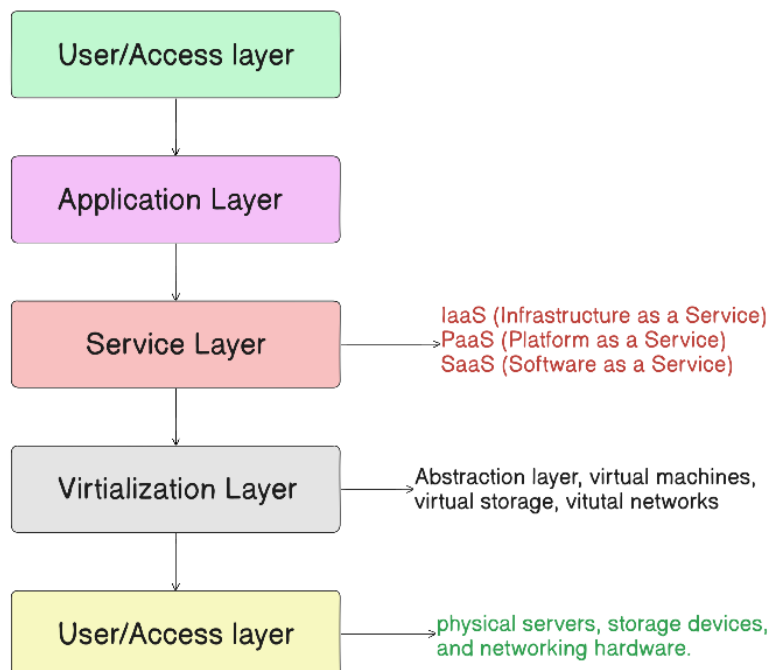
Distributed computing + networking + virtualization + utility/pay-per-use computing + service-oriented systems + web-based access

Cloud providers such as Amazon Web Services (AWS), Google Cloud and Microsoft Azure build large infrastructures and provide these resources as services.

Cloud Reference Model

A cloud reference model is a high-level architectural blueprint that describes the major components, roles, and interactions in a cloud computing environment. It provides a common framework for designing, implementing, and managing cloud systems without prescribing a specific technology or vendor.

A common cloud reference model is based on five layers:



Think of it as a map rather than a detailed construction plan. It helps organizations understand how cloud services fit together and who is responsible for what. Key Components of a Cloud Reference Model A typical cloud reference model includes:

1. Cloud Consumers

Individuals or organizations that use cloud services. Cloud consumers are individuals, businesses, or organizations that use cloud-based resources and services according to their requirements. They may use cloud services for storing

data, running applications, or hosting websites. For example, a company may use Microsoft Azure or AWS to host its business applications.

2. Cloud Service Provider

The organization that provides cloud services. cloud service provider is an organization that delivers cloud computing services to consumers over the internet. It is responsible for managing the infrastructure, platforms, applications, security, maintenance, and availability of the services provided to users.

3. Cloud Services

Cloud services provide computing resources and applications to users over the internet. Cloud services are generally divided into three main service models: IaaS, PaaS, and SaaS. **Infrastructure as a Service (IaaS)** provides virtual machines, storage, and networking resources. **Platform as a Service (PaaS)** provides development platforms, databases, and middleware for application development. **Software as a Service (SaaS)** provides complete software applications that users can access through the internet.

4. Management Layer

Manages and controls cloud resources and services. The management layer is responsible for managing the operation of cloud resources and services. It handles activities such as resource provisioning, monitoring, billing, automation, orchestration, and reporting. This layer helps ensure that cloud resources are used efficiently and services remain available.

5. Security and Governance

Protects cloud resources and ensures compliance with rules and policies. Security and governance are important parts of a cloud reference model. They include identity and access management (IAM), data protection, compliance, auditing, and risk management. These functions help protect cloud resources and data from unauthorized access and ensure that the organization follows required standards and regulations.

6. Underlying Infrastructure

Provides the physical and virtual resources required for cloud computing. The underlying infrastructure forms the foundation of cloud computing. It includes physical servers, storage systems, networking equipment, and virtualization technologies. These resources provide the computing power, storage capacity, and network connectivity required to deliver cloud services.

Benefits of a Cloud Reference Model

1. Common Language

Provides a common language for stakeholders. A cloud reference model provides a common way for cloud users, providers, developers, and administrators to understand and discuss cloud components and services.

2. Standardized Architecture

Standardizes cloud architecture across projects. It provides a structured approach to designing cloud systems, helping organizations maintain consistency and standardization across different cloud projects.

3. Identifies Components

Helps identify required components and their relationships. A reference model makes it easier to identify the different components required in a cloud environment and understand how these components interact with each other.

4. Security and Compliance

Supports security and compliance planning. It helps organizations identify security requirements and plan measures for data protection, access control, auditing, and regulatory compliance.

5. Cloud Migration

Simplifies migration from on-premises systems to the cloud. A cloud reference model provides a clear structure for analyzing existing systems and determining how their resources and applications can be moved to a cloud environment.

6. Interoperability

Improves interoperability between cloud solutions. It helps different cloud components and services work together more effectively, making it easier for organizations to integrate multiple cloud solutions and technologies.



Cloud adoption

Cloud adoption is the process of adding and using cloud services in an organization's IT infrastructure.

This shift from on-premises storage, where an organization's applications and data are hosted on its own physical servers, is an important part of digital transformation. By hosting IT infrastructure in the cloud, organizations can access data from any location, increase or decrease resources based on demand, and make systems more reliable and able to handle failures.

Modernizing in this way can be easier said than done. In 2025, Gartner predicted that 25% of organizations will be unhappy with their cloud adoption by 2028 because of cost or poor management.

There is an important difference between cloud adoption and cloud migration. While cloud migration is the technical process of moving data from on-premises servers to the cloud, cloud adoption is the broader process of bringing cloud technology into business operations.

Cloud adoption is also a change in how an organization works. It requires support from both business and technical staff, along with careful attention to cost, compliance, and training for the new cloud practices being introduced.

Common cloud adoption approaches include:

- **“Lift and shift”** — directly moving applications and data to the cloud with little or no changes.
- **Replatforming** — making specific improvements to applications during migration so they can use cloud capabilities, without changing the main structure of the application.

- **Rebuilding IT infrastructure** — redesigning and rebuilding systems so they are fully cloud-native, meaning they are specifically designed to run in and take advantage of the cloud.

The process of cloud transformation looks different depending on which approach an organization chooses. However, it often follows the same basic roadmap: an organization will review its existing IT resources, determine the cost, choose an approach, select one or more cloud providers, set up any new IT resources needed for the migration, begin a step-by-step migration, and continuously review and improve the process.

In practice, cloud adoption might look like a university deciding to first move its email services to Google's Gmail. Next, over several years, it moves its website to Google Cloud, and then moves its main student database and classroom technology to the cloud, before eventually removing all of its on-premises servers.

How does cloud adoption work?

Each organization's cloud journey is different, but they generally follow the same sequence: assessment, procurement, construction, migration, testing and optimization.

- **Assessment**

Many leading cloud providers, such as Amazon Web Services (AWS), Microsoft Azure and IBM Cloud, have their own frameworks to guide organizations through cloud adoption. Almost always, these frameworks begin with a simple but important task: determining exactly how using the cloud will help an organization achieve its business goals.

These goals might include reducing onsite infrastructure costs, reducing application latency (the delay before an application responds), or speeding up the DevOps pipeline (the process of developing, testing and delivering software). Identifying these goals—and then determining how prepared the organization is to achieve them, and what people, skills or resources it needs—is the first step in cloud adoption, before choosing a cloud model and providers.

- **Procurement**

After setting its goals and doing a thorough self-assessment, an organization is ready to evaluate different solutions and cloud providers.

There are multiple cloud architectures that organizations can choose from depending on their needs, including public cloud, private cloud, hybrid cloud and multicloud.

The public cloud is a type of hosting where a third-party provider makes cloud services available over the public internet. Its combination of low cost and flexibility makes it the most widely used form of cloud computing. It is most suitable for organizations that need to increase or decrease their cloud resources based on business needs, such as online retailers. Public cloud is also useful for organizations that have limited resources or ability to manage the cloud infrastructure themselves.

The private cloud, also known as an internal or corporate cloud, is a hosting solution where all hardware and software resources are dedicated to and accessible by one organization only. This is most useful when strong security or compliance is required, such as for organizations that handle confidential documents, intellectual property, personally identifiable information (PII), medical records or financial data.

Hybrid cloud is a cloud strategy that combines elements of public and private clouds, often together with onsite infrastructure, to provide greater flexibility. Hybrid cloud is especially important for containerization, which breaks large applications into smaller microservices that can be moved and run across different cloud environments based on business needs.

Agentic AI can also benefit from hybrid cloud. It requires fast and trusted access to data wherever that data is stored, along with strong security and governance (rules and processes for managing data and technology). Hybrid cloud infrastructure is designed to support these needs.

Multicloud environments use cloud services from more than one cloud provider at the same time. This allows organizations to avoid vendor lock-in, where they become too dependent on one provider for their services. Multicloud can also improve resilience by giving an organization access to more than one cloud environment if one experiences an

outage. Businesses typically use open-source frameworks supported by different cloud providers, such as Kubernetes, to manage these environments.

Organizations rarely use only one of these models. More often, they use a combination of these models based on their specific needs and business goals.

- **Construction**

After a cloud model has been chosen and cloud providers have been selected, the next step in cloud adoption is to set up the necessary IT resources.

These components include networking infrastructure, identity and access management (IAM) systems, cybersecurity tools, observability tools to monitor the new cloud environment, and data storage solutions. These components are often built first as a self-contained cloud “landing zone,” which can be tested to make sure everything works correctly before the actual migration begins.

Failing to build and test these resources before starting the migration can lead to higher-than-expected costs and inefficiencies, which can slow down or even prevent an organization’s cloud adoption efforts.

- **Migration**

Organizations use different tools and strategies to move their IT systems to the cloud. For example, the “7 Rs” of cloud migration, based on a framework first introduced by Gartner in 2010, include rehosting, relocating, replatforming, refactoring, repurchasing, retiring and retaining.

Rehosting, also called “lift and shift,” moves applications to the cloud without changing the application’s code or architecture, usually by using virtual machines (VMs). This approach works well when organizations need to migrate quickly, do not have enough resources to redesign applications, or want to get immediate cloud benefits such as lower data center costs.

Relocating moves workloads by transferring virtual machines directly between environments without changing the applications. This approach is commonly used when moving VMware-based workloads to cloud environments.

Replatforming makes specific improvements to applications during migration so they can take advantage of cloud capabilities without changing their main architecture. For example, structured query language (SQL) databases can be moved to a Kubernetes container, allowing them to be moved across hybrid or multicloud environments.

Refactoring means redesigning and rebuilding applications into cloud-native versions. This might involve breaking applications into containerized microservices or using serverless computing. It requires a significant investment, but it can be useful when existing IT infrastructure cannot easily scale to meet growing demand.

Repurchasing means replacing existing applications with cloud-based software as a service (SaaS) alternatives. This can make sense when the cloud-based product is better or when moving the existing application would cost more than simply adopting a new SaaS solution.

Retiring means identifying and removing applications that are unnecessary or outdated during cloud adoption. Organizations might retire applications when usage data shows that very few people use them, another system already provides the same functionality, or the cost of migrating them is greater than their business value.

Retaining means keeping applications in their current environment and migrating them later. Organizations might retain applications when they require major changes before migration, have complicated dependencies on other systems, or need to meet strict compliance requirements.

Just like choosing a cloud model, most organizations use a combination of these strategies for different parts of their IT infrastructure.

• Testing and optimization

During and after migration, organizations carefully test their cloud components to make sure they achieve the expected speed, data portability and resilience.

Common tests during migration include:

- **Functional testing** — checks that the application works correctly after migration.
- **Data integrity testing** — makes sure data has not been lost or corrupted during the migration.
- **Cutover testing** — a **practice run** of the migration steps to make sure everything works properly before the actual migration.
- **Rollback testing** — checks that administrators can return systems to their **pre-migration state** if something goes wrong.

After migration, teams perform:

- **Performance and load testing** — checks how well the new cloud system performs under different levels of demand.
- **Security testing** — identifies security weaknesses and compliance problems in the new cloud environment.
- **User acceptance testing** — allows users and other stakeholders to use the system in their normal workflows and provide feedback.

Cloud adoption vs. cloud migration

Cloud migration is the technical process of moving services to the cloud. Cloud adoption is a broader process in which an organization fully integrates the cloud into its IT infrastructure and business operations.

Cloud migration is an important part of cloud adoption, but it is only one part of the overall process. Adoption also involves identifying the business goals for moving to the cloud, making changes to support cloud services, and continuously testing and improving services to make sure the cloud is helping the organization achieve its goals.

While cloud adoption requires some form of cloud migration, organizations can make the mistake of simply moving to the cloud without preparing for the organizational changes that adoption requires. This can lead to unexpected costs, account and resource sprawl, cloud security and compliance problems, and other organizational challenges.

Benefits of cloud adoption

Benefits of cloud adoption can include a more efficient and flexible IT environment, better security and data protection, and lower infrastructure costs.

- **Modernizing IT**

Cloud solutions provide benefits such as greater speed and flexibility, almost unlimited scalability, and greater strategic value.

For example, cloud technologies can simplify the CI/CD pipeline, allowing DevOps professionals to create and set up new infrastructure when needed. The cloud also helps

organizations avoid buying extra computing capacity that remains unused during periods of low demand. Instead, resources can increase or decrease automatically based on changes in traffic.

Cloud computing also provides benefits for hosting artificial intelligence (AI) tools. For example, the specialized hardware needed to train AI models can be expensive. Cloud providers can provide access to this hardware at a lower cost than buying it directly.

- **Security advantages**

Security benefits of cloud computing include automatic security updates and improved disaster recovery.

Cloud providers generally use automation to deliver updates and patches for known security vulnerabilities in their services. Although IT and cybersecurity teams are still responsible for the security of their systems, fixes provided by the cloud provider can help reduce security gaps and human errors.

Cloud computing is also important for disaster recovery (DR), helping organizations reduce data loss and business disruptions. With cloud infrastructure, organizations can store multiple copies of important data in different locations and quickly switch from one server to another if a failure occurs. Even if part of the system goes down, important operations and services can continue running—or return online more quickly.

This cloud-based resilience helps organizations maintain business continuity during cyberattacks, equipment or supply chain failures, natural disasters, power outages, or other unexpected events.

- **Reducing infrastructure costs**

Cloud adoption can make infrastructure more cost-effective by allowing organizations to “right-size” their server usage according to their actual needs. This can reduce data center costs and free up money for other investments.

When organizations pay as they go for cloud services based on their usage, they can have more money available for other IT investments and have better control over IT costs.

During cloud adoption, organizations often discover that they are wasting money on underused onsite infrastructure during periods of low demand.

Moving to the cloud can also save money for organizations that own or lease physical data center space. Data centers require significant energy, expensive hardware upgrades and regular maintenance. Organizations can reduce or eliminate their own physical data centers and instead use their cloud provider's infrastructure, lowering these costs.

Challenges of cloud adoption

Challenges of cloud adoption include a potentially complex and time-consuming integration process, lack of necessary skills, and problems with managing costs.

- **Cloud integration**

Connecting existing systems to the cloud can be challenging, especially when it comes to networking and making older (legacy) applications work properly.

Cloud integration requires high network bandwidth to maintain the benefits of speed and data portability. This means organizations need to pay extra attention to network monitoring and maintenance. Also, many existing applications may need to be reconfigured, rebuilt, or completely replaced to work correctly in the cloud.

- **Skill gaps**

Organizations need to make sure cloud adoption is supported by the right skills, knowledge, and best practices. Experts who understand the existing (legacy) systems and cloud experts need to work together to make the process smoother.

This might require hiring new employees, retraining existing employees to work with cloud systems, consulting outside experts, or, more commonly, using a combination of these approaches.

- **Cost management**

Without a clear understanding of how to increase or decrease cloud usage based on business needs, organizations may reduce or completely lose the cost savings they expected from cloud adoption.

Cloud providers offer many different pricing models, with detailed charges based on how much and how often services are used. Organizations therefore need to carefully analyze which pricing model best fits their business needs.

This analysis requires a complete understanding of how the organization uses the cloud, which makes observability metrics and practices especially important.

Organizations must also be careful to avoid vendor lock-in, where money and resources invested to make systems compatible with one cloud provider become difficult or expensive to reuse if the organization later chooses a different provider (or adds another provider in a multicloud environment).

Cloud Computing Architecture

Before starting, it's important not to confuse "architecture" with two other common cloud topics:

- **Services (SaaS, PaaS, IaaS)** – these are *types of services* the cloud offers, chosen based on requirement. Not architecture.
- **Deployment models (Private, Public, Hybrid cloud)** – this is *how* the cloud is deployed/used. Not architecture.
- **Architecture** – this is about *how the cloud provider actually implements things behind the scenes*: how resources are physically kept, installed, and configured.

To understand this, we look at how a top provider like **Microsoft Azure** structures its global infrastructure (AWS and GCP follow a very similar model, just with different naming).

Every cloud architecture exists to manage three core resource types:

- **Storage** – to store data properly
- **Networking** – to provide proper connectivity
- **Compute** – to process data

Cloud providers organize their global infrastructure as a **hierarchy**, from largest to smallest:

Level	What it Means	Example
-------	---------------	---------

Geography	Broadest boundary; grouped mainly for legal/data-residency/compliance reasons (data of a country must stay within that country)	Asia, Europe, United States
Region	A specific geographical area where a cluster of data centers is deployed, connected by a low-latency network	India, China (within Asia)
Availability Zone	A physically separate location (one or more data centers) within a region, each with independent power, cooling, and networking	Zone 1 in Delhi, Zone 2 in Mumbai
Data Center	The actual physical building full of servers and hardware	A specific building in Delhi or Beijing
Resources	Storage, compute, and networking components installed inside the data center	VMs, databases, storage disks

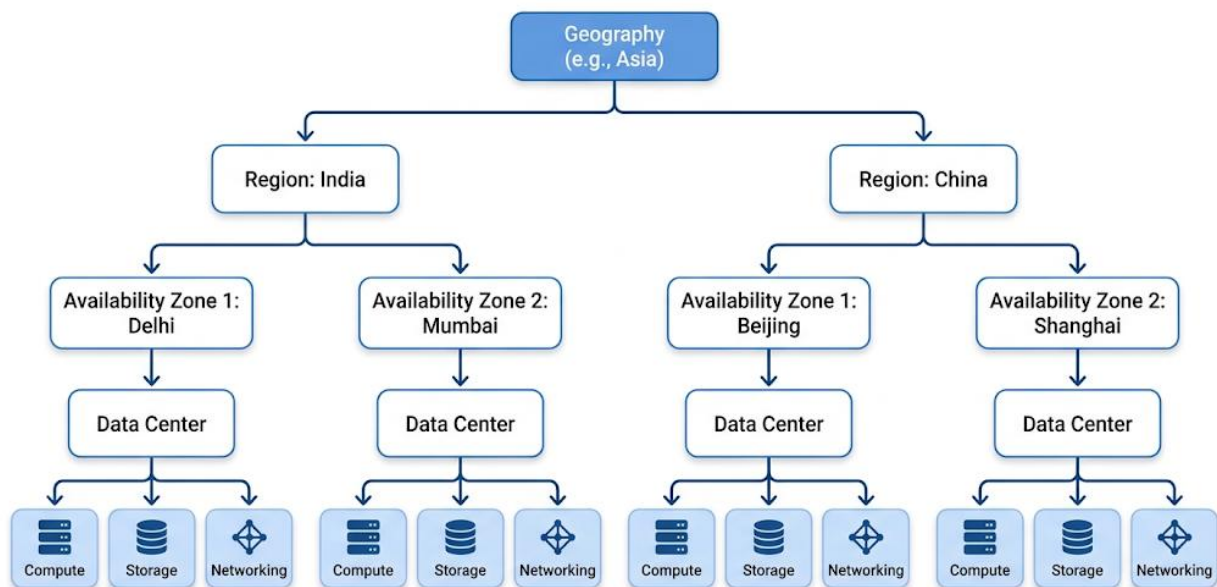


Figure: Cloud Infrastructure Hierarchy

Note: A region typically has at least 2–3 availability zones, and each availability zone contains one or more data centers. Not every region in the world supports availability zones — smaller regions may have just one data center.

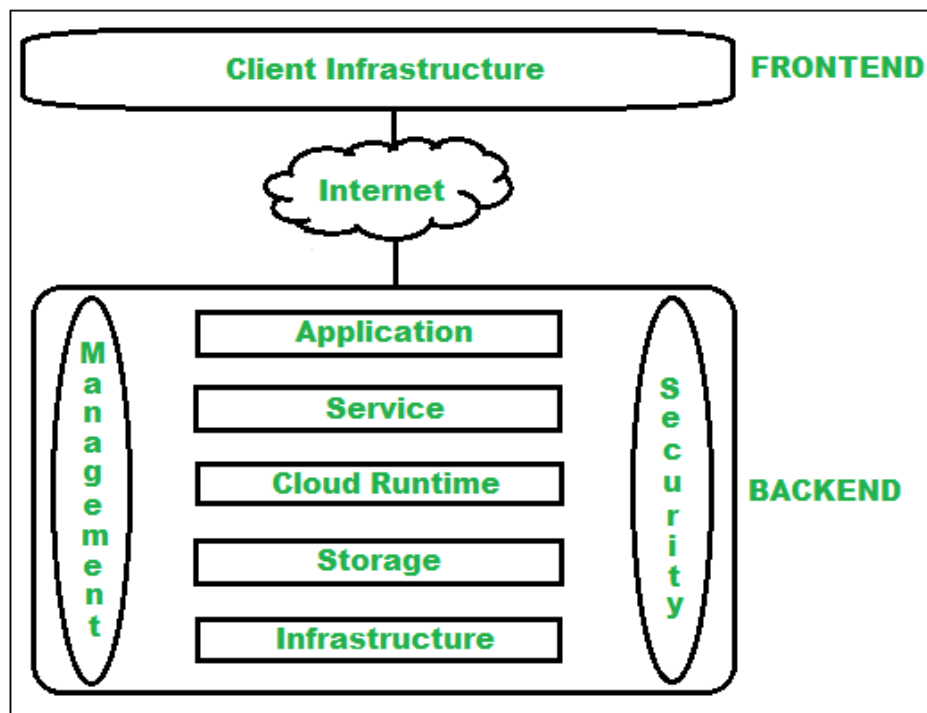


Figure: Cloud computing Architecture

Why a Hierarchical Architecture?

1. Performance

- A user's request is routed to the *nearest* data center.
- This reduces latency (delay), which improves overall performance.

2. Scalability

- If a company (e.g., an e-commerce business) needs more resources suddenly — like during a Black Friday or New Year sale — it can scale from using 1–2 instances (virtual machines) to 4–5 or more instances easily.
- Multiple data centers make this kind of scaling possible without any manual hardware setup.

3. Fault Tolerance

- If a fault/problem occurs in one data center, the user's request is automatically shifted to another data center.
- This keeps availability at 24x7, even during localized failures.

4. Disaster Recovery

- If an entire data center is destroyed (flood, earthquake, fire, etc.), operations shift to another data center — the company doesn't shut down.
- If an entire availability zone fails, another zone takes over.
- If an entire region fails, requests can be redirected to another region entirely (e.g., Asia to Europe).
- This is why Azure also uses region pairs — each region is paired with another region (usually 300+ miles away) in the same geography, so if one region is down, its pair can take over.

Failure Level	Backup Handled By
Data Center fails	Another Data Center (same zone)
Availability Zone fails	Another Availability Zone (same region)
Region fails	Paired/another Region

Subscription and Resource Access

- Users don't own physical resources directly — they take a subscription (similar to subscribing to Netflix/OTT) and use resources through it.

- This creates an abstraction layer: the user doesn't need to know (or care) whether their request is being processed in Delhi, Mumbai, or Beijing — they only care that their data/service is always available.

This hierarchical model isn't unique to Azure — AWS and Google Cloud follow the same underlying idea, just with different terminology:

Concept	Azure	AWS	Google Cloud
Broadest area	Geography	Partition	Multi-region
Main deployment area	Region	Region	Region
Isolated zone within region	Availability Zone	Availability Zone (AZ)	Zone
Physical building	Data Center	Data Center	Data Center

Concept	Explanation	Why it Matters
Region	Large geographical deployment area with multiple data centers	Base unit for choosing where infra is hosted
Availability Zone	Physically isolated data center(s) within a region	Protects against localized failures
Data Center	Physical building with servers	Houses actual hardware resources
Performance	Nearest data center reduces latency	Faster response for users
Scalability	Add more instances/data centers as demand grows	Handles traffic spikes (e.g., sales)
Fault Tolerance	Shifts requests to healthy data center/zone	Keeps service running during small failures

Disaster Recovery	Shifts to another zone/region entirely	Keeps service running during major disasters
Subscription	User's access point to resources	Creates abstraction — user doesn't manage backend

CLOUD SERVICE & DEPLOYMENT MODELS

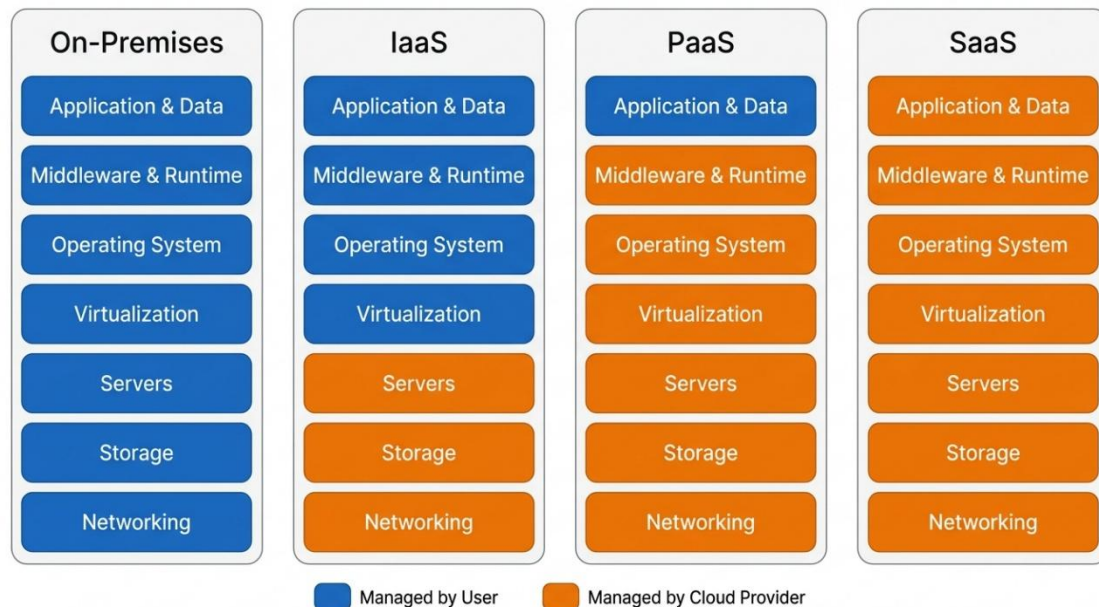
Cloud Service Models

Cloud computing offers three major delivery models — how much a user manages versus how much the cloud provider manages. The choice depends on your requirement, technical ability, and how much control you want.

- **SaaS** – Software as a Service
- **PaaS** – Platform as a Service
- **IaaS** – Infrastructure as a Service

All major providers (AWS, Azure, Google Cloud) offer all three; you simply pick the one that matches your needs.

To understand the difference between SaaS, PaaS, and IaaS, it helps to see the full stack of components any application needs, from top to bottom:



Layer	What it Does
Application & Data	The actual app users interact with, and the data it generates/stores
Runtime	Environment to execute code (e.g., JVM for Java, .NET Framework, Swift)
Middleware	The "bridge" connecting frontend, backend, database, payment gateways, APIs, etc.
Operating System	Manages resources for applications
Virtualization	Creates virtual machines instead of giving physical hardware directly (hypervisor-based)
Servers	Provide actual computation/processing power
Storage	Hard disks and storage systems for data, backup, and recovery
Networking	Ensures devices/packets communicate and transfer data seamlessly

The only real difference between SaaS, PaaS, and IaaS is where you draw the line — i.e., which of these layers *you* manage versus which the cloud provider manages.

Before comparing the three models, it helps to understand the baseline: *on-premises*, where a company manages the *entire* stack itself — application, runtime, middleware, OS, virtualization, servers, storage, and networking — all bought, installed, and maintained in-house.

- Most expensive option (you buy and manage every device yourself)
- Most control, but highest cost and effort

SaaS – Software as a Service

The cloud provider manages everything — application, data, runtime, middleware, OS, virtualization, servers, storage, and networking. The user simply uses the ready-made software.

- User only logs in and uses the application; no technical knowledge needed
- Provider handles development, hosting, updates, security, and infrastructure
- Billing is usually subscription or pay-as-you-go
- **Examples:** Google Docs, Google Workspace, Netflix, Dropbox, Salesforce, Zomato (the underlying platform), Microsoft 365

Advantages: cost-effective (pay only for use), quick deployment, accessible anywhere, automatic updates, scalable.

Disadvantages: limited customization, needs constant internet, data control is reduced, some security risk since a third party holds your data.

PaaS – Platform as a Service

The user manages the application and data (they know how to code — e.g., PHP, .NET). The cloud provider manages everything below that: runtime, middleware, OS, virtualization, servers, storage, and networking.

- You bring your own code/recipe; the provider gives you the platform (compiler, OS, infrastructure) to run it on
- **Examples:** Google App Engine, Microsoft Azure App Service, AWS Elastic Beanstalk, Heroku

Advantages: no infrastructure management, cost-effective, supports the full app lifecycle (build, test, deploy, update), faster development.

Disadvantages: limited infrastructure control, dependent on the vendor's platform, not flexible for every type of application.

IaaS – Infrastructure as a Service

The user manages almost everything above virtualization — application, data, runtime, middleware, and OS. The cloud provider only provides the base infrastructure: virtualization, servers, storage, and networking.

- You've built the whole application yourself and even chosen your own OS — you just need the raw hardware/infrastructure to run and host it on
- **Examples:** AWS EC2, Microsoft Azure Virtual Machines (VMs)

Advantages: cost-effective (no physical hardware purchase), flexible hosting, reduced maintenance (provider manages data centers), highly scalable.

Disadvantages: user is responsible for securing their own apps/data, no control over physical infrastructure, availability may vary by region.

Table: Who manages what

Layer	On-Premises	SaaS	PaaS	IaaS
Application & Data	User	Provider	User	User
Runtime, Middleware, OS	User	Provider	Provider	User
Virtualization	User	Provider	Provider	Provider
Servers, Storage, Networking	User	Provider	Provider	Provider

Example table: Real-Life Analogy: Ordering Pizza

Model	What You Do	What Provider Does	Cost
On-Premises	Buy your own kitchen, oven, ingredients, and cook everything yourself at home	Nothing	Highest
IaaS	Bring your own base, recipe, and toppings; you're the chef	Provides the oven, electricity, basic kitchen counter (rented kitchen)	Low
PaaS	Bring your own ingredients/recipe only	Provides the kitchen + chef to cook it for you	Medium
SaaS	Just order and eat	Makes, packages, and delivers the pizza entirely	Highest (among cloud models)

This mirrors the real stack: in IaaS you do the most work yourself (just renting hardware); in SaaS you do almost nothing (just consuming a finished product).

Beyond SaaS/PaaS/IaaS: XaaS and FaaS

Modern cloud computing extends this model further:

- **XaaS (Anything/Everything as a Service):** A broad umbrella term where virtually any IT capability — infrastructure, platform, software, storage, security, networking — is delivered as a service. In practice, a company might combine IaaS (AWS EC2) + SaaS (Google Workspace) + PaaS (Firebase) together — that combined usage is XaaS.
- **FaaS (Function as a Service):** Lets developers run small pieces of code ("functions") only when triggered by an event, without managing any server at all.

Provider handles scaling and resource allocation automatically; billing is based on actual execution time.

- Example: uploading an image triggers a function that automatically resizes it, then stops running.
- Downsides: possible "cold start" delay when a function runs after being idle, and it's not suited for long-running tasks.

Choosing the Right Model

- Know nothing technical, want it fully done for you → SaaS
- Know how to code, need a ready platform to build on → PaaS
- Know how to code and manage infrastructure, just need raw resources (like an IT firm with in-house developers) → IaaS

Table: Summary

Service	User Controls	Provider Controls	Real-Life Example	Cloud Example
SaaS	Nothing (just uses the app)	Everything	Ordering a full pizza	Google Docs, Netflix
PaaS	Application & data (code)	Runtime, middleware, OS, virtualization, servers, storage, network	Using a rented kitchen with a chef	Azure App Service, Google App Engine
IaaS	App, data, runtime, middleware, OS	Virtualization, servers, storage, network	Renting a kitchen, cooking yourself	AWS EC2, Azure VMs

CLOUD DEPLOYMENT MODELS

Public Cloud

Public cloud is one of the three major cloud deployment models, alongside private cloud and hybrid cloud. The basic idea is simple: instead of an organization buying and maintaining all the physical computing infrastructure it needs, a third-party cloud provider builds and operates large-scale infrastructure and makes computing resources and services available to many customers over the Internet.

Providers such as (AWS), , and are examples of public cloud providers.

The important thing to understand is that public does not mean that everyone's data is publicly visible. "Public" refers to the fact that the cloud infrastructure and services are offered by a provider to the general market and can be used by many different customers. Each customer's resources and data are logically separated and protected from other customers. A simple way to think about public cloud is:

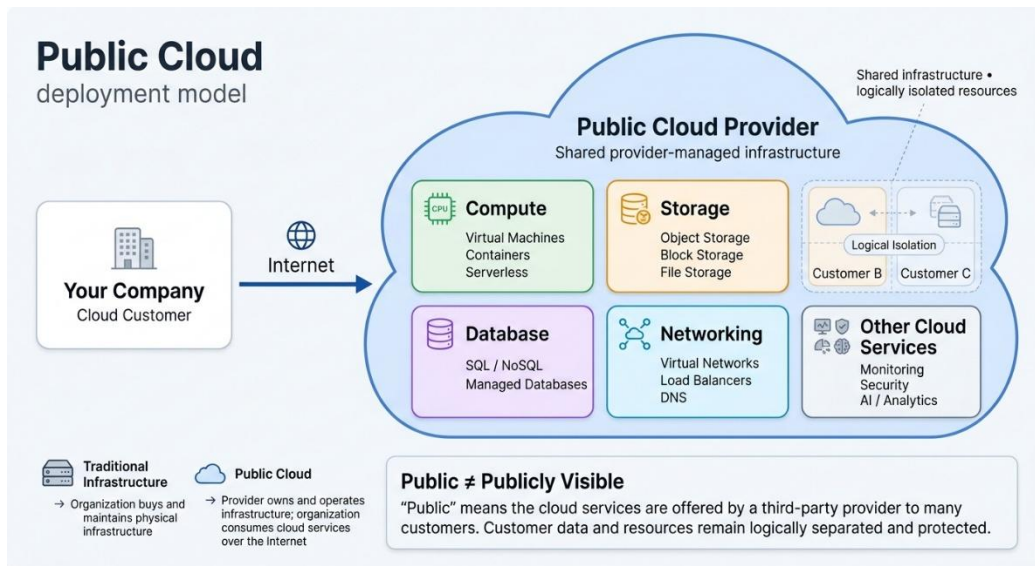


Figure: Public Cloud

You use the provider's infrastructure instead of building the entire infrastructure yourself.

The Starting Point: Why Public Cloud Exists

Before understanding public cloud, imagine a company that wants to run an online shopping website.

Traditionally, the company would need to purchase physical servers, storage devices, networking equipment, backup systems, and other infrastructure. It would also need a place to keep the hardware and would be responsible for electricity, cooling, physical security, hardware failures, upgrades, and maintenance.

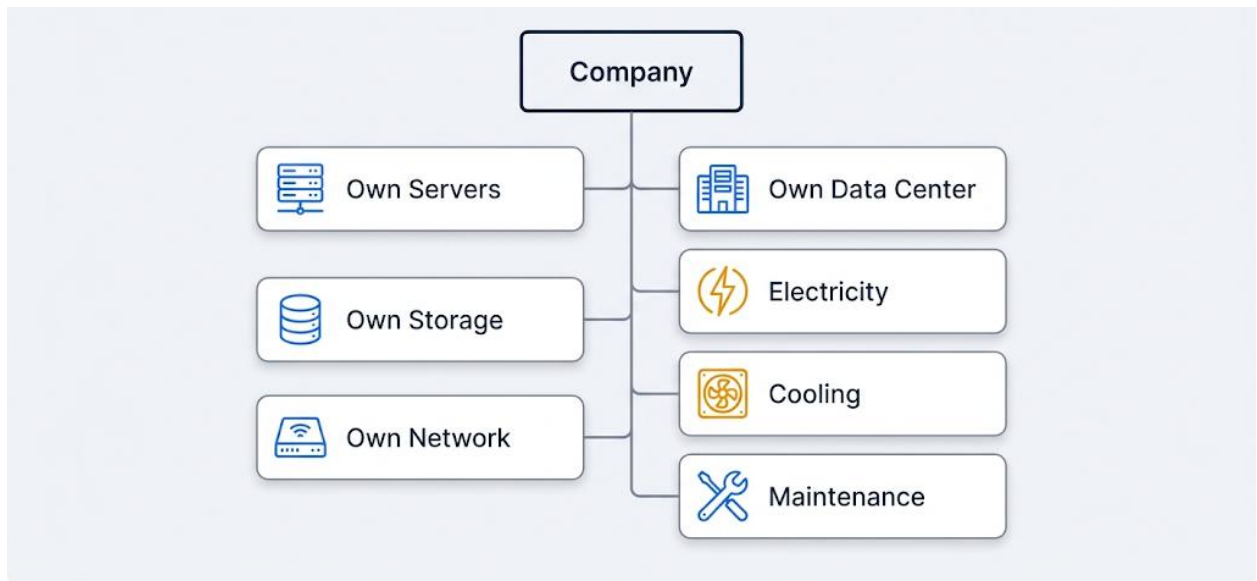
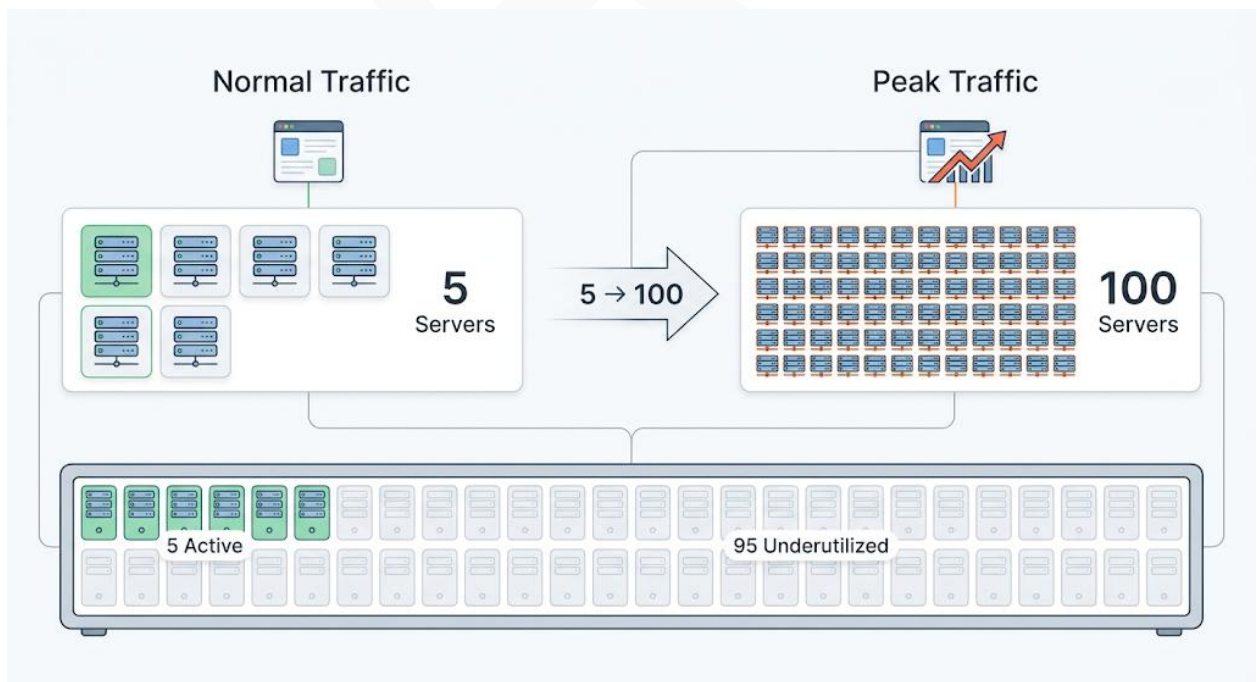


Figure: Traditional IT Infrastructure

This creates two major problems.

First, large upfront investment is required. The company may have to spend a huge amount of money before its business even starts.

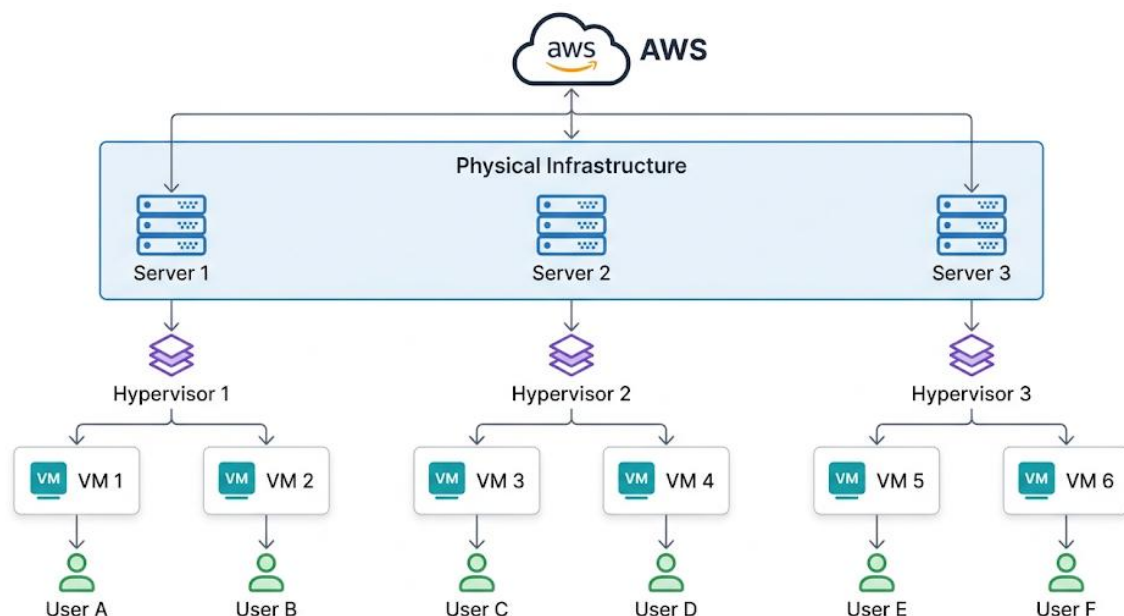
Second, the amount of infrastructure required is difficult to predict. A website might normally need only 5 servers but suddenly need 100 servers during a major sale.



Buying 100 physical servers just to handle occasional peak traffic is inefficient. Most of those servers would remain underutilized during normal periods. Public cloud changes this model. Instead of buying the infrastructure, the company can consume computing resources as a service. The company can increase resources when demand increases and reduce them when demand falls. This ability to grow and shrink resources according to workload is one of the most important characteristics of cloud computing.

Public Cloud Infrastructure: Who Actually Owns the Hardware?

This is where public cloud differs fundamentally from traditional infrastructure. When you deploy a website on AWS, you might create a virtual machine and think of it as "your server." From your application's point of view, it behaves like a server that belongs to you. However, you do not own the physical server underneath it. AWS owns and operates the physical infrastructure.



Customer A might be using VM1, Customer B might be using VM2, and Customer C might be using VM3. The physical infrastructure belongs to the cloud provider, while customers consume virtualized or managed resources from that infrastructure. Deploying your

website on AWS is an example of using the public cloud. You are using someone else's cloud infrastructure, rather than building and operating all the physical infrastructure yourself.

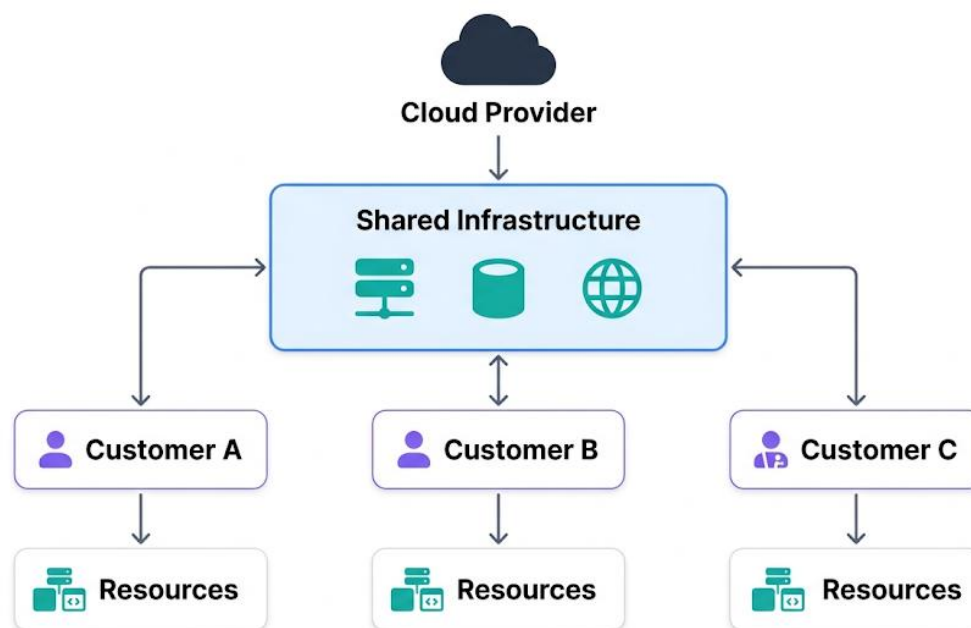
Public Cloud and Multitenancy

One of the most important characteristics of a public cloud is multitenancy. The word can be understood by splitting it into two parts:

Multi = many

Tenant = customer/user

Therefore, multitenancy means that a cloud provider's infrastructure serves many different customers. Imagine an apartment building. The building belongs to one owner, but many tenants live inside it. Each tenant has their own private apartment even though they share the same overall building. Public cloud works in a similar way:



Customers may share physical infrastructure, but their resources must be logically isolated. Customer A should not be able to access Customer B's data simply because both customers are using the same physical infrastructure. This separation is achieved through technologies and security mechanisms such as virtualization, identity and

access controls, network isolation, encryption, and security policies. This is why the following two statements are both true:

- Public cloud infrastructure is shared.
- Customers' environments and data are isolated.

There is no contradiction between them.

Public Cloud Is More Than Just "Renting a Server" A common beginner's understanding is: "Public cloud means renting someone else's server." That is a good starting point, but it is incomplete. A public cloud can provide an entire range of services. At the infrastructure level, you can rent computing resources such as virtual machines, storage, and networks. At a higher level, the provider can manage the operating system and application platform for you. At an even higher level, the provider can give you a complete application that you simply use. This is where IaaS, PaaS, and SaaS come into the picture.

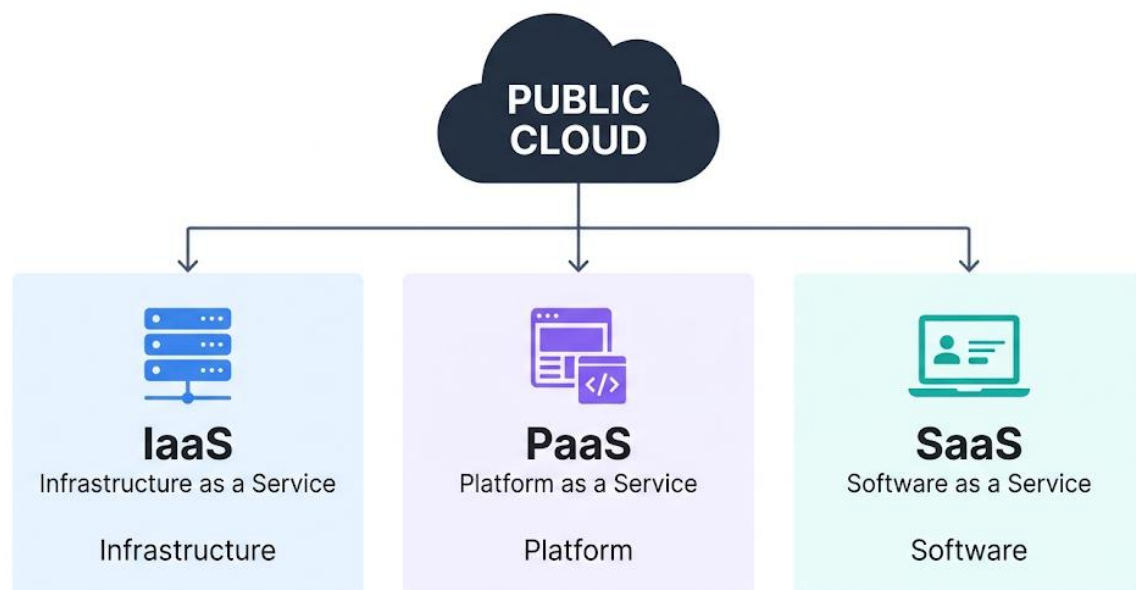


Figure: Service Models

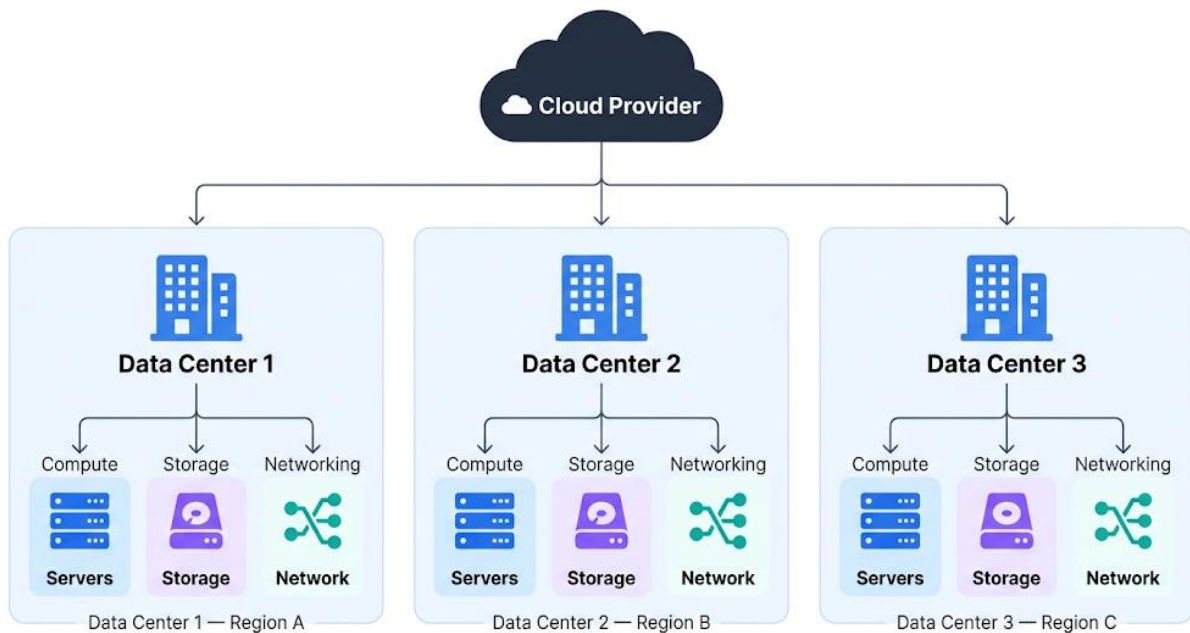
Deployment Models	Service Models
Public Cloud	IaaS
Shared provider infrastructure	Infrastructure as a Service
Private Cloud	PaaS
Dedicated organization infrastructure	Platform as a Service
Hybrid Cloud	SaaS
Combination of public + private	Software as a Service

Figure: Service and Deployment models

The deployment model describes how/where the cloud environment is provided, while the service model describes how much of the technology stack the provider manages for you.

Public Cloud as a Distributed System

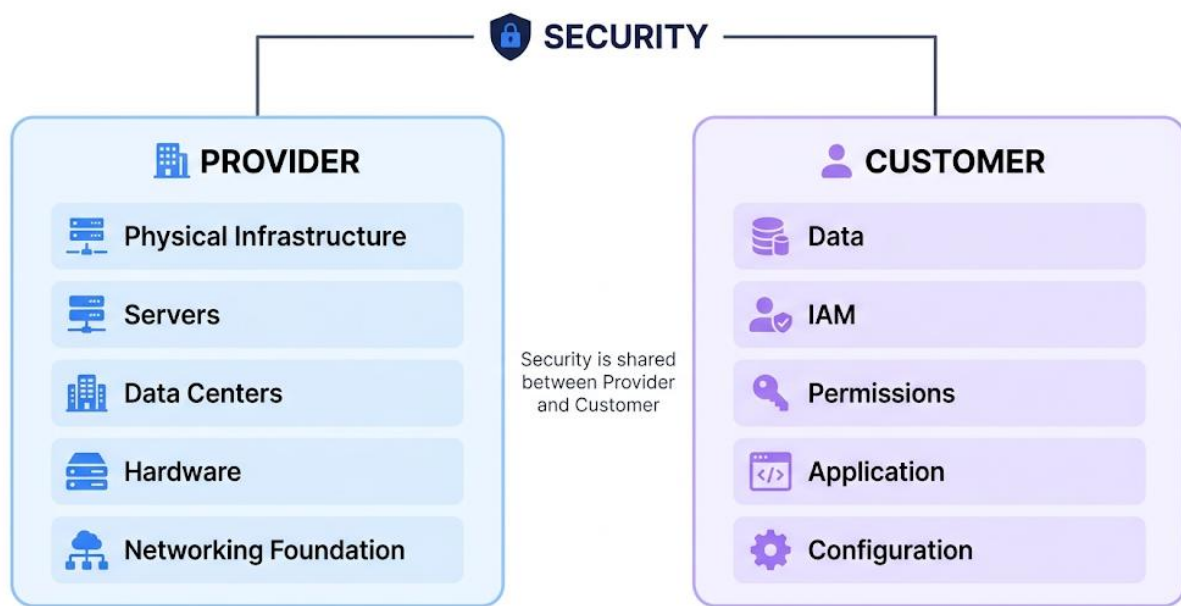
A public cloud is not one giant computer sitting in one location. From an architectural point of view, a public cloud is a distributed system. A distributed system is a system whose components are spread across multiple computers or physical locations and work together through a network. A large public cloud provider may have:



These data centers are connected by high-speed networks and supported by software that manages computing, storage, networking, security, monitoring, billing, and other functions. This distributed architecture is one of the reasons public clouds can support enormous numbers of customers.

Shared Responsibility:

A common misconception is: "If I use AWS, AWS handles all my security." That is incorrect. Security responsibilities are divided between the cloud provider and the customer. At a simplified level:



The provider is responsible for protecting the underlying infrastructure — often described as security of the cloud. The customer is responsible for protecting and correctly configuring what they deploy in the cloud — often described as security in the cloud. For example, the provider may secure the physical server, but if you give an unauthorized person access to your cloud account, that is your responsibility. Similarly, if you configure a storage resource incorrectly and expose sensitive data, the cloud provider's physical security does not automatically protect you from your own configuration mistake.

Advantages of Public Cloud

Public cloud is attractive because it provides capabilities that are difficult or expensive to achieve with traditional infrastructure.

- **Reduced infrastructure management:** The provider handles the physical servers, data centers, power, cooling, and much of the underlying infrastructure.
- **Scalability and elasticity:** Resources can be increased or decreased as workloads change.
- **Lower upfront investment:** Organizations can avoid purchasing large amounts of physical infrastructure before they know what they need.
- **Global reach:** Cloud providers operate infrastructure across multiple geographic locations, allowing applications to serve users in different parts of the world.

- High availability possibilities: Multiple regions, Availability Zones, redundancy mechanisms, and managed services can be used to build resilient applications.
- Rapid provisioning: Instead of waiting days or weeks to purchase and install physical servers, cloud resources can often be provisioned much more quickly.

Limitations and Challenges

- Vendor lock-in: If an application becomes heavily dependent on provider-specific services, moving it to another provider can become difficult and expensive.
- Internet/network dependency: Cloud resources are accessed through networks, so connectivity is an important dependency.
- Security configuration remains your responsibility: The provider can secure its infrastructure, but customers can still create security problems through weak passwords, excessive permissions, exposed resources, or incorrect configurations.
- Regulatory requirements: Some organizations and industries have strict requirements regarding where data can be stored and processed. Public cloud can still be used in regulated environments, but the architecture and provider services must satisfy the applicable requirements.
- Cost management: Cloud's flexibility can become expensive if resources are left running unnecessarily or are poorly configured.



Figure: Public Cloud Architecture and Shared Responsibility Model

In simple terms, a **public cloud** is a cloud environment operated by a third-party provider such as AWS, Azure, or Google Cloud. The provider owns and manages the physical infrastructure—such as servers, storage, networking equipment, and data centers—and makes these resources available to many customers over the Internet. The infrastructure is distributed across different **regions and Availability Zones** to support reliability and global access. Technologies such as **virtualization** allow the provider to divide physical resources into virtual resources, while **multitenancy** allows many customers to use the same underlying infrastructure while keeping their data and environments logically isolated from one another. Customers can consume different types of services, such as

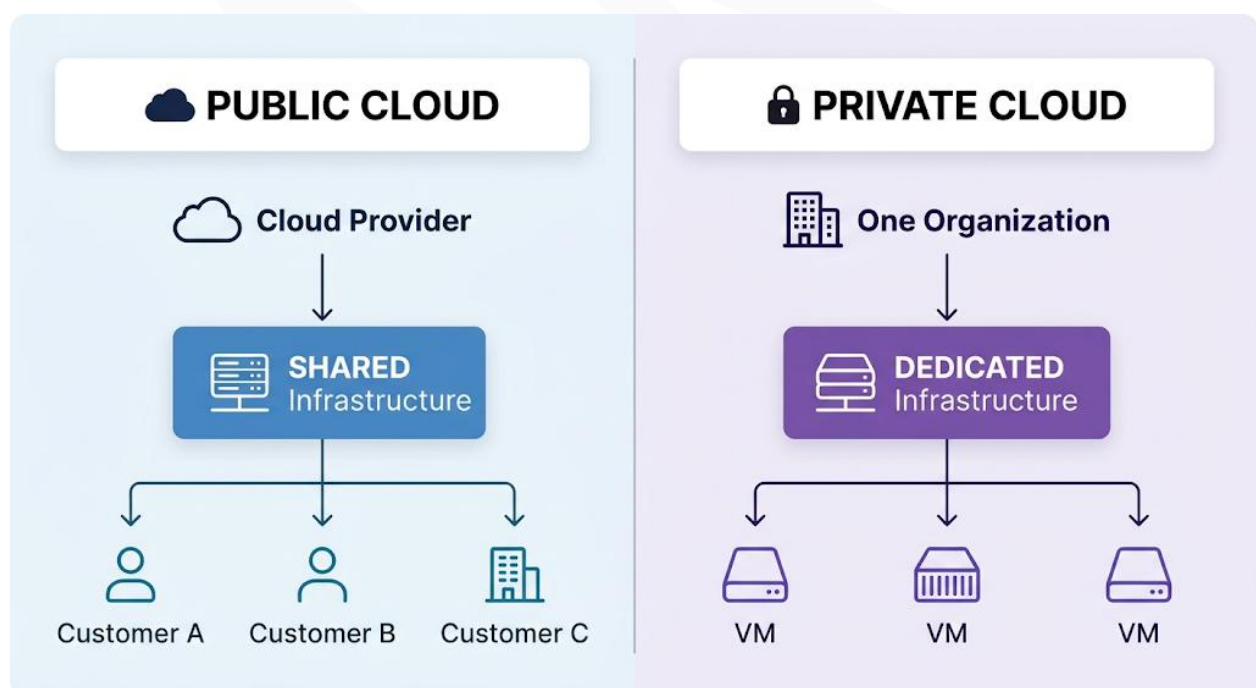
IaaS, PaaS, and SaaS, and can increase or decrease resources according to their workload. Security also follows a **shared responsibility model**, where the provider secures the underlying cloud infrastructure while the customer is responsible for securing and correctly configuring their own data, applications, identities, and resources.



Private Cloud

Public cloud is attractive because organizations can rent computing resources from providers such as AWS, Azure, or Google Cloud without purchasing and maintaining the physical infrastructure themselves. However, public cloud is not suitable for every organization or every workload. Some organizations have sensitive data, strict regulatory requirements, existing infrastructure, or a strong need for control over where and how their computing resources are operated. This is where the private cloud model becomes useful.

A private cloud follows the same general idea of cloud computing—resources are provided dynamically and can be requested when needed—but *the infrastructure is dedicated to one organization rather than being a public, multi-customer cloud environment.*



In a public cloud, the provider owns and operates the infrastructure and makes it available to many customers. In a private cloud, the infrastructure is dedicated to a single organization and is usually located within the organization's own premises or in infrastructure reserved exclusively for that organization.

Need Private Cloud

The main reason for using a private cloud is **control**. Imagine a bank, government organization, military organization, or large enterprise that has highly sensitive information. Such an organization may not want its critical workloads to run on a shared public-cloud environment, even though public-cloud providers offer strong security mechanisms.

The concern is not necessarily that "AWS is insecure." The concern is that the organization may have requirements such as:

- where its data must physically reside,
- who can access the infrastructure,
- which security procedures must be followed,
- which compliance standards must be satisfied,
- how the infrastructure must be monitored,
- and how much direct control the organization needs over the environment.

For example, suppose a bank has already invested heavily in its own data center:

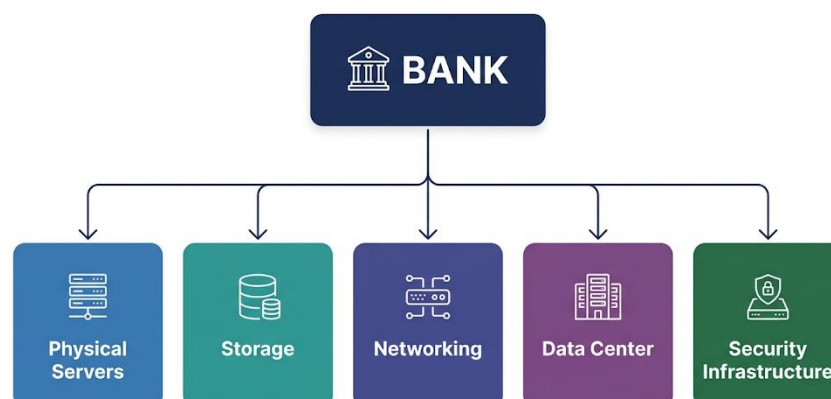


Figure: Bank-Owned IT Infrastructure

It would not necessarily make sense to throw everything away and move all workloads to a public cloud. Instead, the bank could turn its existing infrastructure into a cloud-like

environment using virtualization, automation, resource management, and self-service provisioning.

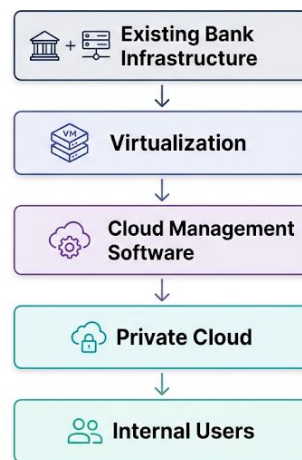


Figure: From Existing Bank Infrastructure to a Private Cloud

Note: Private cloud does not necessarily mean buying new servers. It can mean transforming an organization's existing infrastructure into an on-demand cloud environment.

What Makes It a "Cloud"?

This distinction is extremely important. Suppose a bank owns 500 physical servers. Does that automatically mean the bank has a private cloud? No. It may simply have an on-premises data center.

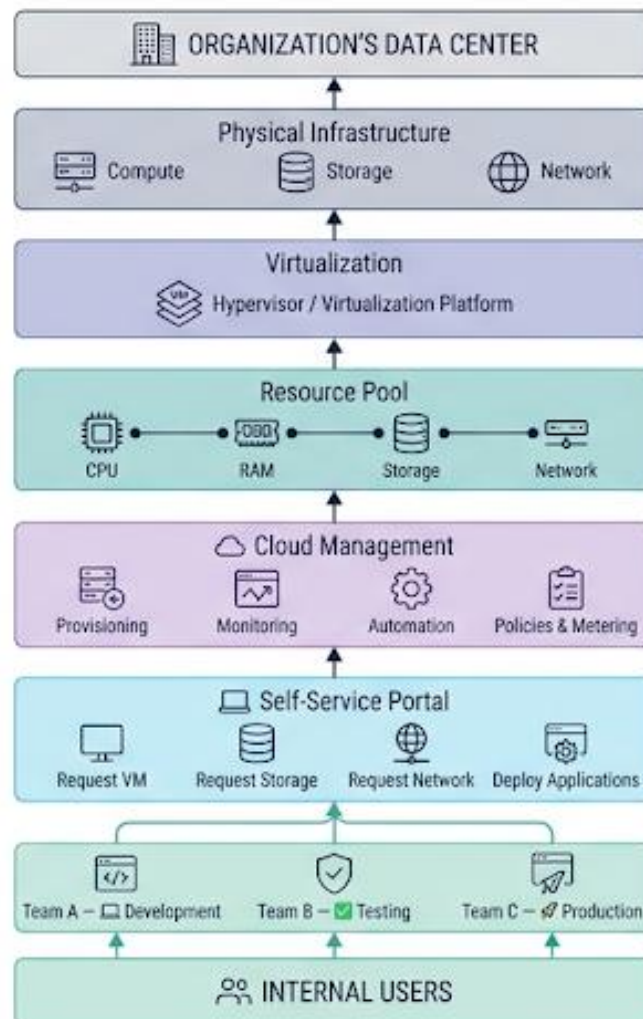
For example: Bank Data Center -> 500 Physical Servers -> Applications running directly That is traditional IT infrastructure. Now suppose the bank introduces virtualization and cloud-management software: Bank Data Center, Physical Servers, Virtualization, Resource Pool, Cloud Management, Self-Service Provisioning, Private Cloud Now employees or internal departments can request resources dynamically.

Owning servers = infrastructure.

Owning servers + cloud technologies and cloud management capabilities = potentially a private cloud.

PRIVATE CLOUD ARCHITECTURE

From physical infrastructure to self-service cloud resources



The logic of the diagram is that a private cloud takes an organization's physical infrastructure and transforms it into a flexible, centrally managed environment of virtualized and pooled resources that authorized internal users can consume on demand. At the foundation is the **Physical Infrastructure**, which includes compute resources such as physical servers, storage systems such as SAN/NAS, and networking equipment such as switches and routers. On top of this hardware is the **Virtualization and Resource Abstraction layer**, where technologies such as hypervisors abstract the underlying hardware and allow multiple virtual machines and other virtual resources to run on the available infrastructure. The resulting compute, memory, storage, and network capacity can then be organized into **resource pools**, allowing resources to be allocated

dynamically rather than being permanently tied to a particular physical server or department. Above this is the **Cloud Management layer**, which provides centralized control over resource provisioning, monitoring, automation, policies, access control, and, where required, usage metering. It manages the available resources according to the organization's rules and policies and coordinates how resources are allocated to workloads. On top of cloud management is the **Self-Service layer**, which may be provided through a web portal, API, CLI, or automation tools. It allows authorized internal users to request resources such as virtual machines, storage, networks, or application environments without manually configuring the underlying physical infrastructure. At the top are the **Internal Users**, such as development, testing, and production teams. These are not sequential infrastructure layers; they are independent consumers of the private cloud that request and use resources according to their needs. Therefore, the architecture can be understood as **Organization's Infrastructure → Virtualization and Resource Abstraction → Resource Pool → Cloud Management → Self-Service → Internal Users**. The important idea is that users generally do not need to know or select which physical server will run their workload. Instead, they request the resources they need, and the cloud-management system provisions those resources from available capacity according to defined policies and constraints. This combination of resource abstraction, resource pooling, centralized management, automation, and self-service provisioning allows an organization's infrastructure to operate as a private cloud rather than simply as a collection of physical or virtual servers.

Private cloud = cloud infrastructure dedicated to a single organization.

Public Cloud vs Private Cloud

Aspect	Public Cloud		Private Cloud
Primary users	Many customers	independent	One organization
Infrastructure	Provider's infrastructure	shared	Dedicated to one organization

Ownership	Usually cloud provider	Organization or dedicated provider
Multitenancy	Multiple customers	Primarily one organization
Control	Less direct control	Greater control
Scalability	Very high	Limited by organization's infrastructure
Upfront investment	Low for customer	Usually higher
Infrastructure management	Mostly provider	Organization/provider depending on setup
Customization	More standardized	More customizable
Typical motivation	Flexibility, scalability, low upfront cost	Control, compliance, existing

Note: Private cloud can reduce certain external exposure and give an organization greater control over security, but it does not automatically make the environment secure. A private cloud still has to deal with: Unauthorized users, Malware, Misconfiguration, Insider threats, Network attacks, Weak authentication, Data loss, Hardware failure, Disaster recovery

Hybrid Cloud

A hybrid cloud combines a private cloud with one or more public clouds, allowing an organization to use both environments as part of the same overall computing system. The main reason for using a hybrid cloud is that public and private clouds have different strengths. A private cloud provides greater control over sensitive data and allows an organization to continue using its existing infrastructure, but its capacity is limited by the resources the organization owns. A public cloud, on the other hand, provides access to a much larger pool of resources and can scale quickly when additional capacity is required. A hybrid cloud attempts to combine these advantages: keep workloads that require greater control within the private environment, while using public-cloud resources when additional capacity or flexibility is needed. The basic idea can be visualized as:

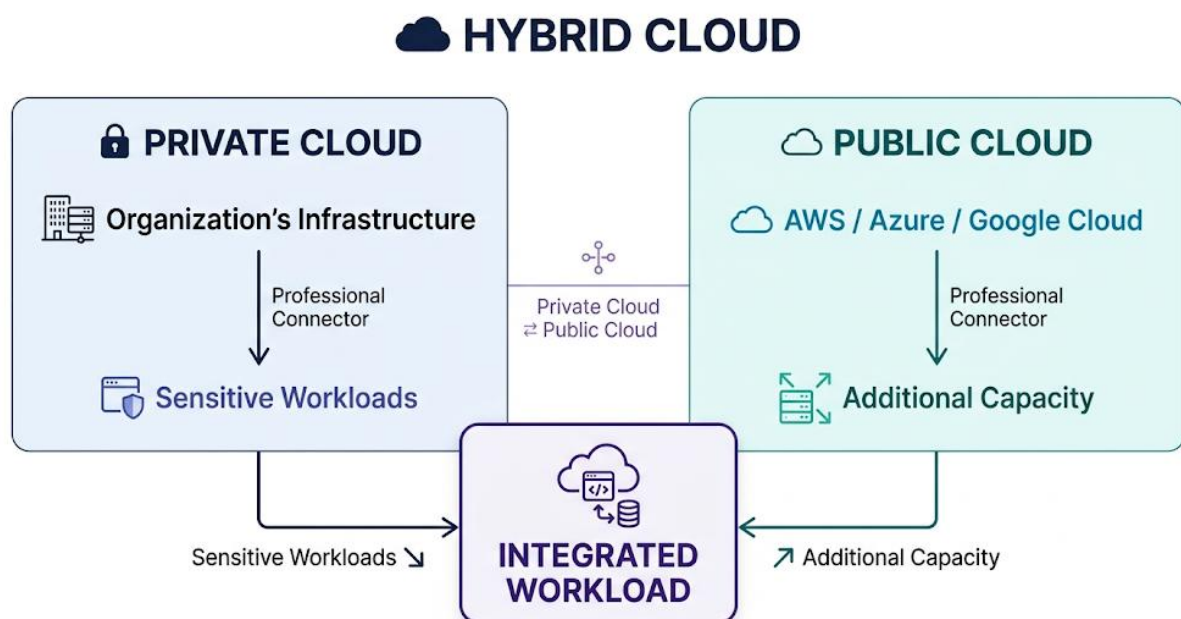


Figure: Hybrid Cloud — Integrating Private and Public Cloud Environments

The important point is that the two environments are not simply two completely separate clouds. They are connected and managed in a way that allows workloads or resources to use the appropriate environment when required.

Cloud bursting

Cloud bursting is a hybrid-cloud technique in which an organization normally runs its workloads in a private cloud, but temporarily uses additional public-cloud resources when demand exceeds the capacity of the private cloud. **Cloud bursting means that when the private cloud does not have enough capacity to handle a temporary increase in workload, additional resources are obtained from a public cloud.** When the extra capacity is no longer needed, those resources are released.

Think of it like this:

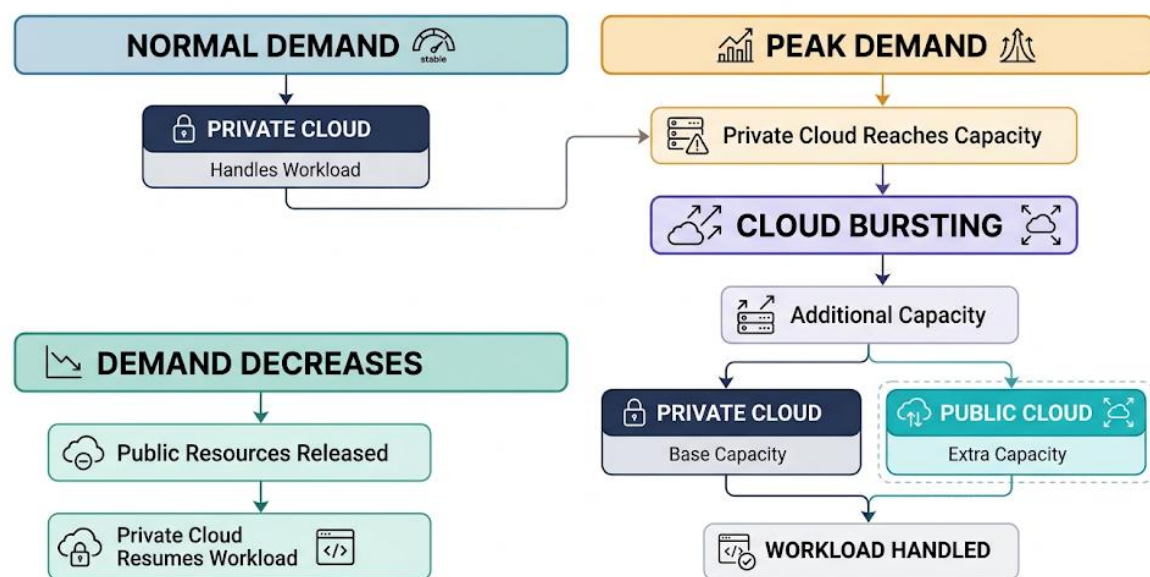


Figure: Cloud Bursting Lifecycle

A typical organization may have enough capacity in its **private cloud** during normal operation. However, workloads can sometimes increase suddenly because of events such as a major sale, examination period, financial reporting, seasonal traffic, or a sudden increase in website usage. When the private cloud reaches its available capacity, the organization can temporarily extend its computing capacity into a **public cloud**. This process is called **cloud bursting**. The word “burst” refers to the temporary spike or sudden increase in workload demand. During normal operation, the private cloud handles the workload using its existing capacity. When demand exceeds that capacity, additional workload or workload components can temporarily run in the public cloud. For example, suppose a company normally needs **20 virtual machines (VMs)**. During normal

operation, the **Private Cloud → 20 VMs**. If a sudden traffic spike increases the requirement to **50 VMs**, the private cloud can continue running its normal 20 VMs while the public cloud temporarily provides the additional 30 VMs: **Private Cloud → 20 VMs + Public Cloud → 30 VMs = Total 50 VMs**. When the traffic returns to normal, the organization can release the temporary public-cloud resources, resulting in **Private Cloud → 20 VMs + Public Cloud → 0 VMs**. Therefore, the organization does not need to permanently purchase and maintain enough physical infrastructure to handle an occasional peak. Instead, it can maintain capacity for normal workloads in its private cloud and temporarily use public-cloud capacity when demand exceeds that capacity. Cloud bursting is therefore an important capability in a **hybrid-cloud environment**, allowing an organization to handle temporary workload spikes without permanently overprovisioning its private infrastructure.

Dynamic Provisioning

Dynamic provisioning means automatically obtaining and releasing computing resources based on current workload demand. In cloud bursting, the basic process is:

Monitor workload → Demand increases → Private capacity becomes insufficient → Provision public-cloud resources → Run additional workload → Demand decreases → Release public-cloud resources.

The system must decide when to add resources, how many resources are needed, where to place the workload, and when to release the extra resources. These decisions can consider factors such as available private capacity, workload requirements, performance, cost, resource availability, and organizational policies. Therefore, cloud bursting is not simply “use the public cloud when the private cloud is full.” It requires automated resource allocation and provisioning to efficiently use private and public cloud resources.

A hybrid cloud allows an organization to choose the appropriate environment for different workloads.

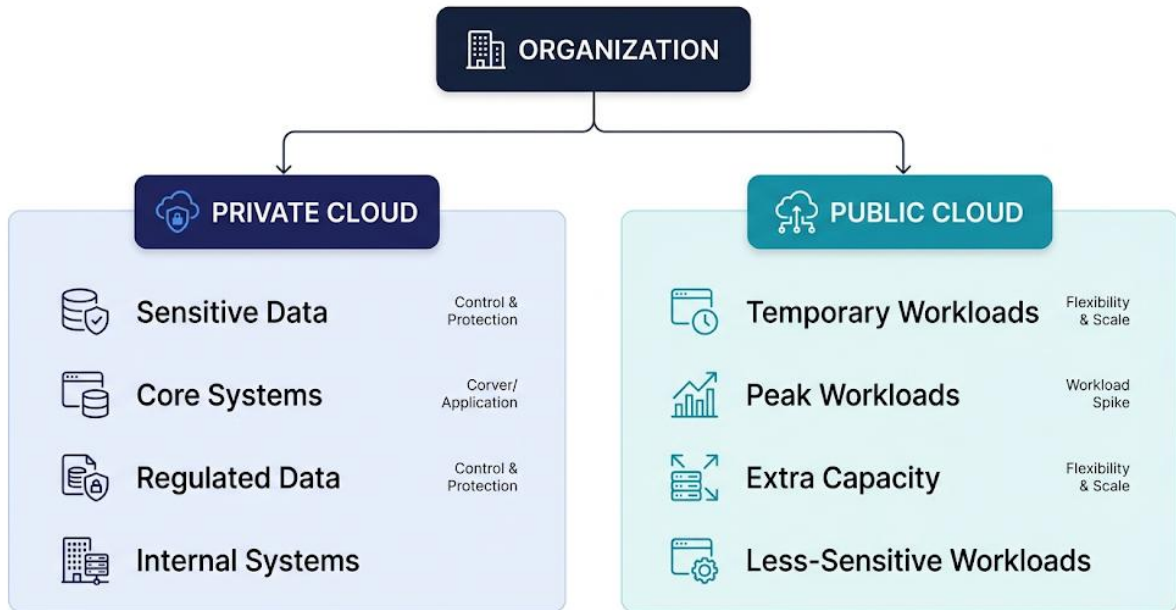


Figure: Hybrid Cloud Workload Placement

This allows the organization to maintain tighter control over its most sensitive workloads while using the scalability of public cloud where appropriate. However, this does not mean that everything sensitive must always stay private or that public cloud cannot securely handle sensitive workloads. The actual decision depends on the organization's security, regulatory, technical, and business requirements.

Hybrid Cloud Is More Than “Using AWS and a Private Server”

A common misconception is that an organization has a hybrid cloud simply because it uses both a private data center and a public-cloud provider such as AWS. That is not necessarily true. If some workloads run in the company's data center and other workloads run independently in the public cloud, the organization is using both environments, but this alone does not make them a hybrid cloud. The key requirement is integration. In a hybrid-cloud architecture, the private and public environments are connected and managed in a way that allows the organization to use them together according to its workload and business requirements.

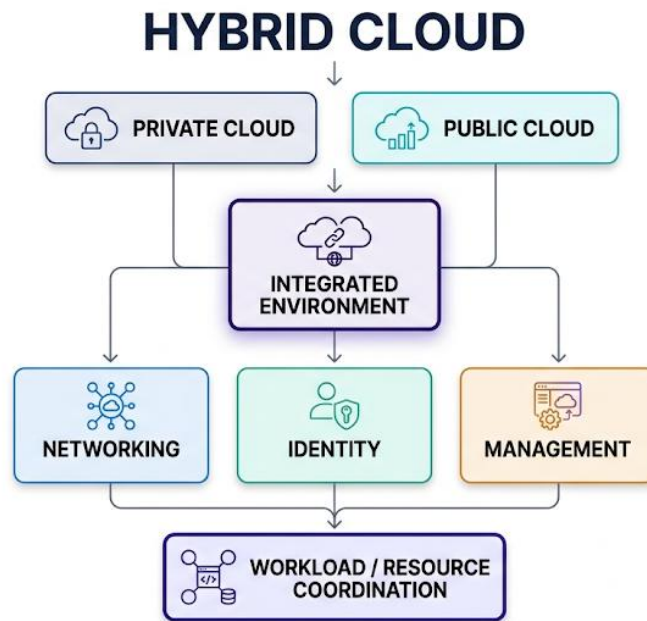
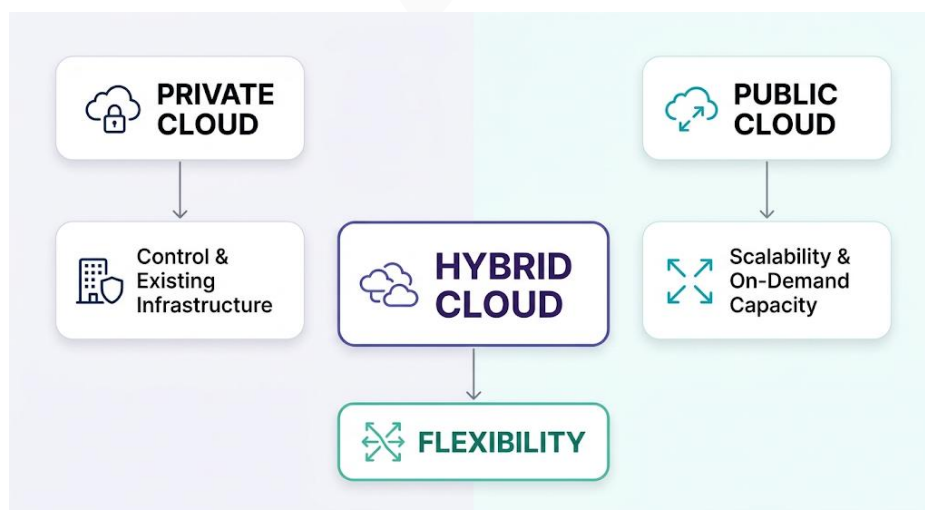


Figure: Hybrid Cloud Integration

The integration can involve networking, identity and access management, workload deployment, monitoring, data movement, orchestration, and resource provisioning. The exact technologies can vary, but the important point is that the two environments are able to work together, rather than simply existing separately.

The main advantage of hybrid cloud is flexibility. An organization can keep workloads in a private environment when it needs greater control, security, or use of existing infrastructure, while using public-cloud resources when it needs additional capacity or scalability.



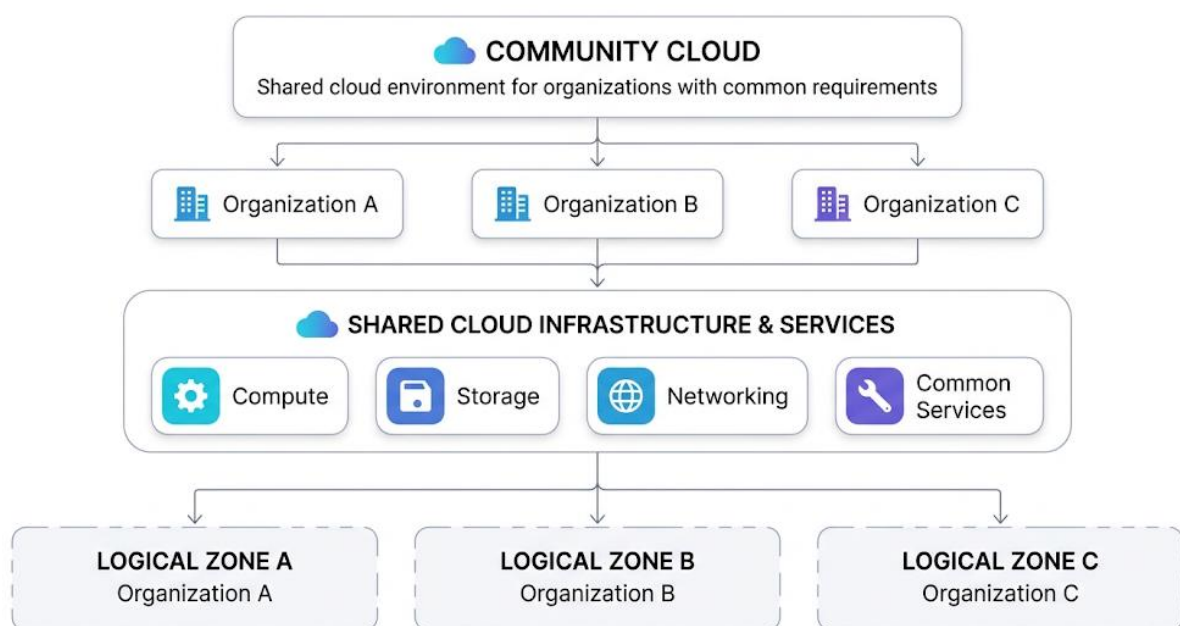
However, this flexibility comes with additional operational complexity. The organization must coordinate the private and public environments and manage areas such as connectivity, security, identity, workload placement, monitoring, data movement, and public-cloud costs. Therefore, hybrid cloud is not simply “private cloud + public cloud.” It is an integrated operating model in which private and public cloud environments can work together to meet an organization’s technical and business requirements. The trade-off is straightforward: hybrid cloud provides greater flexibility and scalability, but managing two integrated environments is more complex than managing a single environment.

Summary: A **hybrid cloud** combines a private cloud with one or more public clouds and integrates them so that an organization can use resources from both environments according to its workload and requirements. The private cloud can be used for workloads that require greater control, existing infrastructure, or specific security and compliance arrangements, while the public cloud can provide additional capacity and scalability when the private environment is not sufficient. A key feature of hybrid clouds is **dynamic provisioning**, where public-cloud resources are obtained when additional capacity is required and released when they are no longer needed. This temporary expansion of a private cloud into public-cloud resources is known as **cloud bursting**. For example, an organization may normally run an application using its private cloud but temporarily provision additional virtual machines from a public cloud during a sudden increase in demand. Hybrid cloud therefore combines the control and dedicated resources of private cloud with the scalability and flexibility of public cloud, although managing and integrating both environments introduces additional complexity.

Community Cloud

A community cloud is a cloud infrastructure shared by multiple organizations that belong to the same community or have common requirements. Unlike a public cloud, which is designed to serve customers with very different needs, a community cloud is designed specifically for organizations that have something important in common, such as the same industry, security requirements, regulations, policies, or mission. Unlike a private cloud, which is dedicated to a single organization, a community cloud is shared by several organizations.

Community cloud = shared cloud infrastructure + multiple organizations + common requirements.



For example, imagine several hospitals that need to exchange healthcare information and use common computing services. They may have similar security and compliance requirements because they all handle sensitive medical information. Instead of each hospital building a completely separate cloud, they could participate in a shared cloud designed specifically for healthcare organizations.

The organizations share the underlying infrastructure and services, but this does not mean that their data is freely shared. Each organization's resources and information can

remain logically isolated through access control, virtualization, security policies, and other mechanisms.

From an architectural perspective, a community cloud can be built across multiple administrative domains. This means that different organizations may contribute infrastructure, resources, or services to the overall cloud, or a third-party provider may manage the infrastructure on their behalf. The infrastructure can exist on-premises, off-premises, or across multiple locations. What makes it a community cloud is not who physically owns every server, but that the cloud is designed and shared specifically for organizations with common requirements.

Community clouds are particularly useful in areas such as healthcare, government, scientific research, energy, and media. For example, research institutions can share large amounts of computing and storage resources for scientific workloads, while government agencies can use a shared environment designed around their common security and compliance requirements.

Cloud Model	Infrastructure / Usage Model	Key Characteristic
Public Cloud	Shared by many organizations and users	Resources are provided by a cloud provider to customers with different requirements
Private Cloud	Dedicated to one organization	Provides greater organizational control over infrastructure, resources, and policies
Community Cloud	Shared by several organizations	Organizations share infrastructure because they have common requirements, such as security, compliance, or industry needs
Hybrid Cloud	Private + Public cloud environments	Integrates private and public enviro

Therefore, the central idea of a community cloud is not simply "many organizations sharing a cloud." The important point is that those organizations form a defined community because they have shared concerns, requirements, policies, or goals, and the cloud infrastructure is designed around those common needs.



Cloud Design and Implementation Using SOA

Cloud computing enables organizations to access computing resources such as applications, storage, processing power, and networking services on demand. However, designing cloud applications requires an architecture that can support scalability, flexibility, interoperability, and efficient communication between different application components.

Service-Oriented Architecture (SOA) is an architectural approach that organizes application functionality into independent and reusable services. Each service performs a specific function and communicates with other services through well-defined interfaces. When SOA principles are applied to cloud computing, applications can be designed as a collection of services that can be deployed, managed, and scaled within a cloud environment.

SOA therefore provides a useful architectural foundation for developing distributed cloud applications.

Understanding Service-Oriented Architecture

What is SOA?

Service-Oriented Architecture is a software architecture in which application functionality is provided through a collection of services. A service is a self-contained component that performs a specific business or technical function and makes that functionality available to other applications or services.

For example, an online shopping application could contain separate services for customer management, product management, order processing, payment processing, inventory, and notifications.

Each service has a defined responsibility and communicates through an interface without requiring other services to know how it is internally implemented.

Principles of SOA

SOA is based on several important principles.

- **Loose coupling** means that services should have minimal dependencies on one another. A service communicates through a defined interface rather than depending on another service's internal implementation.
- **Reusability** means that a service can be used by multiple applications or business processes. For example, an authentication service can be reused by web applications, mobile applications, and other cloud services.
- **Interoperability** allows services implemented using different technologies or programming languages to communicate with each other through standardized interfaces and protocols.
- **Service abstraction** means that users of a service do not need to know how the service works internally. They only need to know how to interact with its interface.
- **Composability** allows multiple services to be combined to implement a larger business process. For example, an online order may involve order, inventory, payment, shipping, and notification services.

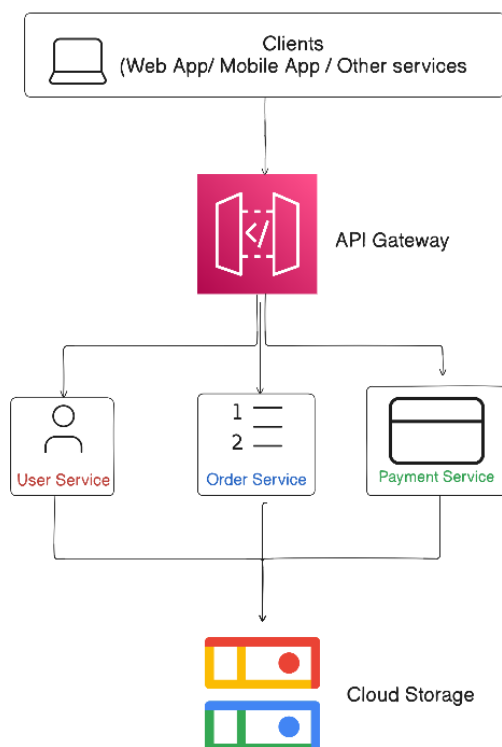
These principles make SOA particularly useful for distributed computing environments.

SOA in Cloud Computing

SOA and cloud computing are related but are not the same thing. SOA is an architectural approach for organizing software, while cloud computing is a model for delivering computing resources and services.

SOA determines how an application can be divided into services, while the cloud provides the infrastructure and platform needed to run those services.

For example, a cloud-based application may contain separate user, order, payment, and inventory services. These services can run on cloud infrastructure and communicate through APIs or messaging systems.



This architecture allows different services to be managed and scaled according to their individual requirements.

Cloud Design Using SOA

Designing a cloud application using SOA begins by identifying the major functions or business capabilities of the application. These functions are then organized into appropriate services.

For example, an e-commerce application might be divided into:

- User Management Service
- Product Service
- Order Service
- Payment Service
- Inventory Service
- Notification Service

The objective is not simply to create as many services as possible. Services should have clear responsibilities and should avoid unnecessary dependencies. After identifying services, developers define service interfaces. These interfaces specify how other applications or services can communicate with them. APIs are commonly used for this purpose. A well-designed service should be independently understandable and should expose only the functionality required by its consumers.

Implementation of SOA in the Cloud

Once the services have been designed, they need to be implemented and deployed in a cloud environment.

Services can be deployed using cloud virtual machines, containers, managed platforms, or serverless technologies, depending on the requirements of the application.

Services may communicate synchronously through APIs or asynchronously through messaging systems.

In synchronous communication, one service sends a request and waits for a response. For example, an Order Service may contact a Payment Service and wait for confirmation.

In asynchronous communication, a service sends a message or event without necessarily waiting for an immediate response. For example, after an order is created, an event can be sent to a Notification Service to inform the customer.

Cloud environments also allow individual services to be scaled according to demand. If a Product Service receives a large number of requests, additional instances of that service can be created without necessarily scaling the entire application.

Communication and Integration

Communication between services is an important part of SOA. Common approaches include REST APIs, SOAP, message queues, and event-based communication.

- REST APIs are widely used for cloud applications because they provide a simple way for services and applications to communicate over HTTP. SOAP is another approach commonly associated with traditional enterprise SOA and can be useful where formal messaging and enterprise integration requirements exist.
- Messaging and event-based communication can reduce direct dependencies between services. A service can publish an event, and other services can respond to that event independently.
- SOA can therefore integrate applications and services even when they use different technologies, programming languages, or platforms.

Benefits of SOA in Cloud Computing

SOA provides several advantages when designing cloud applications.

- **Scalability** is improved because individual services can be scaled according to their workload rather than scaling the entire application.
- **Reusability** allows commonly required functionality to be shared among multiple applications.
- **Flexibility** allows individual services to be modified or replaced without necessarily changing the entire application.
- **Interoperability** allows different systems and technologies to communicate through standardized interfaces.
- **Maintainability** can improve because application functionality is separated into well-defined services.
- **Fault isolation** can also be improved because the failure of one service does not necessarily mean that the entire application must fail.

Challenges of SOA in Cloud Computing

Although SOA provides many benefits, it also introduces challenges. A large number of services can increase:

- **system complexity** because developers and administrators must manage service dependencies and communication.
- **Network dependency** is another challenge because services often communicate over networks, which introduces latency and the possibility of network failures.
- **Security** is important because every service and API can become a potential security boundary. Authentication, authorization, encryption, and access control must therefore be properly implemented.
- **Data consistency** can also be difficult when different services manage different data sources.

Organizations may also require monitoring, logging, governance, and service management to ensure that a large SOA-based cloud environment remains reliable and manageable.

SOA and Microservices

SOA and microservices are closely related because both use services as fundamental building blocks. However, they are not exactly the same.

Traditional SOA generally emphasizes enterprise-wide integration, service reuse, interoperability, and governance. Microservices place greater emphasis on small, independently deployable services and decentralized development.

Both approaches can be used to design distributed cloud applications, but the appropriate choice depends on the organization's requirements, system complexity, and development model.

Conclusion:

Service-Oriented Architecture provides a structured approach for designing cloud applications as a collection of independent, reusable, and interoperable services. Cloud

computing provides the infrastructure and platform capabilities required to deploy, scale, and manage these services.

By combining SOA with cloud computing, organizations can build applications that are more flexible, scalable, reusable, and easier to integrate. However, successful implementation requires careful service design, secure communication, appropriate data management, monitoring, and governance.

Therefore, SOA serves as an architectural approach for organizing cloud application functionality into services, while cloud computing provides the environment in which those services can be deployed and operated efficiently.

Cloud Storage Architecture

Cloud storage isn't just "uploading a file somewhere." Behind the scenes, providers like Microsoft Azure follow a carefully designed strategy to decide what type of storage to use, how many copies to keep, and where to keep them — so that data stays fast to access, highly available, and safe even during a disaster. This note explains that architecture using Azure as the reference (AWS and Google Cloud follow the same logic, just with different service names).

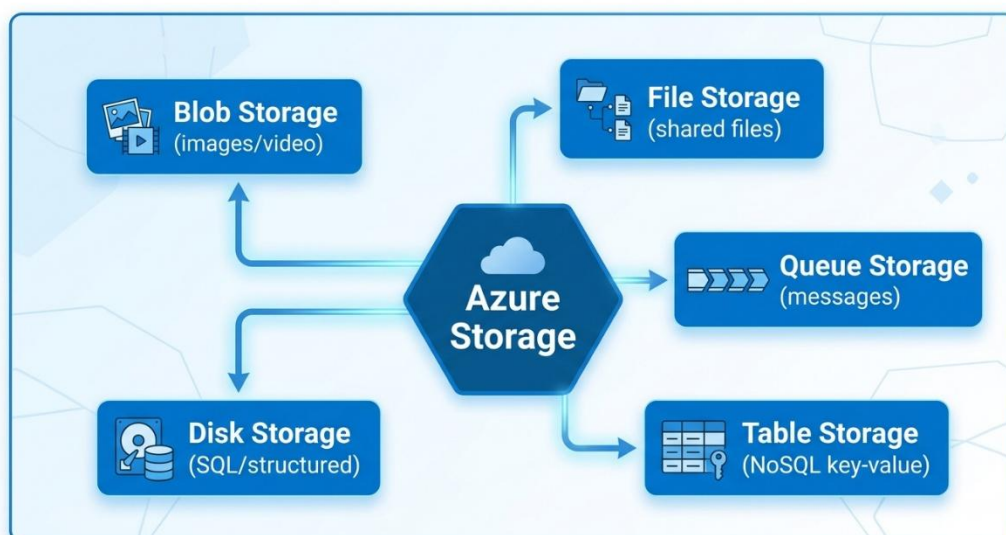
Of the three core cloud pillars — compute, storage, and networking — storage exists to solve four goals: store massive volumes of data, deliver good performance, scale as data grows, and recover quickly if something goes wrong.

Step 1: Classifying Data by Type

Before any data is stored, the cloud first identifies **what kind of data it is**, because each data type is handled differently for performance and cost reasons. Azure offers five storage services, each purpose-built for one data type:

Storage Type	Data It Handles	Real-World Example
Blob Storage	Unstructured binary data — images, videos, audio	Netflix or a news channel (BBC, Aaj Tak) storing video/audio content
File Storage	Shared files (text, documents) accessed by multiple users/systems	A multinational company where remote teams share files through a common repository

Queue Storage	Messages/requests that need asynchronous processing	Uber/Ola storing thousands of ride requests during a rush (e.g., right after a concert ends) before assigning drivers
Table Storage	NoSQL, schema-less data in key-value pairs	IoT sensors (temperature, humidity) continuously generating data identified by unique keys
Disk Storage	Structured, relational (SQL) data	Standard business/transactional databases needing table structure



This classification step matters because using the wrong storage type for a given data category would either hurt performance or waste money — e.g., storing millions of small IoT sensor readings in a relational table structure would be inefficient, while storing video files as key-value data would make no sense either.

Step 2: Redundancy – Never Trust a Single Copy

The single most important principle in cloud storage is **redundancy**: never keep just one copy of data. Recall from cloud architecture that a **region** contains multiple **availability**

zones, and each zone contains multiple **data centers**. Redundancy uses this same hierarchy as insurance against failure.

Azure's baseline rule is that **any piece of data must have at least 3 copies within a region** — so 1 TB of data actually results in at least 3 TB being stored. Beyond this, providers also replicate data **across regions** (inter-region), so that even if an entire region goes down, a copy still exists elsewhere. The logic is simple: if one data center fails, its neighboring data center in the same zone still has the data. If the whole zone fails, another zone in the same region takes over. If the whole region fails, another region entirely (geographically distant) has the backup copy.

This is why Azure actually provides several specific redundancy options, each covering a different scope of failure:

Redundancy Type	What It Protects Against	How Data Is Copied
LRS (Locally Redundant Storage)	Disk/server/rack failure	3 copies within a single data center
ZRS (Zone-Redundant Storage)	Data-center-level failure	3 copies spread across 3 availability zones in one region
GRS (Geo-Redundant Storage)	Entire region failure (disaster)	3 copies in primary region + 3 copies in a paired secondary region (6 total)
RA-GRS (Read-Access GRS)	Same as GRS, plus lets you <i>read</i> from the secondary region even during normal operation	Same as GRS, with read access enabled on the secondary
GZRS (Geo-Zone-Redundant Storage)	Zone failure <i>and</i> regional disaster together	ZRS in primary region + LRS copy in secondary region

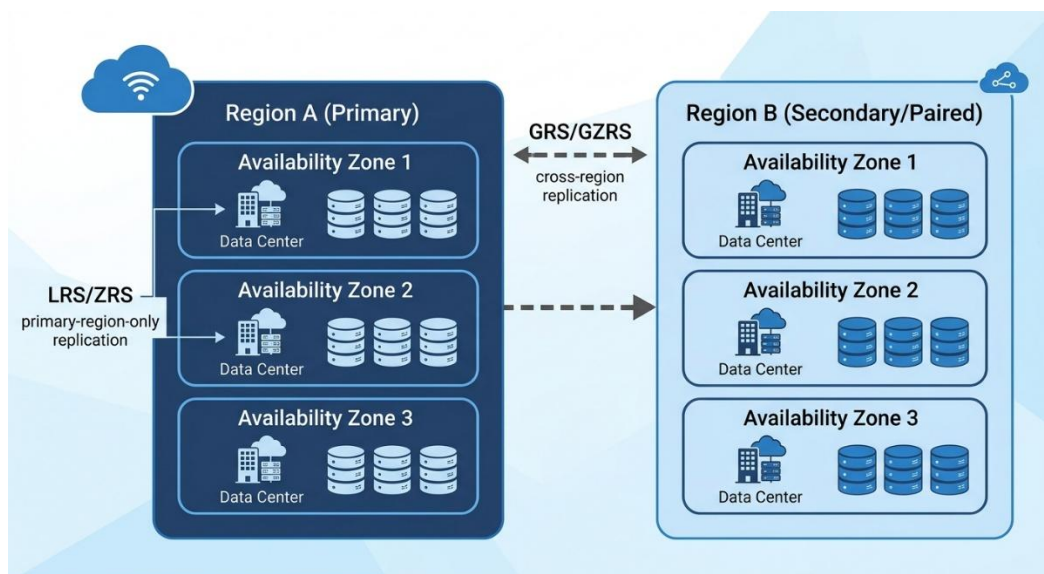


fig: hierarchical diagram

The result of all this redundancy is expressed as a durability percentage, often described using "nines" — for example, 99.999999999% (11 nines) or higher. The more 9's in that number, the more redundant copies exist, and the lower the chance of ever losing data permanently — even LRS alone protects against hardware failure, but higher-tier options like GRS push durability up toward 16 nines by protecting against full regional disasters too.

Step 3: Tiering – Not All Data Is Equally "Hot"

Providers handle petabytes to zettabytes of data, so it isn't practical (or affordable) to keep everything on the fastest storage hardware (like SSDs) all the time. Instead, data is labeled by how frequently it's accessed, and moved between "tiers" accordingly — a strategy directly tied to cost optimization.

Tier	Access Frequency	Storage Hardware Used	Example
Hot Tier	Accessed very frequently (recent/active data)	Fast SSDs	A newly arrived Gmail email — most likely to be opened soon

Cool/Cold Tier	Accessed occasionally (e.g., every few days to weeks)	Standard hard disks	An email from a week or a month ago
Archive Tier	Rarely or never accessed (e.g., untouched for 6 months to a year)	Tape drives, low-cost archival disks	Old emails or transaction records you haven't opened in a year

Frequently accessed ("hot") data is also cached — kept on servers geographically nearest to where it's likely to be requested — so it can be delivered with minimal delay. As data ages and is accessed less, it's automatically shifted down from hot → cold → archive, balancing performance needs against the very real cost of running massive infrastructure entirely on expensive fast storage.

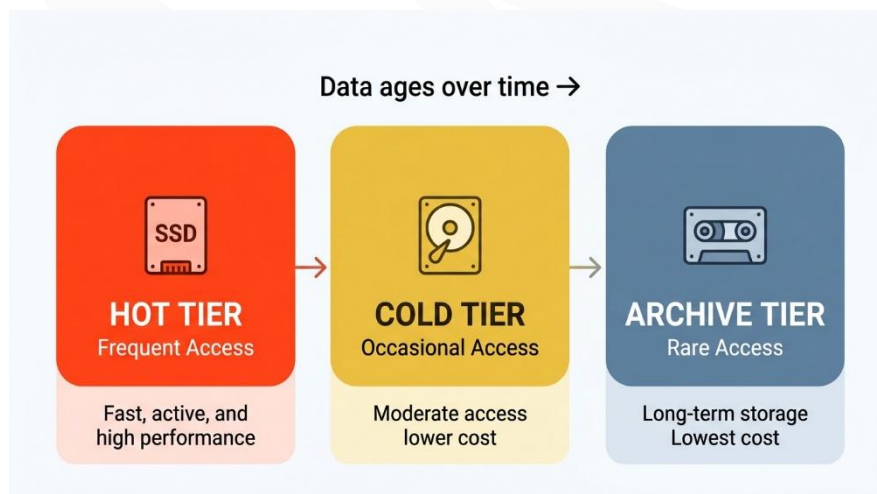


Figure: Data Storage Tiering Diagram

When data enters Azure's storage system, it effectively goes through this pipeline:

1. **Classify** the data type → routed to Blob, File, Queue, Table, or Disk storage
2. **Replicate** it for redundancy → LRS/ZRS/GRS/GZRS, based on how much disaster protection is needed
3. **Tier** it by usage frequency → Hot, Cold, or Archive, to balance speed and cost

This same three-step logic — classify, replicate, tier — applies across AWS (S3, EBS, Glacier) and Google Cloud as well; only the service names differ, not the underlying architecture.

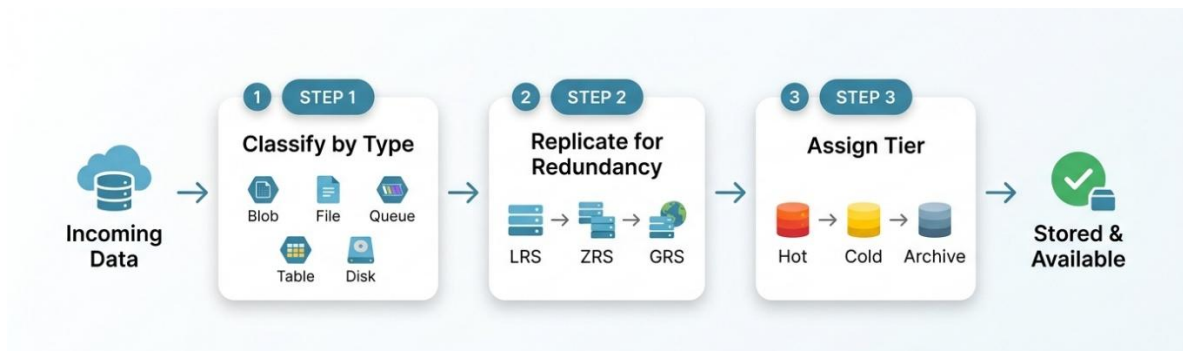


Figure: Data Storage Flow

Table: Summary

Concept	Explanation	Example
Blob Storage	Unstructured binary data	Netflix videos
File Storage	Shared file access across systems/users	Multinational company file sharing
Queue Storage	Asynchronous message/request handling	Uber ride requests
Table Storage	NoSQL key-value data	IoT sensor readings
Disk Storage	Structured/SQL data	Standard business databases
Redundancy (LRS/ZRS/GRS)	Multiple copies within/across regions	Protects against disk, data center, or regional failure
Hot/Cold/Archive Tiering	Data placement based on access frequency	Gmail inbox aging behavior

Virtualization in Cloud Computing and Types

Virtualization is the fundamental technology that powers Cloud Computing. It allows you to create multiple simulated environments (Virtual Machines or VMs) from a single physical hardware system. Before virtualization, a physical server could only run one Operating System (OS) and often one task. This wasted massive amounts of resources if your app only used 10% of the CPU, the other 90% was idle.

- Better hardware utilization
- Reduced infrastructure costs
- Easy scalability and flexibility
- Improved isolation and security
- Faster deployment of servers
- Foundation for cloud services like AWS EC2, Azure VMs, and Google Compute Engine

Types of Virtualization

1. Server Virtualization

Server virtualization divides a single physical server into multiple virtual machines (VMs). Each VM acts as an independent computing environment with its own operating system, applications, and allocated resources. A software layer called a hypervisor creates and manages the virtual machines and controls how physical resources such as CPU, memory, storage, and network interfaces are shared among them.

There are two common types of hypervisors. A Type 1 hypervisor, also called a bare-metal hypervisor, runs directly on the physical hardware, while a Type 2 hypervisor runs on top of a host operating system. Type 1 hypervisors are commonly used in data centers and cloud environments because they provide efficient resource management and strong isolation between virtual machines.

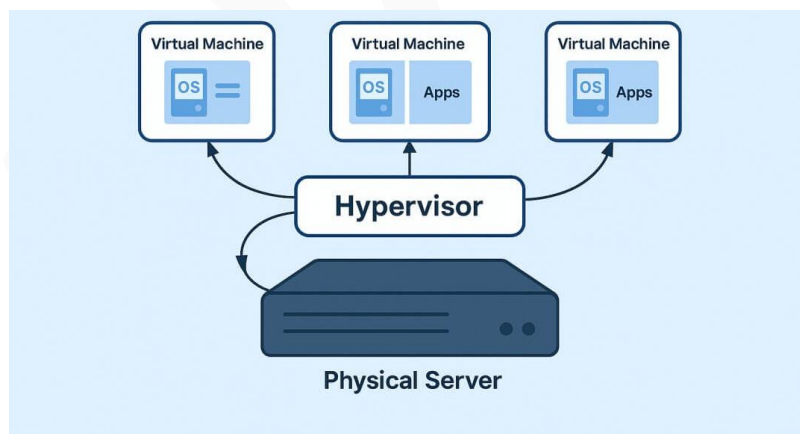


Figure: Server Virtualization

For example, a single physical server with sufficient resources could run separate VMs for a web server, database server, and application server. Each VM can be started, stopped, resized, or migrated independently. Server virtualization improves hardware utilization because multiple workloads can share the same physical machine instead of requiring a separate server for each workload.

Server virtualization is one of the most important forms of virtualization in cloud computing because cloud providers use it to create and manage virtual servers for customers. It supports resource sharing, isolation, scalability, flexibility, and efficient utilization of physical infrastructure, making it a major foundation of Infrastructure as a Service (IaaS).

2. Storage Virtualization

Storage virtualization combines physical storage resources from multiple disks, storage devices, or storage systems and presents them to users and applications as a single logical storage resource. The physical location of the data is hidden from the user, allowing storage to be managed independently of the underlying hardware.

For example, an organization may have storage distributed across several physical disks or storage systems. A virtualization layer can combine these resources and provide applications with a logical storage pool. Administrators can then allocate storage to applications without requiring them to know which physical device contains the data.

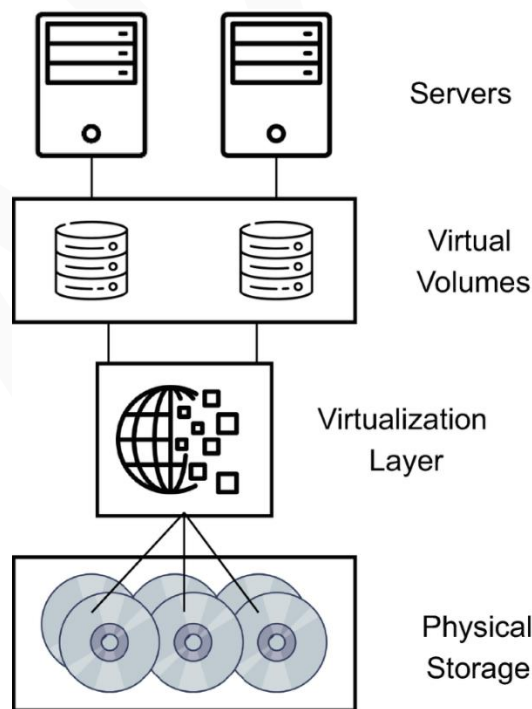


Figure: Storage Virtualization

Storage virtualization can also support features such as storage pooling, replication, backup, migration, and dynamic allocation. Storage resources can be allocated when needed and managed centrally, which can improve utilization and simplify administration.

In cloud computing, storage virtualization allows cloud providers to offer flexible storage services without exposing the underlying physical storage infrastructure to customers.

Users can request storage capacity and access it as a logical resource while the cloud provider manages the physical devices behind the service.

3. Network Virtualization

Network virtualization creates logical or virtual networks on top of a shared physical networking infrastructure. Instead of requiring separate physical network equipment for every organization, department, or application, multiple virtual networks can operate over the same physical network.

For example, an organization could create separate virtual networks for its development, testing, and production environments. Although these environments may share the same physical network infrastructure, network virtualization can logically isolate their traffic and apply different security and routing policies.

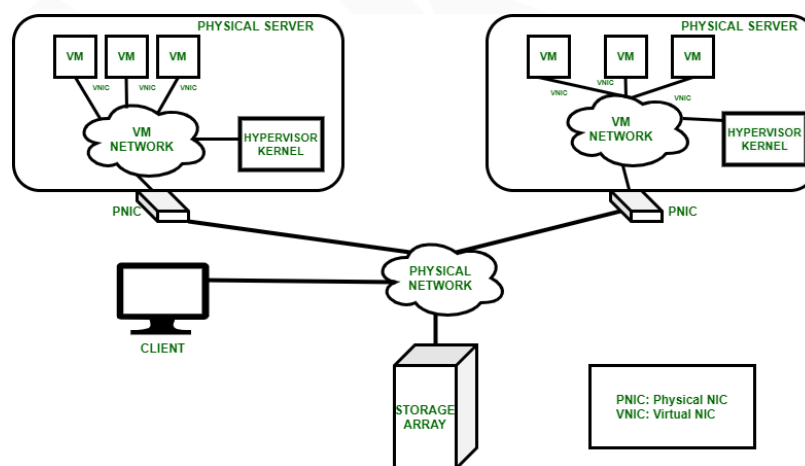


Figure: Network Virtualization In VDC

Network virtualization can include technologies such as virtual switches, virtual routers, virtual networks, VLANs, and software-defined networking (SDN). These technologies allow network resources and policies to be configured through software rather than relying entirely on physical network devices.

Network virtualization is particularly important in cloud computing because cloud providers must allow many customers and applications to share physical infrastructure while maintaining network isolation, security, flexibility, and independent configuration.

4. Desktop Virtualization

Desktop virtualization separates a user's desktop environment from the physical device used to access it. The desktop operating system, applications, and user environment can run on a centralized server or cloud infrastructure, while the user interacts with the desktop through a computer, laptop, tablet, or other client device.

In a desktop virtualization environment, the user's desktop can be centrally managed by an organization. Administrators can control applications, security policies, updates, and user access from a central location rather than configuring every physical device individually.

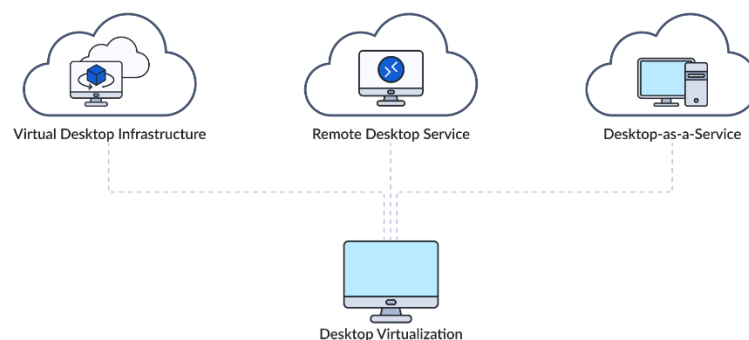


Figure 1: Desktop Virtualization Deployment Models

Desktop virtualization can support different deployment models. In some environments, each user receives a dedicated virtual desktop, while other models provide shared or remotely delivered application environments.

This approach allows users to access their desktop environment from different devices and locations, provided they have the appropriate network connection and permissions. It is particularly useful for remote work, centralized administration, flexible device access, and reducing the management burden of physical desktops.

In cloud computing, desktop virtualization can be delivered as a cloud service, allowing organizations to provide virtual desktops without maintaining all of the underlying desktop infrastructure themselves.

5. Application Virtualization

Application virtualization allows an application to operate in an isolated or abstracted environment rather than being installed and integrated into the operating system in the traditional manner. The application can be separated from the underlying operating system to simplify deployment and management.

In a virtualized application environment, the application and its required components can be packaged or delivered through a controlled environment. Users can then access the application without administrators having to perform a conventional installation on every individual device.

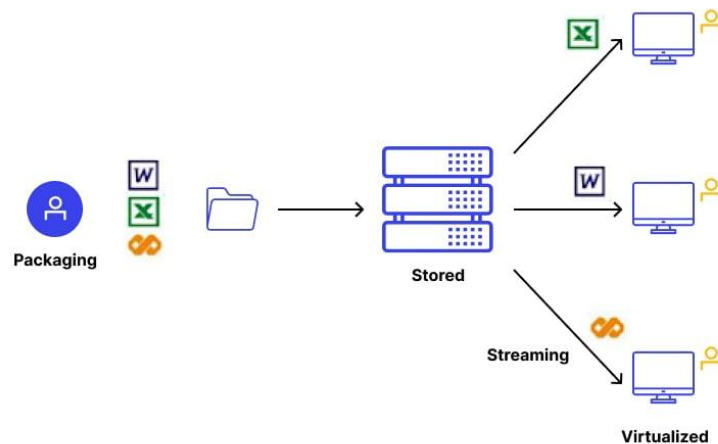


Figure: Application Virtualization

Application virtualization can help prevent conflicts between applications because applications can operate in separate environments. It can also simplify application deployment, updates, maintenance, and centralized management.

For example, an organization can centrally manage a business application and make it available to authorized employees without manually installing and updating the application on every workstation.

In cloud environments, application virtualization can also support the delivery of applications remotely, allowing users to access applications without necessarily having the complete application environment installed locally.

6. Operating System Virtualization

Operating system virtualization creates multiple isolated environments within a single operating system kernel. These environments are commonly known as containers.

Unlike traditional virtual machines, containers generally do not require a separate operating-system kernel for every environment. Instead, multiple containers share the host operating system kernel while maintaining separate application environments.

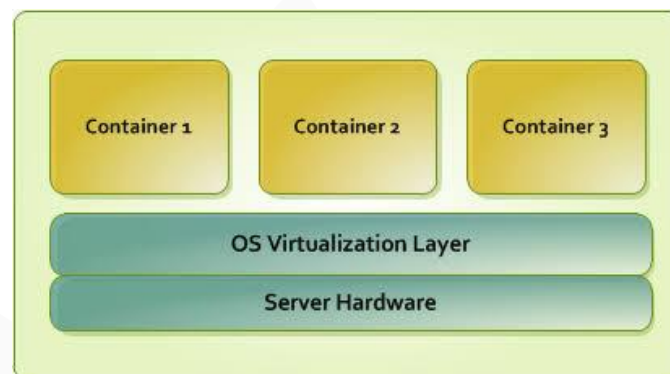


Figure: Basic OS Virtualization

Because containers share the host kernel, they are generally more lightweight than full virtual machines. They can start quickly, require fewer resources, and allow applications to be packaged together with their dependencies.

Operating system virtualization is particularly useful for cloud-native applications, application deployment, continuous integration and delivery, and distributed service architectures. Containers allow applications to be moved consistently between development, testing, and production environments.

However, containers and virtual machines provide different levels of isolation and serve different purposes. Virtual machines virtualize complete computing environments, whereas containers primarily isolate applications while sharing the host operating system kernel.

7. Data Virtualization

Data virtualization provides a unified way of accessing data from multiple sources without requiring the data to be physically moved into a single storage location. Modern organizations often store information across different environments, such as:

- Cloud databases
- On-premises databases
- Data warehouses
- Files
- SaaS applications

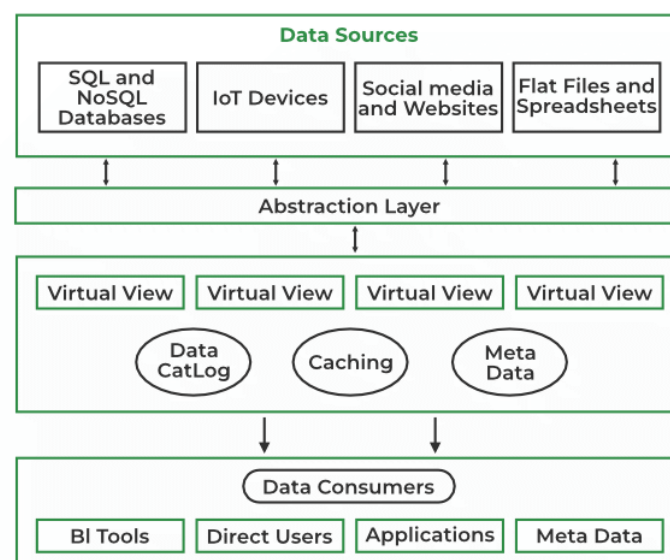


Figure: Data Virtualization

A data virtualization layer provides applications and users with a logical view of these different data sources. The underlying data can remain in its original locations while the virtualization layer handles access and integration.

For example, an organization might need to create a report using customer information stored in a cloud database and sales information stored in an on-premises database. Data virtualization can provide a unified view of the information without requiring all of the data to be copied into one physical database.

Data virtualization can improve data accessibility, integration, flexibility, and development efficiency, particularly in hybrid and multicloud environments where information is distributed across multiple systems.

8. Hardware Virtualization

Hardware virtualization abstracts physical hardware resources and presents them as virtual resources to operating systems and applications. A hypervisor is commonly used to create and manage virtual machines by providing each VM with virtual CPU, memory, storage, and network interfaces.

For example, instead of installing three operating systems directly on three separate physical servers, an organization can run three virtual machines on one physical server:

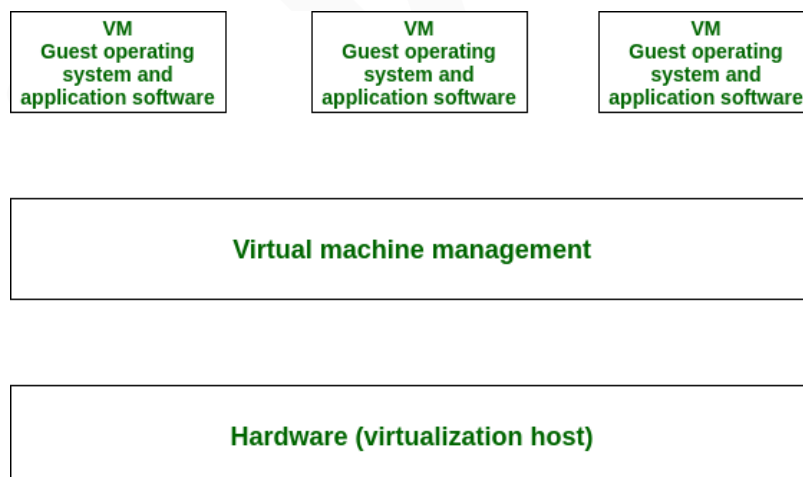


Figure: Hardware Virtualization

Each virtual machine operates as if it has its own physical hardware, even though the underlying resources are shared.

Hardware virtualization improves hardware utilization, workload isolation, flexibility, and resource allocation. Virtual machines can also be created, deleted, copied, migrated, and resized more easily than physical servers.

Hardware virtualization is closely related to server virtualization and is a major foundation of cloud computing. Cloud providers use it to abstract physical servers and provide customers with virtual computing resources. This enables services such as

Infrastructure as a Service (IaaS), where users can provision virtual servers without directly managing the physical hardware.



Virtualization – Operating System Virtualization

Most of us run multiple operating systems on our laptops using simple methods:

- **Dual boot** – choose an OS at startup, but only one OS runs at a time
- **Live USB** – boot into an OS from an external USB drive

Both approaches share a big limitation: only one OS runs at any given moment, and there's no real isolation between them.

At the enterprise level, dedicating one physical machine to just one task is wasteful — a company doesn't want expensive servers/storage sitting idle just because they're "assigned" to a single user or task. The solution is to run resources in a shared mode — this is exactly what virtualization enables.

What Is Operating System Virtualization?

Creating virtual instances of an operating system that run concurrently (at the same time) on a single physical machine.

The key difference from dual-boot: with virtualization, multiple OS instances run simultaneously and independently, each unaware that the other exists — this is called isolation.

Example: Using Oracle VirtualBox — if your laptop runs Windows (the base OS) and you install VirtualBox with Linux running inside it, Windows doesn't know Linux is running, and

Linux doesn't know Windows is the underlying system. Both share the same physical hardware (CPU, RAM, storage) without needing any extra hardware purchase.

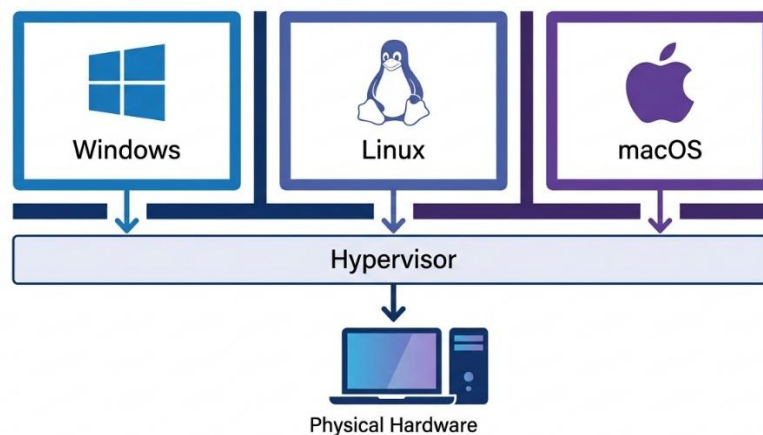


Figure: Virtualization Basic Architecture

Term	Meaning
Host Operating System	The primary/base OS installed directly on the physical machine (e.g., Windows on your laptop)
Guest Operating System	The OS installed inside the virtualization software (e.g., Linux running inside VirtualBox)
Hypervisor	The software layer that manages and allocates hardware resources (CPU, RAM, storage) among all the operating systems

The Role of the Hypervisor

Think of a hypervisor like a hotel manager: in a normal hotel (like dual boot), only one guest can use a room at a time. But with a manager efficiently allocating rooms, resources, and services, multiple guests can stay and be served simultaneously without interfering with each other.

Similarly, the hypervisor allocates CPU, RAM, and storage to each operating system, allowing all of them to run their applications properly at the same time — without any of them being aware of the others.

(Note: the two types of hypervisors — Type 1 and Type 2 — are covered in detail in the next section.)

Characteristics of Virtual Machines:

- **Independent operation:** Each virtual machine (VM) runs on its own; if one VM crashes, it does **not** affect the others — a major advantage over dual boot/live USB, which offer no such isolation.

Advantages of OS Virtualization

Advantage	Explanation
Isolation	Each VM operates independently; a failure in one doesn't impact others
Cost Efficiency	No need to buy separate hardware for each OS/task — same hardware runs multiple environments
Efficient Resource Utilization	CPU/RAM/storage don't "get tired" — maximize usage instead of leaving them idle
Scalability	Scale usage on the same device without buying/configuring new hardware
Flexibility & Portability	VMs can be easily moved or run across different environments
Easy Testing & Development	Test the same application across multiple OS environments (Windows, Linux, Mac) on one machine — no extra hardware needed
Disaster Recovery & Backup	Snapshots allow quick backup/restore of a VM's exact state; if a VM fails, reboot/reinstall/reconfigure easily using the snapshot
Replication & Migration	Entire VMs can be copied (replicated) or moved (migrated) to another physical machine easily

Table: Summary

Concept	Explanation	Example
Dual Boot	One OS at a time, no isolation	Choosing Windows or Linux at startup
OS Virtualization	Multiple OS run concurrently, isolated	VirtualBox running Linux inside Windows
Host OS	Primary OS on physical machine	Windows on your laptop
Guest OS	OS running inside virtualization software	Linux inside VirtualBox
Hypervisor	Manages/allocates hardware resources to each OS	VirtualBox, Hyper-V

Types of Hypervisor (Type 1 vs Type 2)

A hypervisor is the software layer that sits between physical hardware and virtual machines, managing and allocating resources (CPU, RAM, storage, network interfaces) so that multiple operating systems can run on the same physical machine. Hypervisors come in two types, based on whether a host operating system exists between the hardware and the hypervisor.

Type 1 Hypervisor – Bare Metal

The hypervisor is installed directly on the hardware — there is no host operating system in between.

- Hardware (CPU, RAM, disk, NIC, peripherals) → Hypervisor (installed directly) → Multiple VMs, each running its own OS/applications
- Because there's no intermediate host OS, there's one less layer that could introduce vulnerabilities, misconfigurations, or malware
- Used primarily at the enterprise level, where higher security and isolation are critical

Examples: VMware ESXi, Microsoft Hyper-V (server-based), Xen Hypervisor

Type 2 Hypervisor – Hosted

The hypervisor runs on top of a host operating system (like Windows, Mac, or Linux) already installed on the machine.

- Hardware → Host OS (Windows/Mac/Linux) → Hypervisor (runs as an application on the host) → Multiple VMs
- Commonly used for personal use — e.g., installing Oracle VirtualBox on your laptop to run additional operating systems

Examples: Oracle VirtualBox, VMware Workstation, Parallels Desktop

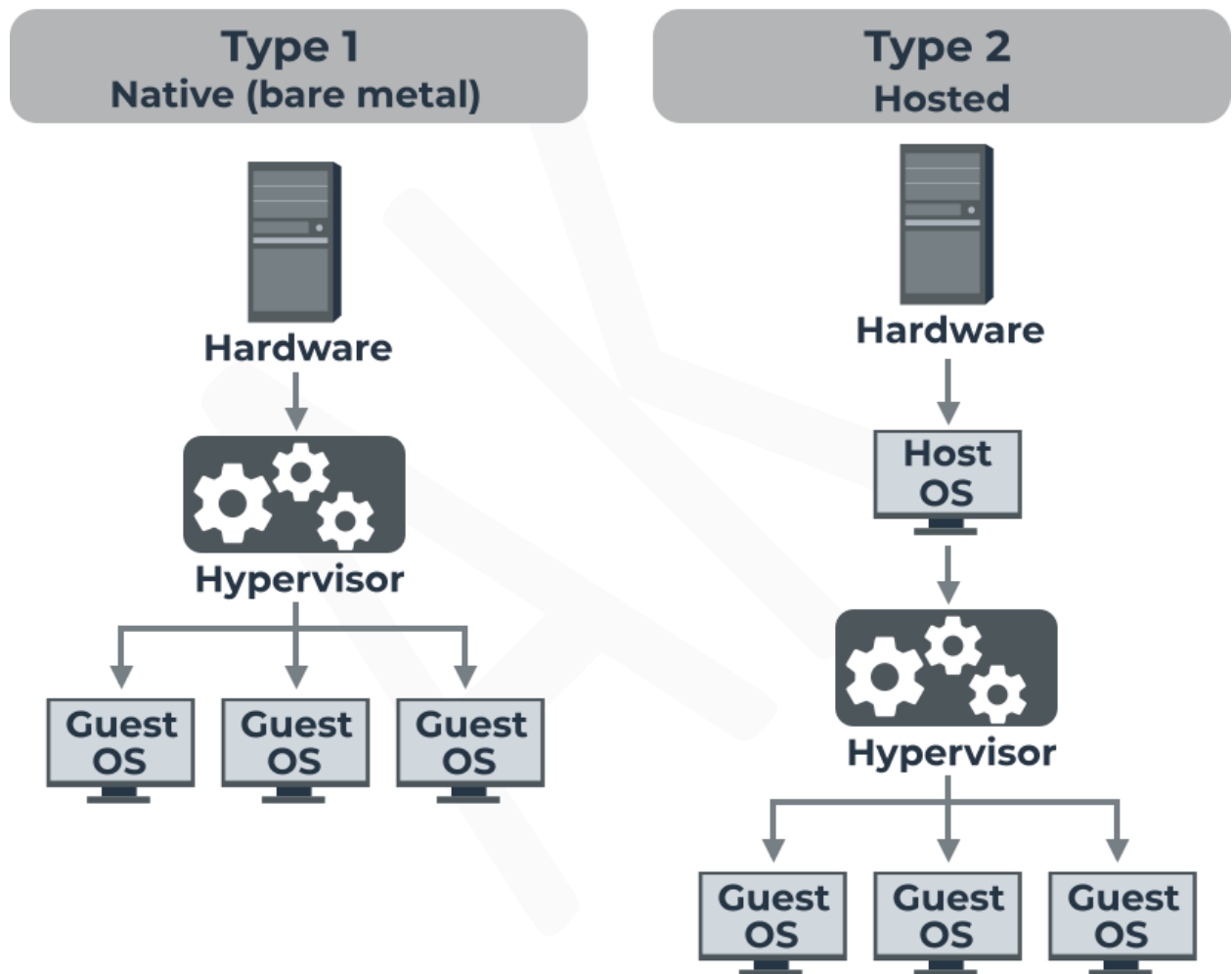


Figure: Hypervisor Architecture Comparison

Table: Type 1 vs Type 2 – Comparison

Aspect	Type 1 (Bare Metal)	Type 2 (Hosted)
Host OS Required?	No — installed directly on hardware	Yes — runs on top of an existing OS

Performance	High (no extra OS layer overhead)	Comparatively lower
Security & Isolation	Higher (fewer layers = fewer vulnerabilities)	Comparatively lower (host OS can introduce risks, needs proper patching)
Setup Complexity	More complex to configure and manage	Simple, easy to set up
Typical Usage	Enterprise/data centers (used by AWS, Microsoft, Google in their backend infrastructure)	Personal use, testing, development on personal laptops
Examples	VMware ESXi, Hyper-V, Xen	VirtualBox, VMware Workstation, Parallels Desktop

Why the Difference Matters

- **Removing the host OS layer (Type 1)** means one less component that could be misconfigured, unpatched, or exploited by malware — this directly improves both **performance** (fewer layers to pass through) and **security** (smaller attack surface).
- **Keeping the host OS (Type 2)** makes setup much simpler and is perfectly fine for personal/testing use, but the extra layer must be maintained (patched, secured) properly, or it becomes a vulnerability point.
- This is exactly why major cloud providers (AWS, Microsoft, Google) run **Type 1 hypervisors** in their actual data centers — they need maximum performance, isolation, and security at scale — while everyday users typically use **Type 2 hypervisors** like VirtualBox for convenience.

Table: Summary

Concept	Explanation	Example
---------	-------------	---------

Type 1 (Bare Metal)	Hypervisor runs directly on hardware, no host OS	VMware ESXi, Hyper-V — used in enterprise data centers
Type 2 (Hosted)	Hypervisor runs on top of a host OS	VirtualBox, VMware Workstation — used for personal/testing use
Performance	Type 1 > Type 2 (fewer layers)	—
Security	Type 1 > Type 2 (smaller attack surface)	—
Ease of Setup	Type 2 > Type 1 (simpler for personal use)	—

Hypervisor Ecosystem

A hypervisor alone only does one job — it creates and runs virtual machines on a physical device. But running virtual machines at scale requires much more: monitoring, network management, storage management, desktop virtualization, migration, and backup. All these supporting tools together are called an ecosystem — a collection of tools built around a core technology.

This is the same idea as the Apache Hadoop ecosystem in Big Data, where Hadoop isn't just one tool — it works alongside MapReduce, Pig, Hive, and many other components. Similarly, a hypervisor doesn't work alone; it's supported by an entire suite of management tools.

Here, we use VMware — one of the most popular hypervisor and virtualization ecosystem providers, used both by individuals and enterprises — as the reference example.

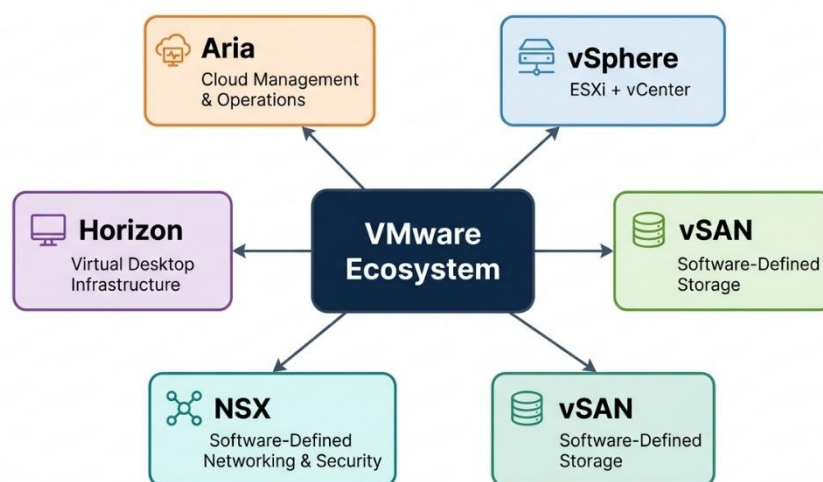


Figure: VMware Infrastructure Components

1. VMware vSphere

vSphere is VMware's foundational suite, made up of two main components:

Component	Type	Purpose
ESXi	Hypervisor	Creates and runs virtual machines directly on hardware
vCenter	Management tool	Centrally manages, monitors, and migrates multiple ESXi hosts/VMs

- **ESXi – The Hypervisor**

ESXi is VMware's Type 1 (bare metal) hypervisor — it installs directly on the physical machine without needing a host operating system, and multiple virtual machines are created directly on top of it. This is the same Type 1 concept covered earlier (bare metal vs. hosted), just VMware's specific product.

VMware also offers Type 2 (hosted) hypervisor products for personal use, which require an existing host OS:

Product	Type	Platform
ESXi	Type 1 (Bare Metal)	Enterprise servers, no host OS needed
VMware Workstation	Type 2 (Hosted)	Windows/Linux machines with an existing OS
VMware Fusion	Type 2 (Hosted)	Specifically designed for Mac users

- **vCenter – Centralized Management**

Once multiple ESXi hosts are running many VMs, someone needs to manage all of them together — that's vCenter's job:

- Centrally manages and monitors multiple ESXi hosts and their VMs
- Handles migration of VMs between hosts (e.g., moving workloads when one server needs maintenance)
- Automatically helps scale up resources — for example, if a business suddenly experiences a spike in sales/traffic and needs more VM instances, vCenter helps manage that scaling

2. VMware vSAN (Storage Area Network)

vSAN solves the storage problem for large-scale virtualized environments. In a cloud/enterprise setup, a single hard disk is never enough — you need a pool of storage made from many physical disks. Working Example:

- Suppose 100 individual hard drives, each 1 TB, are installed in a data center
- Instead of showing them as 100 separate 1 TB disks, vSAN combines them into a single unified pool — presenting it simply as "100 TB available"
- The internal complexity (which physical disk is being used) is completely hidden from the user — this abstraction is the core value of vSAN

If a viral trend causes thousands of users to suddenly upload large numbers of videos/photos at once (e.g., a social media "shorts" trend), storage demand spikes rapidly. vSAN allows the storage pool to be increased, reduced, and managed flexibly to absorb such spikes. vSAN also handles two related storage concepts:

Concept	Meaning	Purpose
Data Duplication (Replication)	Keeping multiple copies of data across different disks	Fault tolerance — if one disk fails, data is still available elsewhere
Data Deduplication	Removing redundant/duplicate copies of the same data	Saves storage space — similar to your phone detecting multiple near-identical selfies and keeping just one

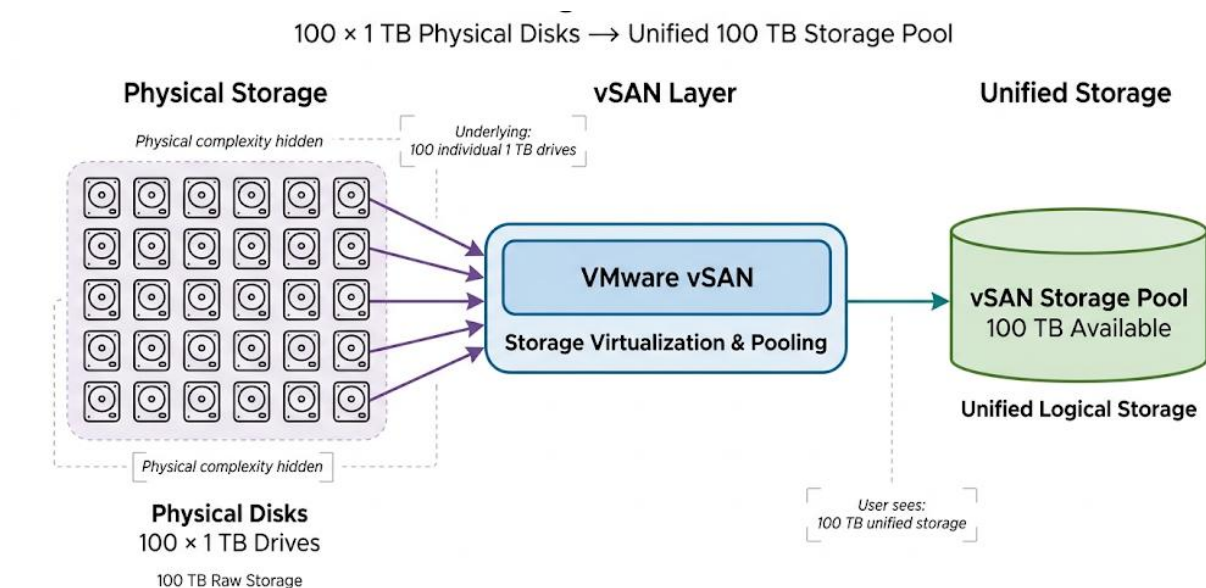


Figure: vSAN Storage Pool Abstraction

3. VMware NSX (Network Virtualization / SDN)

NSX handles network management through Software-Defined Networking (SDN) — managing and securing network traffic at a software level rather than through physical network hardware alone.

The core idea: Micro-segmentation Instead of running an entire application (frontend, business logic, database) on a single machine or network segment, NSX allows you to segment each component onto separate virtual machines/network zones:

Component	Runs On
Frontend (user-facing UI)	Separate VM/segment
Business logic	Separate VM/segment
Database (sensitive data)	Separate VM/segment

Consider a banking application. If everything ran in one place, an attacker who compromises the frontend could potentially reach sensitive financial/database data directly. By segmenting the network so that only authorized queries can reach the

business logic layer — and nothing can bypass it to reach the database directly — NSX significantly reduces the attack surface and protects sensitive data from unauthorized access.

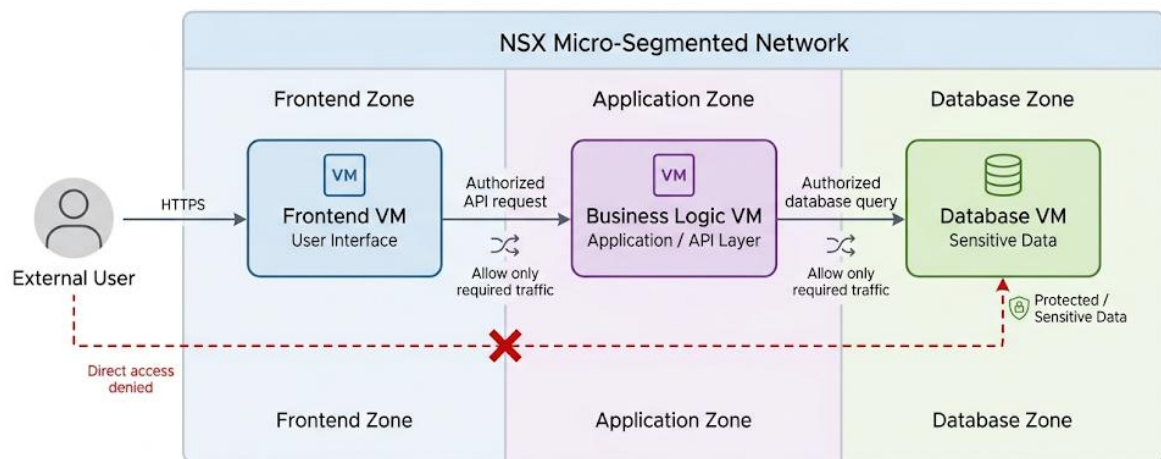


Figure: NSX Micro-Segmentation and Controlled Network Traffic

4. VMware Horizon (Virtual Desktop Infrastructure)

Horizon provides desktop virtualization — letting users securely access a full virtual desktop environment remotely, as if they were sitting at their office computer.

Real-world example: Telecom companies like Airtel or Jio have large customer support teams working from physical office desktops. When COVID-19 forced offices to shut down, employees couldn't physically access their office machines or the customer data/call systems tied to them. Using virtual desktop infrastructure (like Horizon), companies could let employees securely access their same office desktop environment and applications from home, without physically relocating hardware — while still keeping the systems secure.

Why an Ecosystem (Not Just One Tool)?

No single hypervisor can handle everything needed for enterprise virtualization. Each component of the ecosystem solves one specific problem:

Need	Tool	Solves
Create/run VMs	ESXi	Core hypervisor function
Manage/monitor multiple VMs	vCenter	Centralized VM management & migration
Handle massive storage needs	vSAN	Pooled, abstracted storage with redundancy/deduplication
Secure and segment networks	NSX	Software-defined networking, micro-segmentation
Enable remote desktop access	Horizon	Virtual desktop infrastructure (VDI)

Table: Summary

Component	Category	Core Function	Example Use Case
ESXi	Hypervisor (Type 1)	Creates/runs VMs directly on hardware	Enterprise data centers
vCenter	Management	Centralized monitoring & migration of VMs	Scaling VMs during traffic spikes
vSAN	Storage	Pools multiple disks into unified storage	Handling sudden storage demand (viral content uploads)
NSX	Networking (SDN)	Micro-segments network for security	Isolating a bank's frontend, logic, and database
Horizon	Desktop Virtualization	Provides secure remote virtual desktops	Work-from-home during COVID-19 (Airtel/Jio support staff)

Virtual Machines vs Containers

Virtualization (creating virtual machines) is the foundation of cloud computing, but a newer, more advanced approach — containerization — is now becoming extremely popular. Understanding the architectural difference between the two is one of the most important topics in cloud computing, frequently asked in interviews and exams.

The Starting Point: A Physical Server

Before understanding either VMs or containers, start with a plain physical server — think of it as a "full-fledged machine" containing its own infrastructure: CPU, RAM, cache, and secondary storage, along with network components (NIC cards). This is the same idea as a laptop, just at a much larger scale.

To manage and use this infrastructure, an operating system is installed on the machine — this becomes the host operating system (could be Windows, Linux, or Mac).

The core problem: If you just run one application directly on this machine, it uses only a small fraction of the available resources — the rest sit idle. This is like buying a 5-room house for a couple: most rooms remain empty most of the time. Since companies invest heavily in servers, they want maximum utilization, not idle resources. This is exactly the problem virtualization was invented to solve.

Virtual Machines (VM) Architecture

Instead of running one OS directly on the hardware, a hypervisor is installed, which creates multiple virtual machines on the same physical server — similar to renting out

different rooms of the house for different purposes. Each virtual machine is completely self-contained and includes:

Component Inside Each VM	Purpose
Guest Operating System	Its own separate OS (Windows, Linux, Mac — independent of others)
Application Code	The actual software/app running
Libraries & Runtime	Everything needed to execute the code
Drivers	Hardware interaction components

Because each VM gets its own full guest operating system, every VM believes it has an entire machine to itself — the hypervisor's job is to make each VM feel like it owns all the resources, while actually distributing the real hardware among all of them fairly.

Example: One VM might run a database application, another VM runs a website, another runs an email server — all on the same physical server, each fully isolated with its own OS.

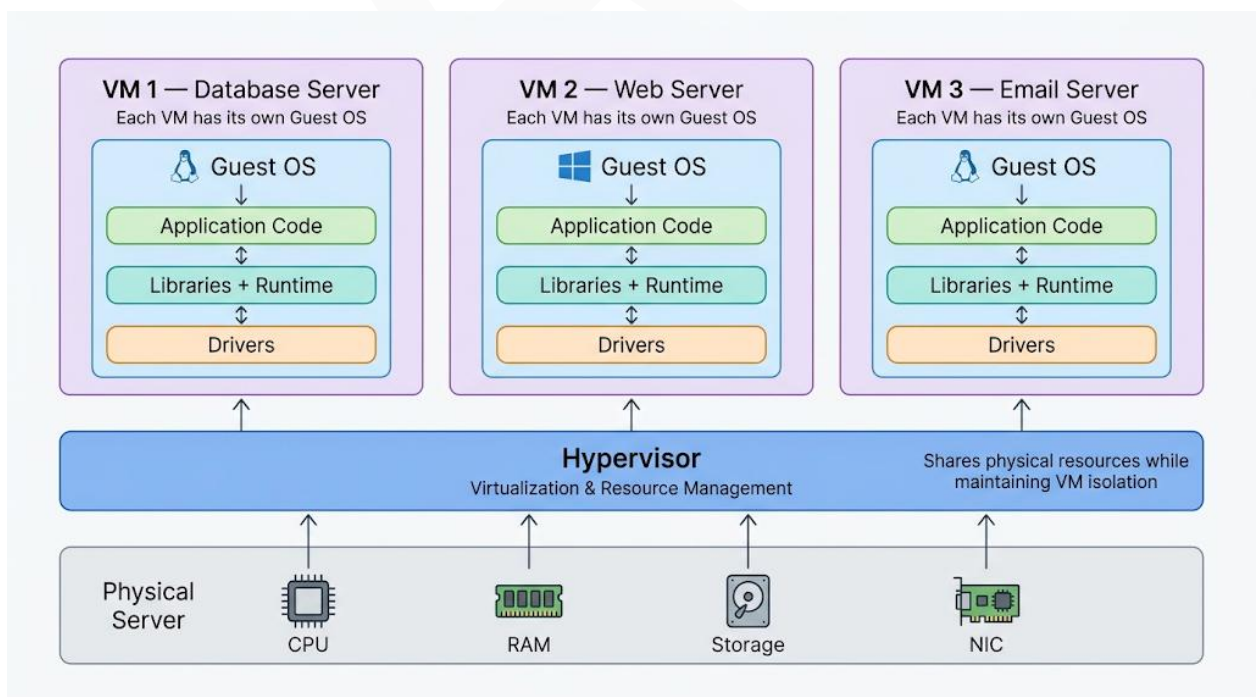


Figure: Virtual Machine Architecture with Hypervisor

Containers

Containers take a fundamentally different approach to solving the same "utilize resources properly" problem. Think of a shipping container used in ports for import/export — a sealed box that holds everything needed for goods to travel safely. In computing, a container similarly packages everything an application needs to run:

What Goes Inside a Container
Application code (e.g., website's HTML, CSS, static pages)
Libraries
Dependencies
Runtime
Drivers

Containers are created and managed using a container engine — the most popular example being Docker. Just like VMs, multiple containers can run side by side on the same machine — one website's code in one container, another application's code in a second container, and so on.

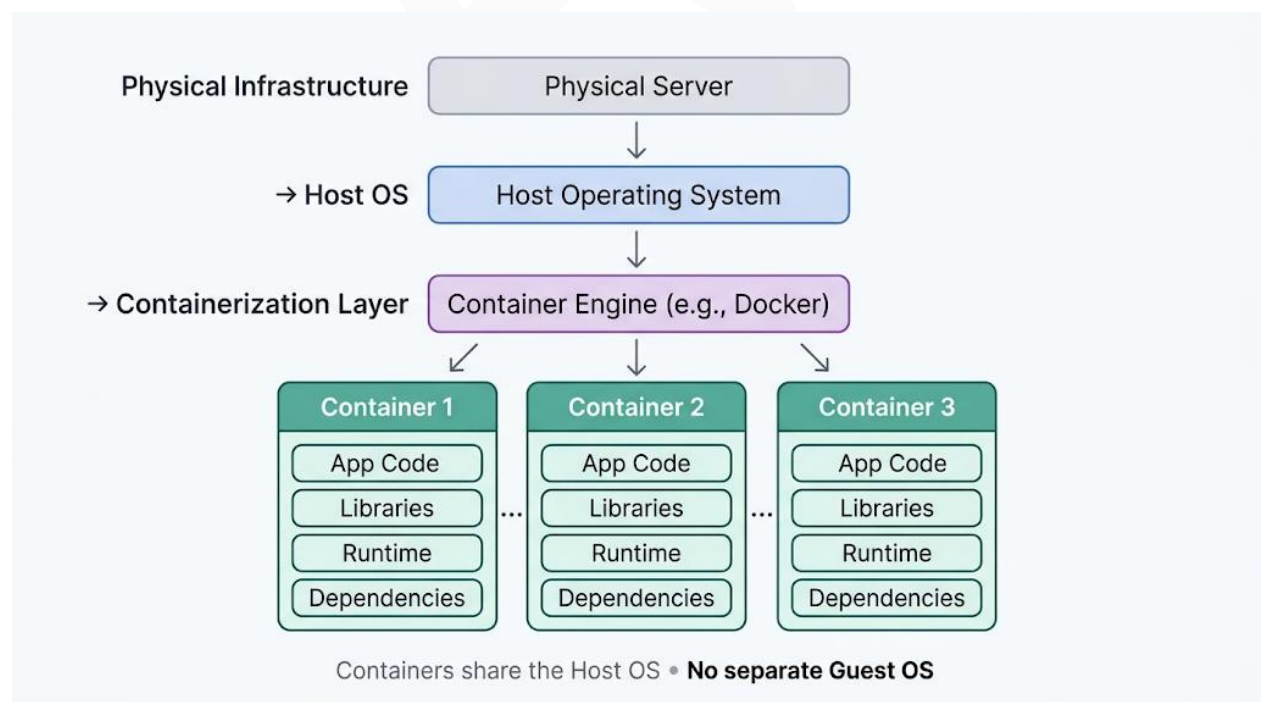


Figure: Container Architecture and Shared Host Operating System

Table: Architectural Difference

Aspect	Virtual Machines	Containers
Guest Operating System	Each VM has its own full OS	No separate OS — containers share the host machine's OS
Size/Weight	Heavier (each VM carries a full OS)	Much lighter (no OS overhead)
Startup Speed	Slower (has to boot a full guest OS first)	Very fast (just the container starts, no OS boot needed)
Portability	Harder to move due to size	Very easy to port between systems
Isolation	Stronger (separate OS per VM = more isolated)	Slightly less isolated (shared OS), but still secure — each container has its own private environment
OS Dependency	Can run different OS types on the same host	Container code must match the host machine's underlying OS (this is the main drawback)

Advantages and Drawback of Containers

Advantages:

- Lightweight — no guest OS to load, so containers start almost instantly
- Easy to port from one system/environment to another
- Good security/isolation — each container still has its own private application environment, even without a separate OS
- You can run thousands of containers efficiently on the same hardware, since there's no per-container OS overhead

Drawback:

- A container's code depends on the host machine's OS. If the host runs Windows, the containers running on it must be compatible with that same underlying OS — this restricts flexibility compared to VMs (where each VM can run a completely different OS independently).

Managing Containers at Scale: Docker & Kubernetes

Because a container isn't meant to just be created once and run untouched for years, ongoing management is needed:

- Docker – used to build, package, and run containers
- Kubernetes – used for orchestration, meaning it manages multiple containers together automatically: scaling up when there's heavy load on a website, scaling down when load decreases, restarting failed containers, applying patches/updates, and distributing traffic across multiple container instances

This is why containerization is usually discussed alongside Docker (to create/manage containers) and Kubernetes (to orchestrate/scale them) — together they solve not just "how to run an app in a container" but "how to reliably run and scale hundreds of containers in production."

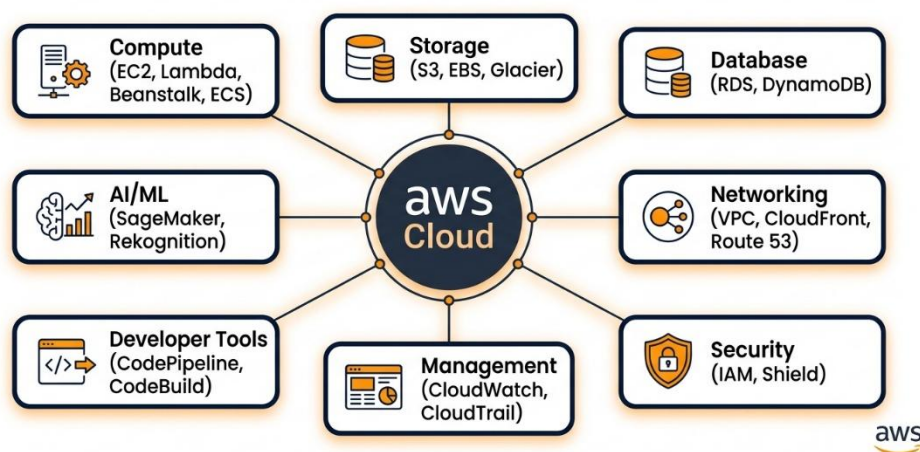
Table Summary

Concept	Virtual Machines	Containers
Core Idea	Virtualize entire machines with their own OS	Package only app + dependencies, share host OS
Managed By	Hypervisor	Container Engine (e.g., Docker)
Weight	Heavy	Lightweight
Boot Time	Slower (full OS boot)	Very fast
Isolation	Higher (separate OS per VM)	Slightly less, but still strong

OS Flexibility	Can run different OS per VM	Must match host OS
Scaling/Orchestration Tool	—	Kubernetes
Real-world Example	Running Windows, Linux, Mac apps side-by-side on one server	Running multiple websites/apps in isolated containers on one server

AWS – Major Components

Amazon Web Services (AWS) is the world's leading cloud platform, offering over 200 services in total. You don't need to memorize all of them, but the major components are frequently asked in interviews and exams. AWS services are grouped into categories based on what they do: computation, storage, database, networking, security, management, developer tools, and AI/ML.



1. Compute Services

Compute services provide the processing power needed to run your website/application code (PHP, .NET, Node.js, Java, Python, etc. — AWS is language-independent).

- **Amazon EC2 (Elastic Compute Cloud)**

Amazon EC2 is the core compute service in AWS and provides virtual servers where your website or application actually runs. You provision an EC2 instance, install an operating system, deploy your application code, and the server handles requests

from users. In simple terms, an EC2 instance is essentially a virtual machine (VM) that you rent from AWS.

- **AWS Lambda**

AWS Lambda provides serverless computing, which allows you to run code without having to provision or manage servers yourself. It is best suited for small, event-driven tasks, such as sending a confirmation email or SMS after a customer makes a purchase. Because you do not need a dedicated server running 24/7, Lambda can be more cost-efficient for lightweight tasks. However, "serverless" does not mean that there are no servers; AWS manages the underlying servers for you, and you are charged based on the time your code actually runs.

- **AWS Elastic Beanstalk**

AWS Elastic Beanstalk provides a Platform as a Service (PaaS) environment that simplifies application deployment. Instead of manually setting up and configuring servers, you simply upload your application code, and Beanstalk automatically handles tasks such as building, deploying, running, and scaling the application. This allows you to focus more on your code and less on managing the underlying infrastructure.

- **Amazon ECS (Elastic Container Service)**

Amazon ECS is a container management service that allows you to run applications packaged as Docker containers. Containers package an application together with its required dependencies, making it easier and more efficient to deploy and run applications. Unlike a traditional EC2 setup, containers do not require a complete virtual machine for each application, which can make them faster and more resource-efficient.

Table: Compute Service Summary

Compute Method	How It Works	Best For
----------------	--------------	----------

EC2	Full virtual server; you manage OS + app	Full control, custom setups
Lambda	Serverless; runs code only when triggered	Small event-driven tasks (notifications, etc.)
Elastic Beanstalk	PaaS; auto build/deploy/scale	Quick deployment without infra management
ECS	Container-based (Docker)	Lightweight, portable, faster deployment

2. Storage Services

Computation is useless without storage — every application needs a place to store its data, files, and backups.

- **Amazon S3 (Simple Storage Service)**

Amazon S3 is a storage service used to store static data such as HTML and CSS files, images, videos, and documents. It is especially useful for websites and e-commerce applications that need to store and serve large numbers of files, such as product images. S3 provides reliable and fast access to this static content, making it easy for applications and users to retrieve files when needed.

- **Amazon EBS (Elastic Block Store)**

Amazon EBS provides block-level storage that works alongside Amazon EC2. An EBS volume can be attached to an EC2 instance and used to store files, images, videos, application data, and other information required by the application running on the server. Because the storage is directly connected to the EC2 instance, applications can access the data quickly.

- **Amazon Glacier**

Amazon Glacier is designed for data archiving and long-term backups, such as storing old transaction records, historical documents, or data that is rarely accessed. Since

archived data does not usually need to be retrieved immediately, Glacier provides a lower-cost storage option. This allows frequently used data to remain in services such as S3 or EBS for fast access while older, less frequently accessed data is moved to cheaper archival storage.

Table: Storage Service Summary

Storage Service	Purpose	Speed/Cost
S3	Static files (images, HTML, CSS)	Fast, moderate cost
EBS	Block storage attached to EC2	Fast, works with active servers
Glacier	Archiving/long-term backup	Slow access, very low cost

3. Database Services

Beyond files, applications need structured data storage — this is where database services come in.

- **Amazon RDS (Relational Database Service)**

Amazon RDS is used for SQL-based relational data, where information is organized into structured tables with rows and columns. For example, an e-commerce website can use RDS to store product details such as price, size, category, and product ID in tables. These records can also be linked to other information, such as the product images stored in Amazon S3.

- **Amazon DynamoDB**

Amazon DynamoDB is a NoSQL database designed for data that does not need to follow a fixed table structure. It is well suited for flexible and high-speed applications where data can have different attributes or change over time. For example, an application may use DynamoDB when it needs to handle large amounts of rapidly changing data without requiring the rigid structure of a traditional relational database.

Table: Database Services Summary

Data Type	Service Used
Structured (tables, rows/columns)	RDS
Unstructured/NoSQL	DynamoDB

4. Networking & Content Delivery

These services manage how data flows between users and your infrastructure.

- **Amazon VPC (Virtual Private Cloud)**

Amazon VPC allows you to create and control a private network within AWS. You can divide this network into public and private subnets based on what needs to be accessible from the internet. A public subnet can contain components that need to receive internet traffic, such as a load balancer, while private subnets can contain sensitive backend resources such as EC2 instances and RDS databases. This separation means users do not need direct access to the backend systems, which helps improve security. For example, a user request can first reach a public-facing component, which then communicates with the backend resources inside the private subnet.

- **Amazon CloudFront**

Amazon CloudFront is a Content Delivery Network (CDN) that helps reduce the time it takes for users to access website content. It works by caching frequently requested content, such as images, videos, CSS files, and JavaScript files, at edge locations around the world. When a user requests this content, CloudFront can deliver it from a location geographically closer to the user instead of retrieving it from the application's main server every time. This reduces latency and makes websites and applications load faster.

- **AWS Route 53**

AWS Route 53 is AWS's DNS (Domain Name System) service. DNS translates a human-readable domain name, such as `www.xyz.com`, into the IP address or

destination associated with the service hosting the website. When a user enters a domain name in their browser, Route 53 helps determine where that request should go. In this way, it acts like the internet's directory service, helping direct users to the correct application or server.

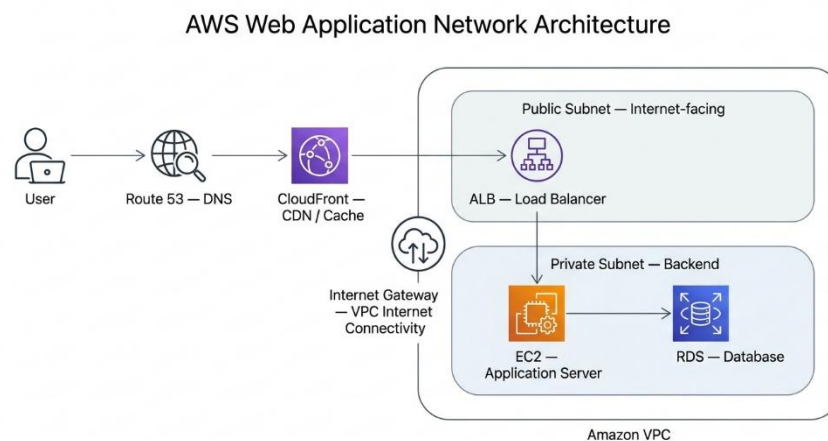


Diagram might have few tweaks

5. Security & Identity Management

Since companies store sensitive customer/employee/transaction data, security is one of the most critical aspects of any cloud platform.

- **AWS IAM (Identity and Access Management)**

AWS IAM is used to manage who can access AWS resources and what they are allowed to do. It manages users, credentials, roles, and permissions, allowing administrators to control access to services such as EC2, S3, and RDS. For example, an administrator can give one user permission to view S3 files while allowing another user to create and manage EC2 instances. IAM also helps organizations monitor and audit access so they can understand which identities are accessing which resources and what actions they are permitted to perform.

- **AWS Shield (DDoS Protection)**

AWS Shield protects applications and AWS resources from DDoS (Distributed Denial of Service) attacks. A DDoS attack sends a large amount of malicious traffic to a service in an attempt to overwhelm it and prevent legitimate users from accessing it.

AWS Shield helps detect and mitigate this traffic, keeping applications available to normal users. It is best understood as a DDoS protection service, rather than a traditional firewall.

6. Management & Monitoring

Since cloud resources are shared across many companies (multi-tenant), proper oversight tools are essential.

- **Amazon CloudWatch**

Amazon CloudWatch is a monitoring and observability service that helps you track the performance and health of AWS resources and applications in real time. It collects information such as CPU usage, memory usage, request rates, response times, and other performance metrics. For example, you can use CloudWatch to monitor an EC2 instance and identify whether it is experiencing unusually high CPU usage or receiving a sudden increase in requests.

- **AWS CloudTrail**

AWS CloudTrail is used to record and track activity within an AWS account, particularly API calls and actions performed by users, applications, or AWS services. These logs help organizations with auditing, security, compliance, and troubleshooting. For example, CloudTrail can show who deleted an AWS resource, when the action occurred, and what API request was made.

CloudWatch → What is happening? → Monitors performance and resource usage.

CloudTrail → Who did what? → Records account activity and API actions.

7. Developer Tools (DevOps)

DevOps is a development approach that combines Development and Operations to automate and streamline the software development and deployment process. The goal is to make it easier and faster to develop, test, and release applications while reducing the need for manual work.

- **CI/CD (Continuous Integration/Continuous Deployment)** is a key part of DevOps. It automatically takes code changes through stages such as **building, testing, and deploying** the application. This means developers can make updates to an application and have those changes automatically tested and delivered without manually deploying them each time.
- Two important AWS DevOps tools are **AWS CodePipeline** and **AWS CodeBuild**. **CodePipeline** automates the overall software release process by moving code through different stages, while **CodeBuild** automatically compiles and builds the code and runs tests to make sure it works correctly.

In simple terms, instead of manually deploying every update to a website or application, DevOps and CI/CD create an automated pipeline that takes each code change from development through testing and finally to deployment.

8. Machine Learning & AI

AWS provides ready-made infrastructure and tools to build/run AI and ML models without needing to build the underlying hardware yourself.

- **Amazon SageMaker** – build, train, and deploy machine learning models
- **Amazon Rekognition** – image and video analysis (e.g., object/face detection)
- Other tools support sentiment analysis, text analysis, and various AI-driven applications

Security and AI/ML are considered the two biggest growth areas for cloud providers going forward — the provider offering the strongest security and best AI/ML integration is expected to lead the market.

Table: Summary of AWS Component

Category	Key Services	Core Purpose
Compute	EC2, Lambda, Elastic Beanstalk, ECS	Run application code/processing

Storage	S3, EBS, Glacier	Store files, active data, and backups
Database	RDS, DynamoDB	Store structured (SQL) and unstructured (NoSQL) data
Networking	VPC, CloudFront, Route 53	Secure network division, faster content delivery, DNS routing
Security	IAM, Shield	Access control and DDoS protection
Management	CloudWatch, CloudTrail	Monitoring and auditing
Developer Tools	CodePipeline, CodeBuild	Automate build/deploy (DevOps/CI-CD)
AI/ML	SageMaker, Rekognition	Build and run AI/ML models

Cloud Security – Standards & Frameworks

Cloud gives flexibility, scalability, and pay-as-you-go cost benefits — but many organizations still hesitate before moving their data to the cloud because of **security concerns**:

- You don't know exactly where your data is physically stored
- If data is lost or breached, it's unclear who is accountable and whether legal action is possible
- Uncertainty around misuse of data and lack of clear compliance/regulatory oversight

This is why cloud providers need to follow recognized security standards and frameworks — so users can trust that their data is handled properly, backed by clear rules (similar to how a country's constitution defines punishments so justice is consistent and not arbitrary).

ISO Standards for Cloud Security

These extend traditional information security standards (ISO 27001/27002) specifically into the cloud environment.

ISO/IEC 27017 – Cloud Security Guidelines

- Provides security control guidelines specifically for both cloud providers and cloud users
- Defines how security controls should be designed and implemented
- Covers proper auditing and accounting — so if an auditor reviews the system, clear records of how data/resources were used are available
- Also defines what practices/controls a cloud user should follow

ISO/IEC 27018 – Protection of Personal Data in the Cloud

- Focuses specifically on personal data (private user information) stored in the cloud
- Clearly defines how personal data should be stored, including encryption requirements and handling guidelines

Why ISO certification matters: If a company (even a small startup) wants to become a cloud provider, getting ISO certified proves it has followed proper guidelines. This certification builds **trust** with customers — since it can be shown/marketed as proof of compliance.

Table: Other Key Frameworks

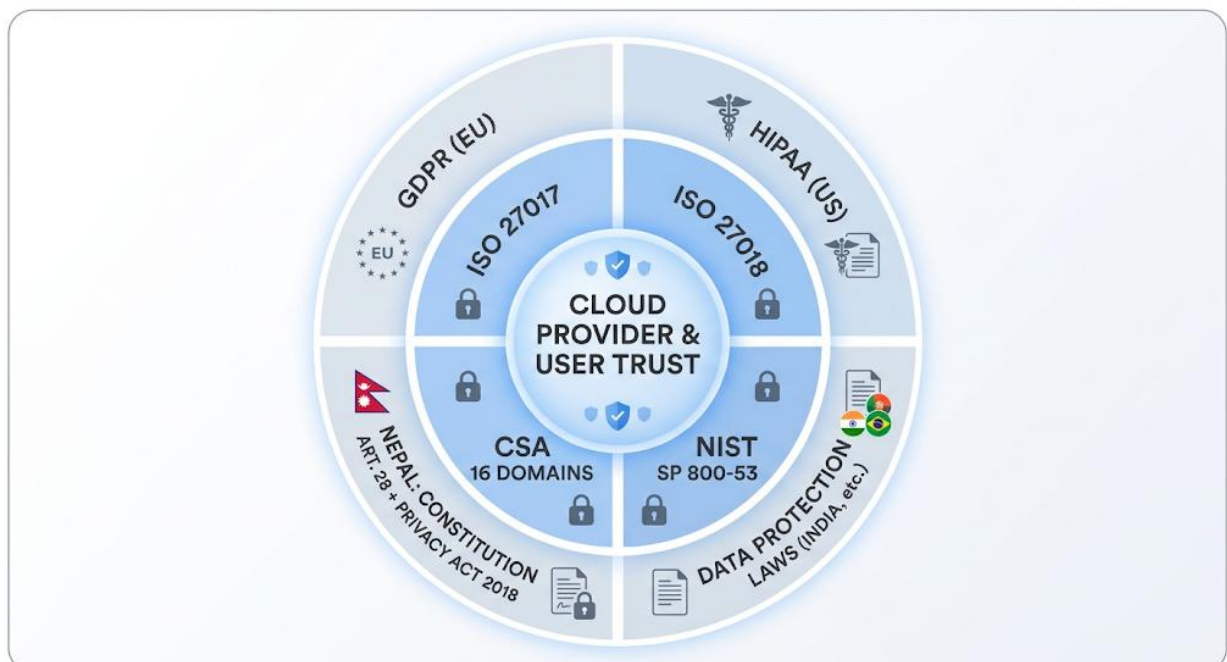
Framework	Focus
NIST SP 800-53 (National Institute of Standards and Technology)	Provides standardized security controls and requires organizations to classify the impact of any security incident as low, medium, or high , so data handling responses are appropriately scaled
CSA (Cloud Security Alliance) Controls	Defines 16 security domains that a cloud provider should fulfill to be considered trustworthy and secure
SOC (System and Organization Controls)	Independent audit-based certification showing an organization meets defined security/operational criteria

CIS Benchmarks	Configuration best-practice guidelines for securing systems and cloud services
-----------------------	--

Government Data Protection Regulations

Apart from technical standards, governments enforce legal regulations that cloud providers and companies must comply with when handling personal data:

- **GDPR (General Data Protection Regulation)** – European Union's law on how personal data/information must be collected, stored, and protected
- **HIPAA** – U.S. regulation ensuring health-related and personal information is protected; allows users to sue if their data is breached or sold to third parties without consent
- Similar **digital data protection laws** exist in other countries (e.g., India's Digital Personal Data Protection Act, 2023) — each defining consent requirements, breach notification duties, and penalties for misuse



Nepal does not yet have a single, dedicated, comprehensive data protection or cloud security law like GDPR or HIPAA. The current legal landscape is fragmented:

- **Constitution of Nepal, Article 28** – recognizes the **right to privacy** as a fundamental right, which implicitly covers data protection principles, though no detailed enforcement mechanism is specified
- **Electronic Transaction Act, 2006** – Nepal's main cyber law; addresses electronic transactions and cybercrime but has only limited, indirect provisions on data protection
- **Right to Information Act, 2007** – restricts public agencies from disclosing personal information, offering a narrow layer of privacy protection
- **Individual Privacy Act, 2018 (2075 B.S.)** – Nepal's dedicated privacy legislation, covering collection, use, and protection of personal information, though it predates the cloud-specific frameworks discussed above and is not cloud-focused
- A draft **Information Technology Bill** has been under discussion, intended to modernize IT/cyber regulation, but it does not yet comprehensively address cloud data protection

Practical implication for Nepal: Since no ISO/CSA-equivalent local regulatory body certifies cloud providers domestically, most Nepali businesses and government bodies relying on cloud services (AWS, Azure, GCP) depend on the **international certifications (ISO 27017/27018, SOC, GDPR-compliance)** of these providers as the practical basis for trust, rather than a dedicated Nepali cloud law.

Table: summary

Type	Name	Purpose
Standard	ISO 27017	Cloud security guidelines for providers & users
Standard	ISO 27018	Protection of personal data in the cloud

Framework	NIST SP 800-53	Impact-based security control classification
Framework	CSA Controls	16 domains for cloud provider trustworthiness
Audit/Cert	SOC, CIS Benchmarks	Independent compliance verification
Regulation	GDPR (EU)	Personal data protection law
Regulation	HIPAA (US)	Health & personal data protection, allows legal recourse
Nepal	Article 28, Electronic Transaction Act 2006, Individual Privacy Act 2018	Fragmented privacy protections; no dedicated cloud data law yet

Security, Trust, and Privacy in Public Cloud

Security, trust, and privacy are some of the most important challenges in cloud computing. Cloud computing provides organizations with scalable computing resources, storage, applications, and services without requiring them to own and maintain all the underlying infrastructure. However, this advantage comes with an important trade-off: the customer gives up some direct control over the infrastructure and depends on a cloud service provider (CSP) to protect and manage resources correctly.

In a traditional data center, an organization may control the physical servers, network, storage devices, operating systems, security mechanisms, and administrators. In a public cloud, these responsibilities are distributed between the cloud provider, the customer, and sometimes additional third-party service providers. The customer's data may be stored on infrastructure that the customer cannot physically see, applications may execute inside virtual machines, and the same physical infrastructure may be shared by many unrelated customers. Therefore, cloud security is not simply about protecting a server or encrypting a file. **It is about protecting data, applications, identities, communication, virtualized infrastructure, and services while operating in an environment that the customer does not completely control.**

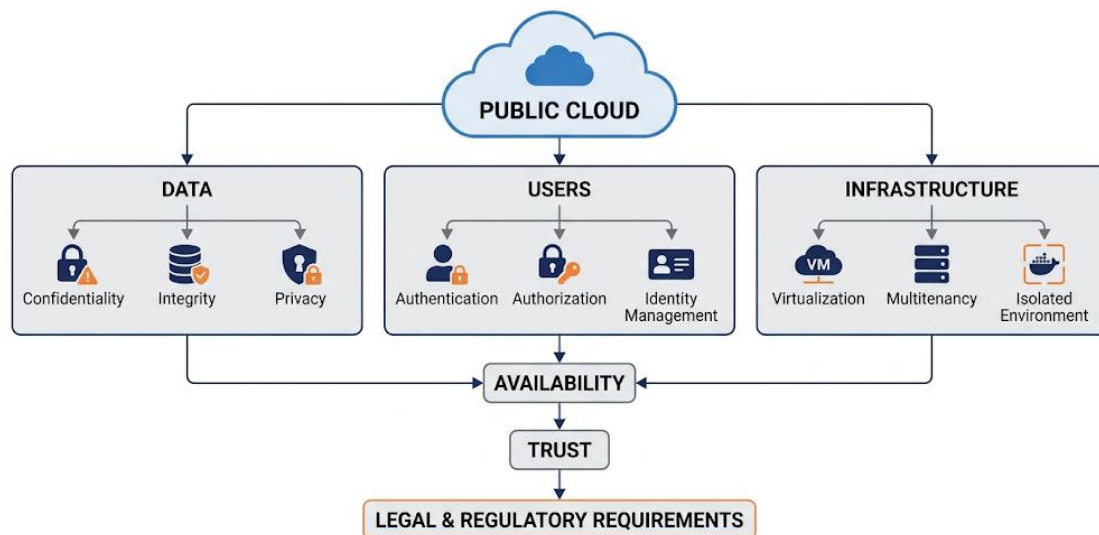


Figure: Security, Trust, and Privacy Dimensions of Public Cloud

Why Security Becomes More Difficult in the Cloud

The fundamental security problem in cloud computing comes from loss of direct control. Suppose an organization has a sensitive customer database. In its own data center, the organization can decide where the server is located, who can physically access it, which administrators can log into it, how the network is configured, and how the storage devices are protected.

When the same database is moved to a public cloud, the organization still owns the information, but it no longer directly controls every layer of the environment in which that information exists.

The data may be replicated across several storage systems. The application may run inside a virtual machine. The virtual machine may run on a physical server shared with other customers. The physical infrastructure is managed by the cloud provider. The cloud service itself may also depend on other services supplied by third parties.

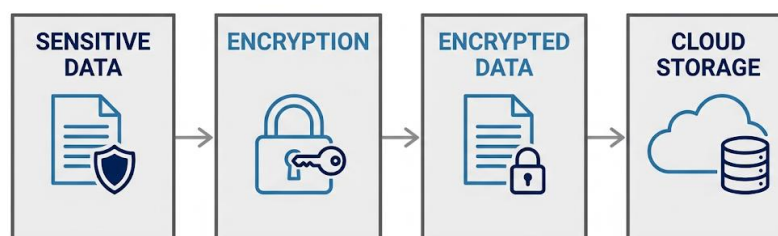
This creates a fundamental cloud-security principle: **The customer owns or controls the data logically, but the cloud provider controls much of the underlying environment in which that data is stored and processed.** That separation between ownership and control is responsible for many cloud security, trust, and privacy concerns.

The Three States of Data

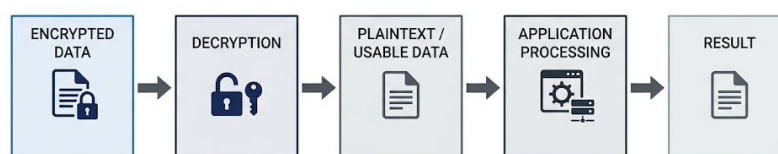
One of the easiest ways to understand cloud security is to look at what happens to data throughout its lifecycle. Data normally exists in three important states:

State	Meaning	Typical protection
Data at rest	Data stored in disks, databases, object storage, backups, etc.	Encryption, access control
Data in transit	Data moving between user, application, and cloud services	TLS, secure network protocols
Data in use	Data currently being processed by an application	Access control, isolation, trusted execution, advanced cryptography

Traditional security mechanisms are relatively effective for data at rest and data in transit. The difficult problem is data in use. For example, imagine that a company stores an encrypted database in the cloud.



An attacker who steals only the encrypted storage should not be able to understand the information without the necessary cryptographic key. However, eventually an authorized application needs to use the data. To perform normal computation, the application generally needs the information in a usable form. Therefore:



During this processing stage, the information exists in memory in a form that the application can understand. This creates a deeper cloud security problem.

Virtualization and the Data-in-Use Problem

Public clouds make extensive use of virtualization. Applications commonly execute inside virtual machines rather than directly on physical servers. The hypervisor, also called a virtual machine monitor, is responsible for managing virtual machines and controlling how they use physical CPU, memory, storage, and other resources. The problem described by the textbook is that although the customer's application may legitimately decrypt sensitive information in memory, the underlying virtualized environment is managed by the cloud provider. A highly privileged virtualization layer can potentially have visibility into or control over resources used by virtual machines. This creates a security boundary that does not normally exist in the same way when an organization completely controls its own physical infrastructure. The important point is not that every cloud provider simply reads customers' memory. The important point is that the customer has less direct control over the execution environment.

Therefore, encryption alone cannot solve every cloud security problem. Traditional encryption protects information when it is stored or transmitted, but protecting information while it is actively being processed is considerably more difficult. This does not mean the CSP is automatically reading customer data; rather, the customer has less direct control over the environment where its sensitive data is being processed.

This is one reason why advanced technologies such as homomorphic encryption, confidential computing, trusted execution environments, memory protection, and stronger virtualization isolation are important areas of cloud security research.

Confidentiality, Integrity, Availability, and Privacy

Before going deeper, four security properties must be clearly separated. Confidentiality means preventing unauthorized people or systems from viewing information. For example, if a cloud database contains customer credit-card information, unauthorized

users should not be able to read it. Encryption is one of the major mechanisms used to provide confidentiality.

Integrity means ensuring that information has not been improperly changed, deleted, or manipulated. For example, suppose a bank stores:

Account Balance = \$5,000

An attacker changes it to:

Account Balance = \$500,000

The information may still be confidential, but its integrity has been violated. Integrity mechanisms include cryptographic hashes, digital signatures, authentication mechanisms, access control, audit logs, and other validation techniques.

Availability means that authorized users can access resources and services when they need them. A cloud application can have excellent confidentiality and integrity but still fail as a service if legitimate users cannot access it. For example: Customer -> Cloud Application -> Server overloaded -> Request fails. Availability attacks such as Denial-of-Service (DoS) and Distributed Denial-of-Service (DDoS) attacks attempt to make services unavailable by exhausting resources or overwhelming network/application capacity.

Privacy is related to confidentiality but is broader. Confidentiality asks: Can an unauthorized party see the information? Privacy asks: How is information collected, stored, processed, shared, and used, and is that use appropriate and permitted? For example, suppose a cloud provider legitimately has access to certain information for providing a service but later uses the information for an unrelated purpose. There may not have been a traditional "hack," but there can still be a privacy violation.

A useful distinction is:

Confidentiality → Prevent unauthorized disclosure

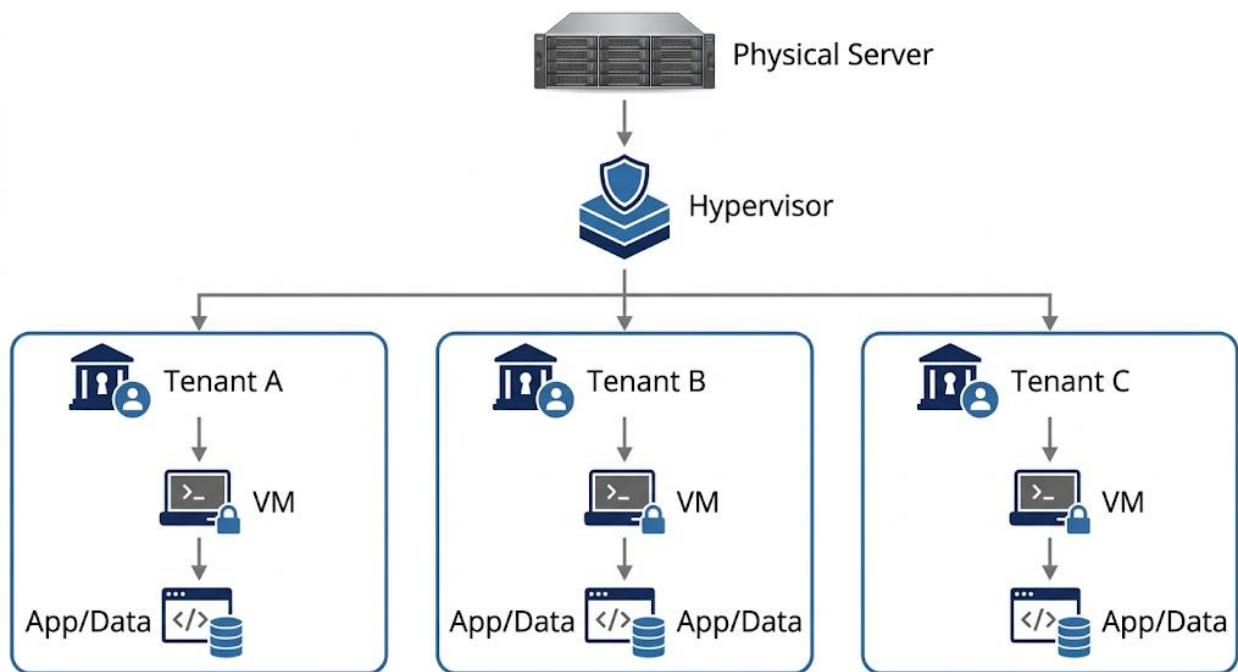
Integrity → Prevent unauthorized modification

Availability → Ensure authorized access when needed

Privacy → Ensure information is handled and used appropriately

Public Cloud and Multitenancy

One of the defining characteristics of public cloud computing is multitenancy. A public cloud provider typically serves many customers using shared physical infrastructure. For example:



Each customer sees their own logical environment, but the underlying hardware may be shared. This is one of the major reasons cloud computing can be economically efficient. The provider does not need a completely separate physical server for every customer. However, sharing infrastructure creates security challenges. The provider must ensure that:

- one customer's data cannot be accessed by another customer;
- one customer's virtual machine cannot improperly interfere with another VM;
- one customer cannot consume all shared resources;
- vulnerabilities in virtualization technology cannot expose other tenants;
- malware cannot easily spread between tenants;
- administrative privileges are properly controlled.

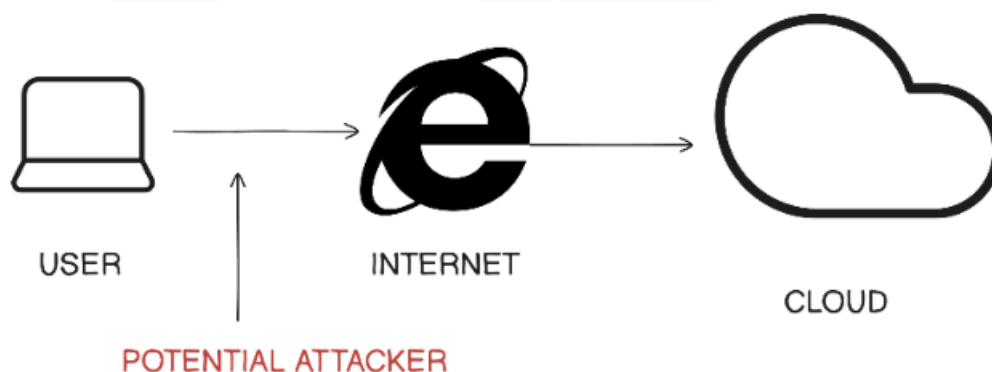
Note: Virtualization provides isolation, but logical isolation is not identical to complete physical separation.

The Noisy-Neighbor Problem

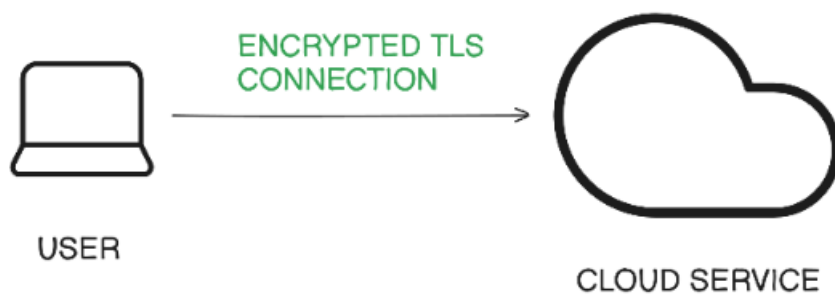
Suppose Tenant A suddenly consumes huge amounts of CPU, memory, disk I/O, or network bandwidth. If the cloud provider's resource-management mechanisms are not effective, Tenant B may experience degraded performance even though Tenant B has done nothing wrong. This is often described as the noisy-neighbor problem.

Network Security in the Cloud

Cloud computing depends heavily on remote access. Users normally access cloud applications and services over networks, often through the public Internet. Therefore, data must be protected while moving between the user and the cloud.



An attacker positioned somewhere on the communication path may attempt to intercept or manipulate traffic. This is why protocols such as Transport Layer Security (TLS) are important. TLS provides encrypted communication and supports authentication using digital certificates.

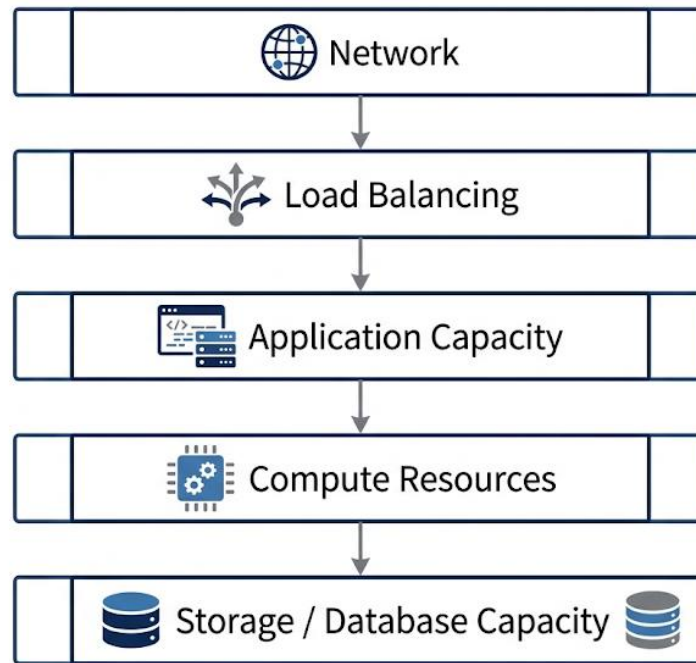


Instead of an attacker seeing the actual application data, the attacker should see protected ciphertext. TLS therefore helps protect data in transit. However, remember the earlier distinction: **TLS protects the communication channel; it does not automatically protect information after the cloud application has received and decrypted it.** This is why network security and application/data security must be treated as separate but connected problems.

Availability and DoS/DDoS Attacks

Cloud services are expected to remain available even when traffic increases or attacks occur. A Denial-of-Service (DoS) attack attempts to consume enough resources that legitimate users cannot obtain the service.

When many compromised devices participate in the attack, it becomes a Distributed Denial-of-Service (DDoS) attack. Cloud environments have mechanisms for handling large traffic volumes and scaling resources, but scalability does not make availability attacks impossible. The network itself can become a bottleneck. Application resources can also become exhausted. Therefore, cloud availability requires protection at several levels:



Malware and Shared Environments

Cloud users may connect from very different environments. One user might access a service from a well-protected corporate computer behind firewalls and endpoint security. Another might access it from an insecure public computer. This matters because compromised client systems can become a source of attacks. Cloud providers therefore need to consider not only attacks originating from outside the cloud, but also threats that enter through legitimate users, compromised accounts, applications, or services.

Data Storage and Data Location

Another major cloud concern is lack of visibility into where data is stored. A customer may upload a file to a cloud storage service and simply see: myfile.pdf The provider may customer does not necessarily know the physical disk or server containing that data. The provider may automatically:

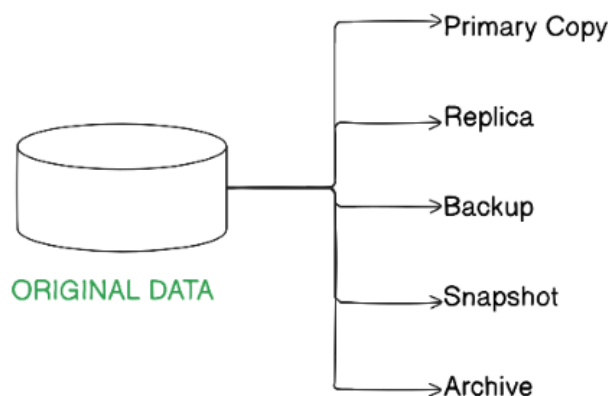
- replicate the data;
- move it between storage systems;
- create backups;

- create snapshots;
- distribute it across data centers;
- migrate workloads for performance or maintenance.

This is one of the benefits of cloud abstraction: the user does not need to manage the underlying infrastructure. But that abstraction also creates a trust problem. The customer has to trust the provider to handle the information according to the agreed policies and applicable laws.

Data Deletion and Data Remanence

Deleting data in the cloud can be more complicated than deleting a file from your own computer. Consider:



A user may delete the primary copy, but other copies may continue to exist for some period. This is called data remanence: residual information that remains after deletion. This does not automatically mean the provider is acting improperly. Backups and replicas are necessary for reliability and disaster recovery. The security question is: What happens to all copies of the information when the customer asks for deletion or terminates the service? Therefore, cloud contracts and policies should address data retention, deletion, backup management, and secure disposal.

Access Control: Who Can Do What?

Access control is another fundamental cloud security mechanism. It determines: Which user or system is allowed to perform which operation on which resource?

For example:

Alice → Read → File A

Bob → Read + Write → File B

Administrator → Manage → Cloud Infrastructure

Guest → No access → Confidential Database

A good cloud access-control system must be able to support potentially millions of users and huge numbers of resources. It should also provide fine-grained authorization. Instead of simply: User = Allowed the system may need to evaluate:

User + Role + Department + Resource + Operation + Location + Time + Security policy

before deciding whether access should be allowed.

Authentication vs Authorization

These two concepts are extremely important and frequently confused.

Authentication: Who are you?

For example, logging in with a password, security key, biometric factor, or another authentication mechanism.

Authorization: What are you allowed to do?

For example, after Alice logs in successfully, the system may determine that Alice can read a particular file but cannot delete it.

A system can have strong authentication but poor authorization. For example, if an attacker cannot log in without a password, authentication is working. But if every authenticated employee can access every database, authorization is weak.

Insider Threats

One of the most important cloud-security threats is the insider threat. Cloud providers have employees and administrators who require significant privileges to maintain infrastructure. A malicious administrator could potentially misuse those privileges. But

insider threats do not necessarily require malicious intent. An employee could accidentally:

- expose sensitive data;
- configure a storage system incorrectly;
- grant excessive permissions;
- leak credentials;
- disable a security control.

Therefore, insider threats include both malicious insiders and negligent insiders. The risk is particularly serious in cloud environments because a privileged account may have access to infrastructure serving many customers. Important protections include:

- least-privilege access;
- separation of duties;
- strong authentication;
- privileged-access management;
- logging and monitoring;
- auditing;
- background checks and organizational controls;
- cryptographic protection that limits what administrators can see.

Usage Creep and Privacy

A particularly interesting privacy issue is usage creep. Suppose a customer provides data to a cloud provider for a specific purpose. For example: "Store this customer database and provide analytics for our application." The customer expects the data to be used for that purpose. If the provider later uses the same information for another purpose that the customer did not expect or authorize, this becomes a privacy concern.

Usage creep: *When a cloud provider uses customer data beyond its original agreed purpose, creating a potential privacy violation*

This demonstrates why privacy is broader than encryption. Even perfectly encrypted communication does not solve the question: What is the provider allowed to do with information it legitimately receives?

Identity Protection and Re-identification

Cloud services process enormous amounts of information about users.

Examples include:

- search queries;
- account activity;
- payment activity;
- location patterns;
- browsing behavior;
- communication patterns;
- application usage.

Even when explicit identifiers such as a person's name are removed, the remaining information may still identify the individual when combined with other datasets.

Identity protection and re-identification: Privacy in cloud computing is not guaranteed simply by removing direct identifiers such as names or account numbers. Different pieces of seemingly anonymous data can be combined to infer or reconstruct a person's identity, especially when cloud providers process large-scale behavioral, location, search, and usage data. This process is known as re-identification and demonstrates that privacy depends not only on protecting data from unauthorized access, but also on controlling what can be inferred from legitimately collected data. Furthermore, cloud analytics can create detailed user profiles, raising deeper concerns about consent, transparency, purpose limitation, data minimization, and regulatory control over how personal information is analyzed and used.

The Cryptography Problem

Cryptography protects cloud data by transforming readable plaintext into unreadable ciphertext using an encryption key. Data can then be stored or transmitted without exposing its original content to unauthorized parties. However, encryption creates a fundamental challenge in cloud computing: encrypted data cannot normally be processed directly. If a cloud service needs to perform calculations, search, analysis, or other operations on protected data, it generally has to decrypt the data first, perform the computation on the plaintext, and potentially encrypt the result again. This creates a plaintext exposure point inside the cloud environment, meaning that although the data is protected while stored or transmitted, it may become accessible to the cloud infrastructure during processing.

This creates a gap in traditional encryption: data is protected at rest and in transit, but not necessarily while it is being actively processed. The fundamental question therefore becomes: *Can computation be performed on encrypted data without revealing the underlying plaintext to the cloud provider?* Homomorphic encryption addresses this problem by allowing certain computations to be performed directly on ciphertext, producing an encrypted result that, when decrypted, corresponds to the result of performing the same computation on the original plaintext. Thus, the goal is to maintain confidentiality throughout the entire data lifecycle—storage, transmission, and computation.

Homomorphic Encryption

Homomorphic encryption (HE) is a form of encryption that allows a cloud system to perform computations on encrypted data without first decrypting it. This is important because, with conventional encryption, the cloud can protect data while it is stored or transmitted, but it generally needs the plaintext during computation. Homomorphic encryption changes this model by making the ciphertext itself computationally usable. The cloud can manipulate encrypted values and produce an encrypted output, while the secret key remains with the data owner.

The basic idea can be understood as:

Plaintext → Encrypt → Ciphertext → Cloud computes → Encrypted result → User decrypts → Correct result

Suppose a user has two values, 5 and 3, but does not want the cloud provider to know those values. The user encrypts them:

$E(5)$ and $E(3)$

The cloud receives only these encrypted values. Under a suitable homomorphic encryption scheme, it can perform an operation on them:

$E(5) + E(3) \rightarrow E(8)$

The cloud never needs to learn that the underlying values were 5 and 3. When the user receives the encrypted result and decrypts it:

$D(E(8)) = 8$

The important property is that the computation performed on the ciphertext corresponds to a meaningful computation on the underlying plaintext.

The main security advantage is that the cloud does not need access to the customer's plaintext data during computation. This addresses an important weakness of traditional encryption in cloud computing.

With conventional encryption:

Encrypted Data → Decrypt → Plaintext → Compute

The plaintext exists inside the cloud environment while the computation is taking place. If the cloud infrastructure, privileged administrator, compromised process, or other component can access that plaintext, the confidentiality protection provided by encryption is reduced during that stage.

With homomorphic encryption:

Encrypted Data → Compute on Ciphertext → Encrypted Result

The sensitive information can remain encrypted throughout the cloud's processing. The cloud can provide computational resources without necessarily being given the ability to read the underlying data.

This is particularly valuable when the cloud provider is not fully trusted with the customer's information. The customer can outsource computation while retaining control of the decryption key. In this sense, homomorphic encryption separates two capabilities that are normally closely connected: the ability to compute on data and the ability to read the data.

Partially Homomorphic vs. Fully Homomorphic Encryption

Not all homomorphic encryption systems provide the same computational capabilities.

Partially homomorphic encryption (PHE) supports a limited type of operation, or a restricted form of computation. For example, a particular scheme may allow additions on encrypted values, while another may support multiplication. This means the encryption scheme preserves a specific mathematical relationship between operations on plaintext and operations on ciphertext. The limitation is that you cannot necessarily perform arbitrary computations. If an application requires operations that the scheme does not support, that encryption system may not be suitable.

Fully homomorphic encryption (FHE) goes much further. It is designed to support arbitrary computations, meaning that encrypted data can, in principle, undergo combinations of additions, multiplications, and more complex computational operations without requiring decryption during the process.

$E(a) + E(b) \times E(c) \rightarrow \text{Encrypted Result}$

The cloud performs the computation while the values remain encrypted, and the authorized user eventually decrypts the final result.

The distinction is therefore:

- PHE: supports a restricted class of computations.
- FHE: supports general computation over encrypted data.

Why FHE Is Difficult

The major challenge with FHE is performance. Performing computation on ciphertext is substantially more complicated than performing the same computation on ordinary plaintext. Encryption schemes introduce mathematical operations and additional data structures that can make ciphertexts much larger and computations much more expensive.

This creates a fundamental trade-off: More privacy → more computational overhead → potentially lower performance. An ordinary cloud application might perform a calculation extremely quickly on plaintext. Performing the equivalent calculation using FHE may require considerably more processing time and computational resources. Therefore, the question is not simply whether FHE is mathematically possible. The practical question is:

Can the computation be performed securely while remaining fast and affordable enough for the application?

This introduces several engineering considerations:

- Security: How strongly does the scheme protect the underlying data?
- Privacy: Does the cloud learn anything meaningful from the encrypted computation?
- Performance: How much slower is encrypted computation?
- Scalability: Can the system handle large datasets and many users?
- Cost: How much additional computing infrastructure is required?

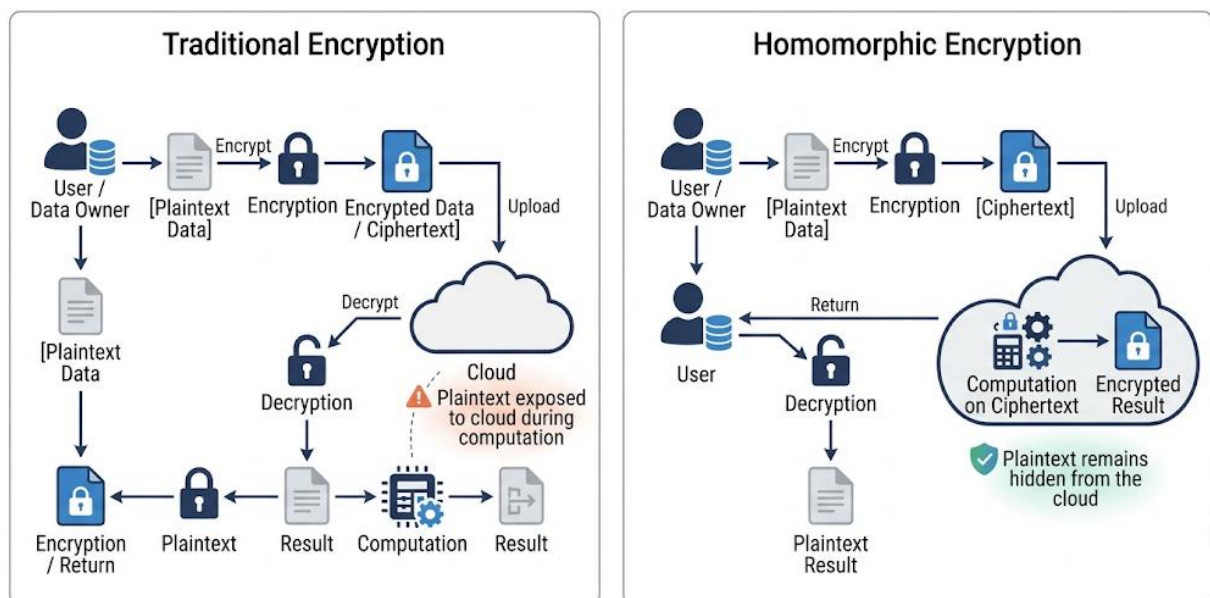
This is why FHE is powerful but not automatically the best solution for every cloud workload. Its usefulness depends on whether the additional privacy protection justifies the computational and operational cost.

Historical Significance

The mathematical idea behind homomorphic encryption existed long before modern cloud computing. However, a major milestone occurred in 2009, when presented a

construction for fully homomorphic encryption. His work demonstrated that general-purpose computation on encrypted data was theoretically achievable. The significance was not simply that encryption could protect information. It showed that computation itself could potentially be outsourced without requiring the computing party to see the data being processed. That idea is particularly relevant to cloud computing because cloud providers specialize in performing computation on behalf of customers. FHE attempts to allow that outsourcing while reducing the provider's visibility into the customer's sensitive information.

Traditional Encryption vs. Homomorphic Encryption in Cloud Computing



Privacy-Preserving Data Mining

Privacy-preserving data mining (PPDM) aims to extract useful patterns from data while preventing sensitive information from being exposed. In cloud computing, organizations may outsource data mining because the cloud provides large-scale storage and computational resources. However, sending the dataset to the cloud may expose sensitive customer or business information. It may want to discover patterns such as:

Customers who buy Bread

↓

often buy Milk

These relationships are useful for:

- recommendations;
- marketing;
- inventory management;
- product placement;
- business intelligence.

This type of analysis can involve **association rule mining**. The company may not have enough storage, computing resources, or expertise to perform the analysis locally, so it outsources the task to a cloud provider. But this creates a problem. The cloud provider now has access to potentially sensitive business information. For example, the transaction database might reveal:

- which products are popular;
- customer behavior;
- sales patterns;
- business relationships;
- commercially sensitive information.

Therefore:

The company wants the cloud to perform the computation without learning the sensitive information contained in the dataset. This is the basic problem of privacy-preserving data mining.

Why Simple Anonymization Is Not Enough

A simple method is to replace meaningful identifiers with arbitrary values:

Bread → 17

Milk → 42

Cereal → 83

Although the cloud no longer sees the product names, it can still observe statistical properties, such as how frequently each value occurs.

If:

$17 \rightarrow 90\%$

$42 \rightarrow 75\%$

$83 \rightarrow 40\%$

and an attacker already knows that Bread is likely to be the most frequently purchased product, the attacker may infer:

$17 = \text{Bread}$

This is a background-knowledge attack. The important point is that privacy can be broken through inference even when direct identifiers have been removed. Therefore, anonymization must consider not only explicit identifiers but also the patterns and statistical relationships remaining in the data.

k-Support Anonymity

Researchers have proposed techniques such as k-support anonymity to make individual items harder to identify through their support or frequency. The basic idea is to make an item indistinguishable from several other items with similar support. If an attacker observes that one item appears with a particular frequency, it should not be obvious which real item that frequency represents. The parameter **k** represents the desired level of indistinguishability.

k-Privacy

k-privacy extends the idea beyond individual items. Instead of only asking whether one item can be identified, it considers whether an itemset—a combination of items—can be distinguished.

For example: {Bread, Milk} may reveal information that neither Bread nor Milk reveals independently. A k-privacy approach attempts to make a transformed itemset indistinguishable from at least $k-1$ other itemsets of the same size.

Distributed and Multicloud Security

Another research direction is to avoid depending completely on one cloud provider. Suppose a company has sensitive encrypted data. Instead of placing all security responsibilities on one provider: Cryptographic information can be distributed among multiple independent servers.

For example, a private key can be divided into several pieces. No single server has the complete key. The security assumption becomes:

The secret remains protected as long as not all participating servers collude.

This is an example of distributed trust. Instead of saying: "I completely trust Provider A."

the model becomes:

"I do not need to completely trust any
single provider, provided the required
independent providers do not collude."

This approach is particularly interesting in multicloud environments, where an organization uses services from multiple cloud providers.

Fine-Grained Access Control

Cloud environments require detailed control over who can access which resources. Instead of simply allowing or denying access to an entire system, fine-grained access control makes decisions based on specific users, resources, roles, attributes, or policies. For example:

Doctor → Patient medical records

Nurse → Assigned patient information

Finance → Billing information

Patient → Own records

Administrator → Infrastructure management

The challenge becomes significant at cloud scale because systems may contain: Millions of users, Millions of resources, large numbers of policies, constantly changing permissions. Therefore, access control must be both secure and scalable.

Cryptographic Access Control

Researchers have developed cryptographic approaches to enforce fine-grained access policies. Important concepts include:

- Attribute-Based Encryption (ABE)
- Ciphertext-Policy Attribute-Based Encryption (CP-ABE)
- Proxy Re-Encryption
- Hierarchical Identity-Based Encryption

The common idea is that access to encrypted information can depend on identity, attributes, roles, or policies.

For example:

Encrypted Document

Policy:

Department = Finance

AND

Role = Manager

AND

Employee = Active

Only a user satisfying the required policy should be able to decrypt the information.

This is more powerful than simply storing a basic "allowed/denied" permission.

The Key Management Problem

Strong cryptographic access control creates another problem: managing the cryptographic keys at scale.

A cloud system may need to handle:

Key generation → Distribution → Storage → Update → Revocation → Re-encryption

Revocation is particularly difficult. If an employee leaves an organization, their ability to decrypt previously accessible resources may need to be removed. If their key was associated with many resources, updating the affected permissions or ciphertexts can become expensive. Therefore, effective cloud security requires not only strong encryption but also efficient and scalable key management.

Centralized vs Distributed Access Control

In **centralized access control**, access policies are managed by a single central authority. This model is relatively simple to manage but becomes difficult when multiple organizations or cloud domains are involved.

In **distributed access control**, multiple authorities participate in making an access decision. This is useful when different organizations control different aspects of access. For example, access to a medical resource may depend on both the hospital and the user's professional organization.

Identity Management and Federated Identity

Cloud users often need access to multiple services. Federated identity management allows trusted organizations or identity providers to authenticate users and share their identities across different cloud services. This avoids requiring separate accounts for every service.

Federated identity is particularly useful in enterprise environments, cloud collaboration, multi-cloud systems, and cross-organization applications

Trust in Cloud Computing

Security and trust are related but different. Security focuses on whether technical mechanisms protect systems and data, while trust focuses on whether a cloud provider

or third party can be depended upon to behave correctly and follow its responsibilities. For example, a provider may claim that customer data is encrypted. The customer must still trust that encryption is correctly implemented, keys are protected, access policies are enforced, and administrators and third parties follow the required security policies. Therefore, cloud computing requires mechanisms for evaluating, establishing, and maintaining trust.

Chain of Trust and Accountability

Cloud services often depend on several organizations, such as storage providers, security services, API providers, and software vendors. These dependencies create a **chain of trust**. As the number of dependencies increases, there are more trust relationships, potential failure points, and accountability problems. If sensitive data is exposed, it may be difficult to determine whether the problem was caused by the customer, cloud provider, subcontractor, configuration error, or another service provider. Therefore, cloud security has not only a technical dimension, but also organizational, contractual, and legal dimensions.

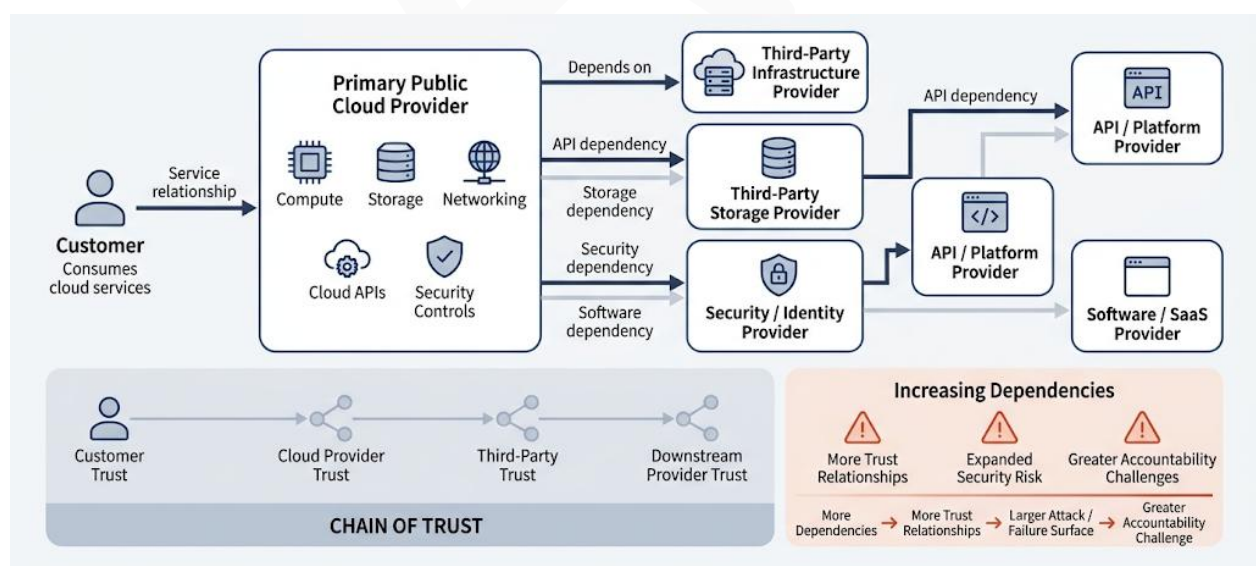


Figure: Cloud Chain of Responsibility

Legal and Regulatory Issues

Cloud security is also affected by laws, regulations, contracts, and organizational policies. These requirements may specify where data can be stored, who can access it, how long it can be retained, how personal information can be processed, and what must happen after a data breach.

These issues become more complex when cloud services operate across countries because different jurisdictions may have different requirements. Organizations must therefore understand both how their data is protected and where the data is processed and stored.

Shared Security Responsibility

Moving to the cloud does not remove the customer's security responsibilities. Security is generally a shared responsibility between the cloud provider and the customer, although the exact division depends on the service model.

In IaaS, customers generally have greater responsibility for operating systems, applications, permissions, and configurations. In PaaS, the provider manages more of the underlying platform while the customer focuses mainly on applications and data. In SaaS, the provider manages most of the infrastructure and application environment, but the customer remains responsible for areas such as account security, permissions, and proper data usage. **Cloud security is a shared responsibility.**

Cloud security problems are often interconnected rather than isolated. For example, weak authentication can allow an attacker to obtain an account. If that account has excessive permissions, the attacker may access sensitive data, causing confidentiality and privacy violations and potentially damaging customer trust. Similarly, weak virtual-machine isolation can allow one tenant to affect another tenant in a shared cloud environment. Third-party weaknesses can also affect customers even when the customer's own systems are secure. Therefore, cloud security cannot be solved by a single technology. It requires multiple security mechanisms working together.

A Complete Cloud Security Model

The complete picture can be organized into several layers:

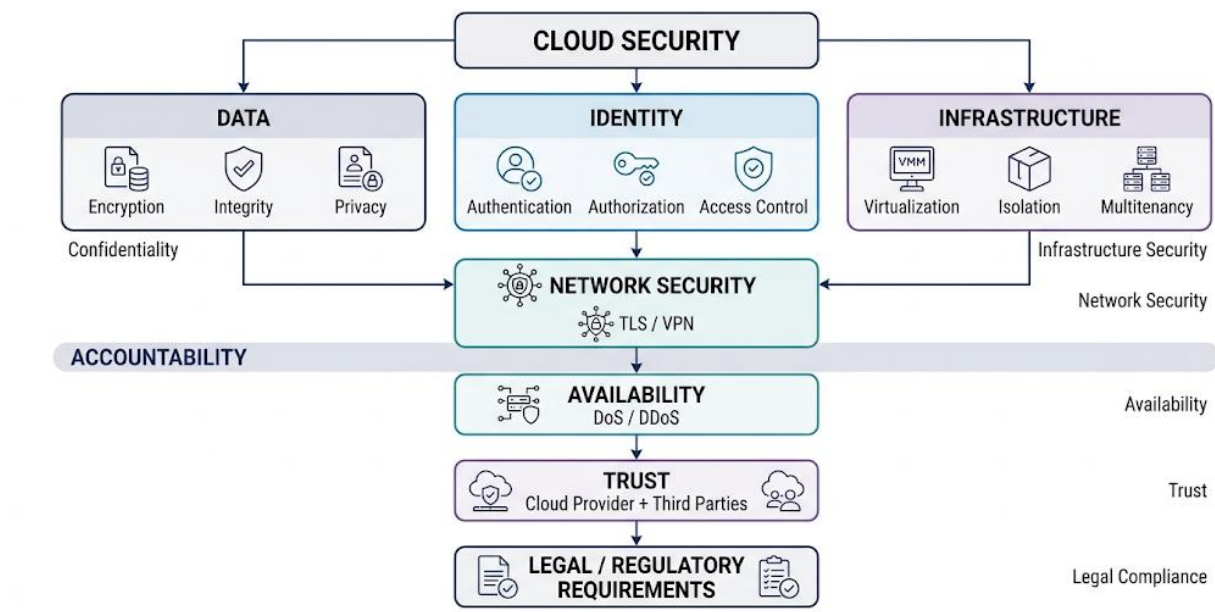


Figure: Cloud Security Model

This is a useful mental model because every layer answers a different question.

Question	Concept
Can someone unauthorized read the data?	Confidentiality
Has the data been modified improperly?	Integrity
Can authorized users access the service?	Availability
Who is the user?	Authentication
What can the user do?	Authorization / Access Control
Is personal information handled appropriately?	Privacy
Can the cloud environment be depended upon?	Trust

Who is responsible if something goes wrong?	Accountability / Legal responsibility
Can multiple customers safely share infrastructure?	Multitenancy / Isolation
Can the cloud compute without seeing sensitive data?	Privacy-preserving computation / Homomorphic encryption

Major Security Challenges and Their Solutions

The entire topic can be revised through the following relationship:

Cloud Security Challenge	Main Concern	Important Techniques / Direction
Data theft	Confidentiality	Encryption, access control
Unauthorized modification	Integrity	Hashes, signatures, integrity verification
Service disruption	Availability	Redundancy, scaling, DDoS protection
Stolen accounts	Identity	Strong authentication, federated identity
Excessive permissions	Authorization	Fine-grained access control, least privilege
Malicious administrator	Insider threat	Monitoring, auditing, separation of duties, cryptography
Shared infrastructure	Multitenancy	VM isolation, secure virtualization
Data intercepted in transit	Network security	TLS, secure communication
Provider sees sensitive data	Privacy	Encryption, homomorphic encryption, privacy-preserving computation
Data deleted but backups remain	Data remanence	Secure deletion and retention policies

Analytics reveals sensitive patterns	Data privacy	Privacy-preserving data mining
Too many users/resources	Scalability	Distributed and cryptographic access control
Multiple providers	Trust	Distributed trust, multicloud security
Third-party dependencies	Accountability	Contracts, audits, trust management
Cross-border data	Legal/privacy	Regulatory compliance and data-governance policies

Security Trade-Offs

Cloud security mechanisms have costs and limitations. Stronger encryption can increase computational requirements. Fine-grained access control can make key management more complex. Homomorphic encryption improves privacy during computation but can introduce significant performance overhead. Using multiple cloud providers can reduce dependence on one provider but increases system complexity. Therefore, the goal is not to maximize security at any cost. Practical cloud security must balance:

Security + Privacy + Trust + Availability + Scalability + Performance + Cost

Summary:

The central security problem of public cloud computing is loss of direct control. Organizations gain scalability, flexibility, availability, and reduced infrastructure-management costs, but they must depend on cloud providers, administrators, technologies, policies, and third-party services.

This creates several connected security concerns. Confidentiality, integrity, and availability protect information and services. Authentication, authorization, and access control manage users and permissions. Privacy focuses on the appropriate handling of personal and sensitive information. Trust and accountability address whether providers and third parties can be relied upon and who is responsible when something goes wrong.

Virtualization and multitenancy introduce risks from shared infrastructure, while network communication creates risks during data transfer. Cloud storage also raises concerns about data location, replication, backup, deletion, and retention. Third-party services further increase the complexity of trust and accountability.

Traditional encryption remains essential, but it does not solve every problem. Encryption protects data at rest and in transit, while data in use is more difficult to protect because computation normally requires access to plaintext. Homomorphic encryption attempts to address this problem by enabling computation on encrypted data, although performance remains a major limitation.

Cloud security is the protection of cloud data, applications, users, and infrastructure against unauthorized access, disclosure, modification, and disruption, while cloud privacy ensures that information is handled appropriately, and cloud trust ensures that customers can depend on providers and their service partners to enforce these protections.

Identity and Access Management (IAM) in Cloud Computing

In today's digital landscape, organizations increasingly rely on cloud computing to store data, run applications, and deliver services. As cloud environments become more distributed, organizations must ensure that only authorized users, applications, devices, and services can access their resources. This is where Identity and Access Management (IAM) becomes essential.

Identity and Access Management is a cybersecurity framework for managing digital identities and controlling access to resources. IAM helps organizations ensure that the right person or system gets access to the right resource, with the right permissions, at the right time, and for the right purpose.

Although IAM is not limited to cloud computing, it has become a core security component of cloud environments. Cloud platforms may contain large numbers of users, applications, virtual machines, containers, APIs, databases, storage systems, and automated workloads. These resources may be accessed from different locations and devices, making traditional perimeter-based security insufficient.

IAM provides a centralized way to create identities, verify users, assign permissions, enforce access policies, and monitor activity. In cloud computing, these capabilities help organizations protect resources while still allowing users and applications to access the services they need.

Understanding Identity and Access Management

What is IAM?

Identity and Access Management (IAM) is a collection of policies, processes, and technologies used to manage digital identities and control access to information systems and resources.

A digital identity represents a specific entity, such as a human user, application, server, device, or automated workload. The identity can contain attributes such as a username, credentials, role, department, permissions, or other identifying information.

IAM uses these identities to determine whether an entity should be allowed to access a particular resource and what actions it is permitted to perform.

For example, an employee may be allowed to read documents stored in a company's cloud storage but may not be allowed to delete them. A database administrator may have permission to modify a database, while a regular employee may only be able to view certain information.

Therefore, IAM is fundamentally concerned with three important questions:

- **Who are you?** — Identity and authentication
- **What are you allowed to access?** — Authorization
- **What are you allowed to do?** — Permissions and access control

IAM in Cloud Computing

Cloud computing changes how organizations manage identities and access because resources are no longer restricted to a single physical network or data center.

In a traditional environment, users and resources may exist within an organization's controlled network. In a cloud environment, employees can access services remotely,

applications communicate through APIs, workloads run across multiple cloud services, and organizations may use multiple cloud providers.

Cloud IAM provides mechanisms for managing these identities and permissions across cloud resources.

For example, a cloud administrator might create an identity for an employee and assign that identity a specific role. The role could allow the employee to access certain storage resources but prevent access to sensitive databases. Similarly, an application running in the cloud can receive its own identity and permissions so that it can communicate with other services without using a human user's credentials.

Cloud IAM therefore applies not only to human users but also to nonhuman identities, including applications, services, virtual machines, containers, devices, APIs, and automated workloads.

Why IAM Is Important in Cloud Computing

IAM is important because cloud environments contain valuable resources that can be accessed from many locations and by many different types of identities.

- **Security**

IAM protects cloud resources from unauthorized access. Strong authentication, access controls, and least-privilege permissions can reduce the likelihood that compromised accounts will be used to access sensitive resources.

For example, even if an attacker obtains an employee's credentials, carefully designed permissions can limit what that account can access.

- **Compliance**

Organizations often need to demonstrate that sensitive information is accessible only to authorized users. IAM provides access policies, logging, and audit trails that can help organizations meet regulatory and security requirements.

Standards and regulations such as GDPR, HIPAA, and PCI DSS may require organizations to implement appropriate access controls and maintain records of access to sensitive information.

• Operational Efficiency

IAM can automate many identity-related tasks. When an employee joins an organization, their identity and required permissions can be provisioned automatically. When the employee leaves or changes roles, access can be modified or revoked.

This reduces manual administration and helps prevent former employees or users with outdated permissions from retaining unnecessary access.

• User Experience

Security does not have to mean making access difficult. IAM technologies such as Single Sign-On (SSO) allow users to authenticate once and access multiple authorized applications without repeatedly entering credentials.

This can improve productivity while maintaining centralized control over authentication and authorization.

The Four Pillars of IAM

IAM can be understood through four major functions: administration, authentication, authorization, and auditing.

1. Administration

Identity administration, also called identity management or identity lifecycle management, involves creating, maintaining, updating, and securely removing digital identities.

When a new employee joins an organization, an identity is created for that person. The organization can then assign appropriate roles and permissions. When the employee changes departments, their permissions can be updated. When the employee leaves, their identity and access can be deprovisioned. This process is known as the identity lifecycle.

Cloud environments make automated identity lifecycle management particularly important because organizations may have thousands of users and large numbers of applications and workloads.

2. Authentication

Authentication is the process of verifying that a user or system is who it claims to be.

A user may authenticate using a password, security key, biometric factor, authentication application, or another credential. Nonhuman identities may authenticate using certificates, tokens, keys, or other machine credentials.

Passwords are widely used but can be vulnerable to theft and reuse. For this reason, cloud environments commonly use Multi-Factor Authentication (MFA).

MFA requires users to provide multiple authentication factors. For example, a user might provide a password and then confirm their identity using a mobile application or security key.

Other authentication technologies include passwordless authentication, passkeys, biometric authentication, and adaptive or risk-based authentication.

3. Authorization

Authentication answers the question “Who are you?”, while authorization answers “What are you allowed to do?” After an identity has been authenticated, the IAM system evaluates its permissions and determines whether the requested action should be allowed.

For example, an authenticated cloud user may be allowed to view files in a storage service but may not be authorized to delete those files.

Authorization is commonly implemented through access policies and models such as Role-Based Access Control (RBAC) and Attribute-Based Access Control (ABAC).

4. Auditing

Auditing involves recording and reviewing identity and access activity. Cloud IAM systems can maintain logs showing information such as who accessed a resource, what action they performed, when the action occurred, and whether the request was allowed or denied.

These audit trails can help organizations detect suspicious behavior, investigate security incidents, identify excessive permissions, and demonstrate compliance with security requirements.

IAM Components and Technologies

IAM is not a single technology. It consists of multiple technologies that work together to manage identities and access.

. Directory Services

Directories store and manage information about identities, credentials, roles, and permissions.

A directory can act as a central source of identity information. Cloud IAM systems can also integrate with existing organizational directories so that users can use their existing identities to access cloud applications.

. Identity Federation

Identity federation allows different systems to trust and share identity information. For example, an organization may maintain its users in one identity provider while allowing those users to access applications hosted by another service provider.

Protocols such as **Security Assertion Markup Language (SAML)** and **OpenID Connect (OIDC)** are commonly used to support identity federation.

• Single Sign-On

Single Sign-On allows users to authenticate once and then access multiple authorized applications and services. Instead of maintaining separate credentials for every application, users can authenticate through a centralized identity provider.

SSO can improve usability while also giving administrators centralized control over authentication policies.

• Access Control

Access control determines which identities can access specific resources and what actions they can perform.

Cloud access policies can specify permissions for resources such as storage buckets, databases, virtual machines, applications, and APIs.

• Privileged Access Management

Privileged Access Management (PAM) focuses on protecting highly privileged identities, such as cloud administrators and system administrators. Privileged accounts are attractive targets for attackers because they can potentially make significant changes to an environment.

PAM can use techniques such as credential vaulting, credential rotation, and just-in-time access to reduce the risk associated with privileged accounts.

• Secrets Management

Cloud applications often need credentials such as API keys, passwords, certificates, and access tokens.

Instead of embedding these secrets directly into application code, organizations can use secrets management systems to securely store and retrieve them.

This is particularly important for nonhuman identities such as applications, servers, containers, and automated workloads.

Access Control Models

Different access control models can be used to determine how permissions are assigned.

. Role-Based Access Control

Role-Based Access Control (RBAC) assigns permissions according to a user's role or job function.

For example, a cloud organization might have separate roles for administrators, developers, security analysts, and regular employees. Each role receives only the permissions required for its responsibilities.

RBAC simplifies administration and can reduce excessive permissions.

. Attribute-Based Access Control

Attribute-Based Access Control (ABAC) makes access decisions based on attributes associated with users, resources, and actions.

For example, access could depend on the user's department, the sensitivity of the resource, the device being used, or the location and time of the request.

ABAC can provide more granular and dynamic access decisions than traditional role-based approaches.

. Principle of Least Privilege

A fundamental IAM principle is least privilege. Least privilege means that users and applications should receive only the minimum permissions necessary to perform their tasks.

For example, an application that only needs to read data from a database should not receive permission to delete the database.

Applying least privilege limits the potential damage caused by compromised accounts and reduces unnecessary access to sensitive cloud resources.

IAM Challenges in Cloud Computing

• Increased Attack Surface

Cloud environments may contain large numbers of users, devices, applications, APIs, and automated workloads. This creates a larger identity attack surface.

Compromised credentials can allow attackers to access cloud resources, particularly when accounts have excessive permissions.

• Identity Sprawl

Organizations may use multiple cloud providers, SaaS applications, and identity systems. Managing identities across these environments can create fragmentation and make it difficult to maintain consistent access policies.

This problem becomes more significant in multicloud environments, where organizations use services from multiple cloud providers.

• Privilege Management

Users and applications can accumulate permissions over time. If these permissions are not regularly reviewed, identities may retain access that they no longer need.

Excessive privileges increase the impact of compromised accounts.

• Nonhuman Identities

Modern cloud environments contain many nonhuman identities. Applications, APIs, containers, virtual machines, automation systems, and AI agents may all require access to resources.

These identities can be difficult to manage because they may operate automatically and can have highly privileged credentials.

. Compliance and Auditing

Organizations need visibility into who can access sensitive resources and what actions they perform.

Maintaining accurate access policies and comprehensive audit logs can become difficult when resources are distributed across multiple cloud platforms.

Best Practices for IAM in Cloud Computing

Organizations can improve cloud security by following several IAM best practices.

. Implement Strong Authentication

Organizations should use strong authentication mechanisms, particularly MFA for privileged and sensitive accounts. Passwordless authentication and phishing-resistant authentication methods can provide additional protection against credential theft.

. Apply Least Privilege

Users and applications should receive only the permissions required to perform their tasks. Permissions should be regularly reviewed and unnecessary privileges should be removed.

. Use RBAC and Appropriate Access Policies

Organizations should define clear roles and access policies based on business requirements.

Permissions should correspond to actual job responsibilities rather than being granted simply for convenience.

. Automate Provisioning and Deprovisioning

Automating the identity lifecycle can ensure that users receive appropriate access when they join an organization and lose unnecessary access when they leave or change roles. Automation can also help manage large numbers of machine and application identities.

. Monitor and Audit Access

Organizations should continuously monitor identity activity and maintain audit logs. Suspicious behavior, unusual login patterns, privilege escalation, and unauthorized access attempts should be investigated promptly.

. Protect Privileged Accounts

Administrative and other highly privileged accounts should receive additional protection through PAM, MFA, credential management, and just-in-time access where appropriate.

. Secure Nonhuman Identities

Applications and workloads should have their own identities instead of relying on shared human credentials. API keys, certificates, tokens, and other secrets should be securely stored, rotated, and monitored.

. IAM and Zero Trust

IAM is closely connected to the Zero Trust security model. Traditional security models often assume that users inside an organization's network can be trusted. Zero Trust takes a different approach: access should not automatically be trusted simply because a user or device is inside a network.

Instead, every access request should be evaluated based on identity, permissions, context, and risk.

The principle of least privilege is an important part of this approach. Users and applications should receive only the access they need, and access can be continuously evaluated as circumstances change.

In cloud computing, Zero Trust is particularly relevant because users, applications, and resources may be distributed across different networks and locations.

IAM and Artificial Intelligence

Artificial intelligence is creating new opportunities and challenges for IAM. Traditional IAM systems already use automation to handle activities such as authentication, provisioning, authorization, and auditing. AI and machine learning can extend these capabilities by analyzing user behavior and identifying unusual access patterns.

For example, an IAM system might identify that a user who normally accesses cloud resources from one location has suddenly started accessing highly sensitive resources from an unusual device. The system could increase authentication requirements or flag the activity for investigation.

Generative AI and autonomous AI agents also introduce new types of nonhuman identities. AI agents may need access to applications, APIs, databases, and other cloud resources.

As these identities become more common, organizations will need strong controls for their authentication, authorization, credentials, and permissions.

Future Trends in IAM for Cloud Computing

IAM is evolving as organizations adopt cloud computing, automation, artificial intelligence, and distributed architectures.

One important trend is the increasing use of AI-driven identity security, where machine learning is used to identify anomalous behavior, assess risk, and assist security teams in investigating identity-related threats.

Another trend is the growth of passwordless authentication, including passkeys and other cryptographic authentication technologies that can reduce reliance on traditional passwords.

Organizations are also increasingly focusing on identity threat detection and response (ITDR). ITDR technologies are designed to identify identity-related threats such as compromised accounts, privilege escalation, and identity misconfigurations.

The number of nonhuman identities is also expected to continue growing as organizations deploy more automation, cloud workloads, IoT devices, APIs, and AI agents. Managing these identities securely will therefore become an increasingly important part of cloud security.

Conclusion:

Identity and Access Management is a fundamental component of cloud computing security. Although IAM is a broader cybersecurity discipline that applies to both traditional and cloud environments, its importance has increased significantly as organizations move applications, data, and infrastructure to the cloud.

IAM enables organizations to manage digital identities, authenticate users and systems, authorize access, enforce least-privilege policies, and monitor activity. Technologies such as MFA, SSO, identity federation, RBAC, ABAC, PAM, secrets management, and auditing provide organizations with the tools needed to control access to cloud resources.

Cloud environments also introduce challenges such as identity sprawl, excessive permissions, compromised credentials, multicloud complexity, and the growing number of nonhuman identities. Organizations can address these challenges by implementing strong authentication, least privilege, automated identity lifecycle management, continuous monitoring, and appropriate access controls.

As cloud computing continues to evolve alongside Zero Trust, automation, and artificial intelligence, IAM will remain a central part of protecting cloud resources. Effective IAM ultimately ensures that the right identities receive the right access to the right resources at the right time—and nothing more.

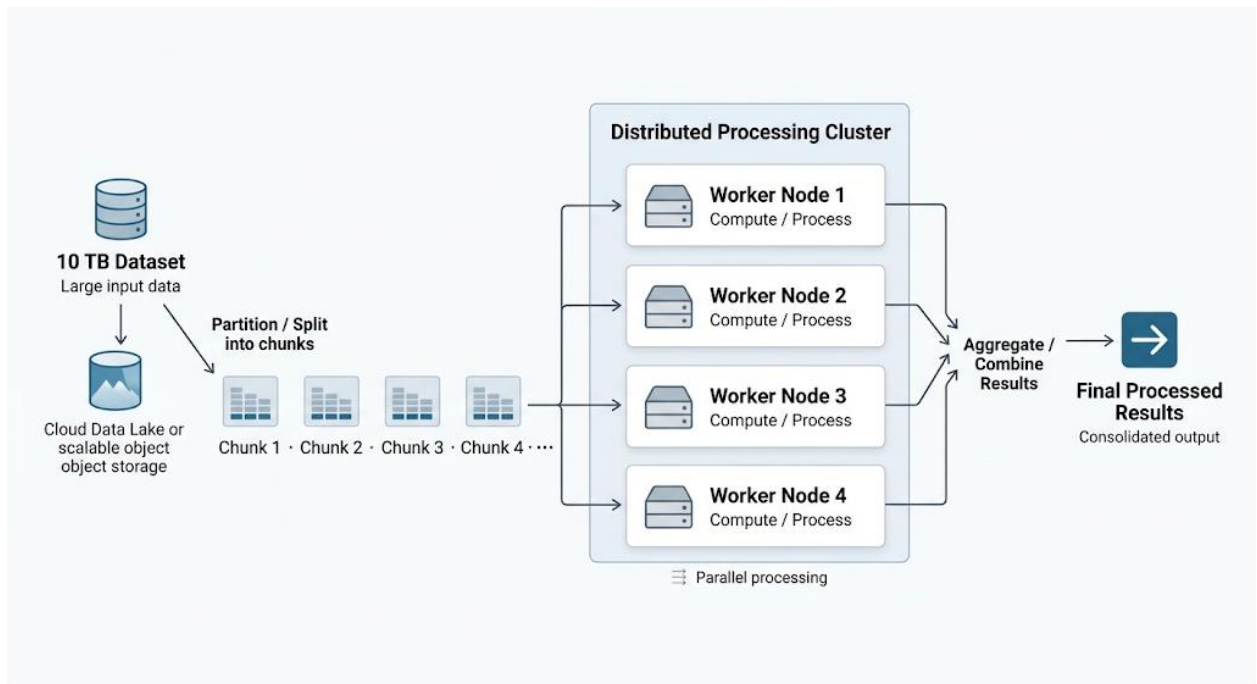
This version keeps the paragraph-based explanatory style of the IBM article while making the organization much more suitable for a Cloud Computing assignment, report, or study

material. It also connects the IBM concepts—especially the four pillars, nonhuman identities, least privilege, PAM, secrets management, identity federation, ITDR, Zero Trust, and AI—to the cloud-specific structure of your second article.



Introduction to Parallel Computing

Parallel computing is a method of solving a large or complex computational problem by dividing it into smaller parts and processing those parts simultaneously using multiple processing units. In traditional sequential computing, a single processor generally performs tasks one after another, which can take a long time when the amount of data or computation becomes very large. The main goal of parallel computing is to reduce execution time, improve computational efficiency, and handle problems that are too large or demanding for a single machine. **For example**, suppose a company has 10 TB of transaction data that needs to be analyzed. If one machine processes the entire dataset, the processing may take a very long time because a single machine is limited by its CPU processing power, available memory, storage capacity, I/O speed, and network capacity. Instead of forcing one machine to handle everything, the large dataset can be divided into smaller portions and distributed across multiple machines or processing units. These machines can then process their assigned portions at the same time, allowing the overall task to be completed much faster. This idea of dividing a problem, distributing the work, and executing multiple parts simultaneously forms the basic foundation of distributed parallel processing.



Parallel computing commonly follows a divide-and-conquer approach, where a large problem is divided into smaller, more manageable subproblems. These smaller problems are then processed simultaneously by different processing units or workers, and their individual results are finally combined to produce the overall result. For example, if we need to count the words in a huge collection of documents, processing all documents on one machine would require that machine to read and analyze the entire collection sequentially. Instead, the documents can be divided into several parts, such as Part 1, Part 2, and Part 3, and each part can be assigned to a different worker. All workers process their assigned documents at the same time and produce their own word counts. These partial results are then combined to obtain the final word count for the complete document collection. This approach allows large workloads to be handled more efficiently because the work is distributed and performed concurrently. MapReduce follows this general idea: the Map phase performs processing on individual portions of the data, while the Reduce phase combines the intermediate results to produce the final output.

Parallelism and Concurrency

Parallelism and concurrency are related, but they are not the same.

Concurrency means multiple tasks are being handled or making progress during the same period. The tasks do not necessarily execute at the exact same time.

Parallelism means multiple tasks are actually executed at the same time using multiple processing units such as CPU cores or multiple machines. For example, with 4 CPU cores:

Core 1 → Task A

Core 2 → Task B

Core 3 → Task C

Core 4 → Task D

Here, the four tasks can execute simultaneously, so this is true parallel execution.

In parallel computing, the main focus is parallelism: dividing a large problem into smaller tasks and processing those tasks simultaneously using multiple processing units. In systems such as MapReduce, this parallel processing can happen across multiple machines or nodes.

Types and Concepts of Parallel Computing

Parallel computing can be understood from different perspectives. We can classify it based on what is being executed in parallel, where the memory is located, and how multiple machines work together.

1. Data Parallelism

Data parallelism means applying the same operation to different portions of data at the same time. For example, suppose we want to multiply every number by 2:

The operation is the same ($\times 2$), but it is performed on different portions of the data simultaneously. MapReduce is strongly related to data parallelism because different workers can process different portions of a large dataset using the same map operation.

Data Parallelism = Same operation + Different data

2. Task Parallelism

Task parallelism means performing different tasks at the same time.

For example:

Processor 1 → Read data

Processor 2 → Calculate statistics

Processor 3 → Generate report

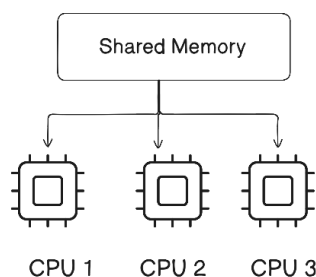
Processor 4 → Perform logging

Task Parallelism = Different tasks + Parallel execution

Here, the processors are not doing the same operation on different data. Instead, they are performing different operations/tasks simultaneously. For MapReduce, data parallelism is generally the more important idea, because large datasets are divided into portions and processed by multiple workers.

3. Shared-Memory Parallel Computing

In a shared-memory system, multiple processors or CPU cores access the same memory space.



All processors can access common data in memory. This makes communication and data sharing relatively fast and convenient. However, as the number of processors increases, managing shared data becomes more difficult. Multiple processors may try to

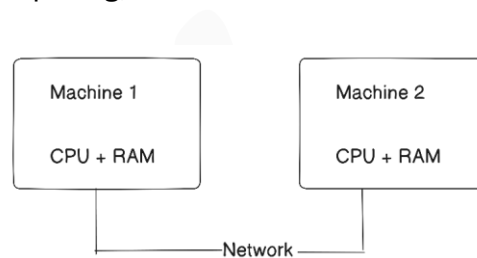
access or modify the same data at the same time, creating problems such as synchronization, race conditions, and memory contention.

Shared memory = Multiple processors/cores + Common memory

This model is common in multicore computers.

4. Distributed-Memory Parallel Computing

In distributed-memory computing, each machine has its own processor and memory.



The machines cannot directly access each other's memory. If they need information from another machine, they must communicate through a network. This is very important for Big Data, because one machine may not have enough CPU power, memory, or storage to handle a huge dataset. Multiple machines can therefore work together as a distributed system.

Distributed memory = Multiple machines + Separate memory + Network communication

This is the model most closely independently, cluster software coordinates them so that they can work together on a larger problem related to Hadoop and MapReduce.

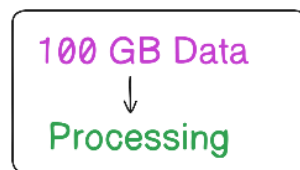
5. Cluster

A cluster is a group of connected computers, called nodes, that work together to perform large computational tasks. Each node is an individual machine. Instead of using the machines independently, cluster software coordinates them so that they can work together on a larger problem. Hadoop MapReduce runs on a cluster, where different nodes can process different parts of a large dataset.

Data Locality

Data locality means performing computation close to where the data is stored, instead of moving large amounts of data across the network. Suppose the data is stored on Node A but the processing needs to happen on Node B: Moving 100 GB through the network can take significant time and consume network resources. Instead, it is better to run the computation near the data:

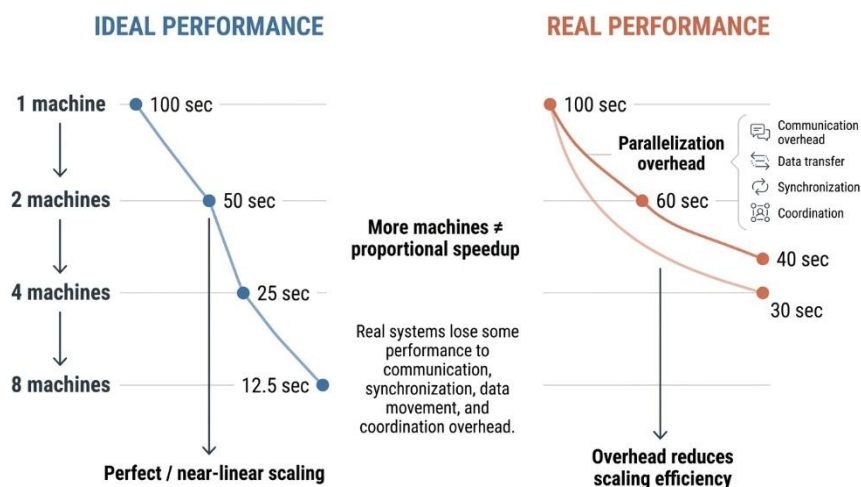
Node A



So, the basic principle is: Move computation to the data rather than moving huge amounts of data to the computation. Data locality is especially important in Hadoop, because large datasets are stored across multiple machines using HDFS.

Parallel Performance

Adding more machines does not always give a proportional increase in performance. Ideally, when the number of machines increases, the execution time should decrease proportionally.



In real systems, performance is usually lower because adding machines introduces overhead. Machines must communicate, exchange data, synchronize, distribute work, combine results, and handle failures.

Why doesn't it scale perfectly?

- Communication between machines
- Data exchange
- Synchronization
- Work/task distribution
- Combining results
- Failure handling

These activities consume additional time and resources, so the performance improvement is less than perfectly proportional. More Machines equals to More Parallel Work, Faster Execution But also more Overhead and Less than Proportional Improvement

Key point: Adding more machines improves performance, but the improvement is usually not perfectly proportional because of parallel overhead. This leads to the concepts of speedup, parallel efficiency, and scalability.

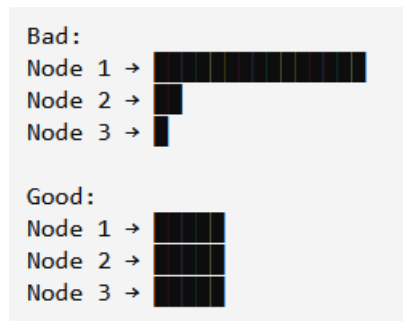
Challenges of Parallel Computing

Parallel computing introduces several challenges that must be managed carefully.

1. Task Distribution

The system must decide which task should be given to which machine. Poor distribution can cause some machines to be overloaded while others remain idle.

2. Load Balancing



The workload should be distributed reasonably evenly. Load balancing tries to prevent one machine from doing much more work than the others.

3. Communication Overhead

In distributed systems, machines communicate through a network. Node A $\leftrightarrow$ Node B $\leftrightarrow$ Node C. Network communication is generally slower than communication between components inside the same machine. Excessive communication can therefore reduce the benefit of parallel processing.

4. Synchronization

Some tasks depend on the completion of other tasks. Suppose Task A, Task B and Task C executes linearly. Task C must wait until the required work from A and B is complete.

5. Fault Tolerance

Machines can fail during processing. A distributed system must detect failures and recover or re-execute the failed work. MapReduce provides mechanisms to handle failed tasks.

6. Data Dependency

Not every task can be executed independently. Consider Task A, B, C if B requires the output of A, B cannot start that part of its work until A provides the required result. Such dependencies can limit the amount of parallelism possible.

Scalability

Scalability is the ability of a system to handle an increasing workload by increasing its resources. A scalable system should be able to process more data or requests efficiently

as the workload grows. There are two main ways to scale a system: vertical scaling (scale up) and horizontal scaling (scale out).

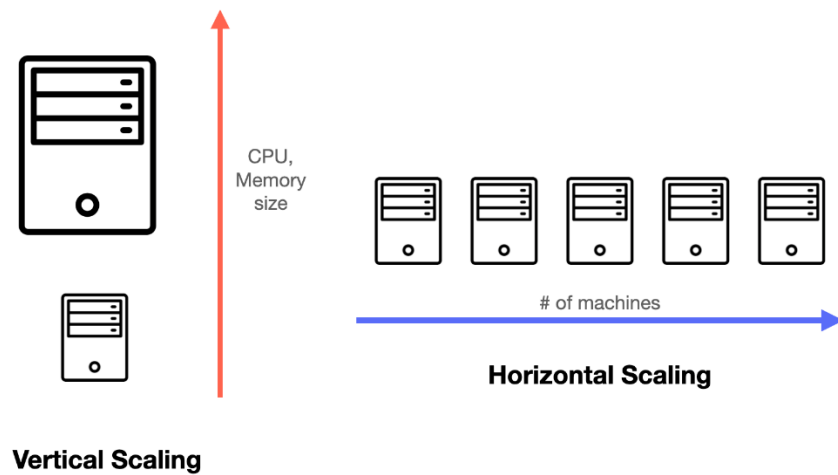


Figure: Vertical Scaling vs Horizontal Scaling

Vertical Scaling — Scale Up

Vertical scaling means making a single machine more powerful. Instead of adding machines, we increase the resources of the existing machine, such as CPU, RAM, or storage. For example, if one server is not powerful enough, we can upgrade its CPU, increase its RAM, or use faster storage. Scale up = Make one machine stronger. However, there are practical limits to how powerful a single machine can become, and upgrading hardware can also become expensive.

Horizontal Scaling — Scale Out

Horizontal scaling means adding more machines to the system instead of making one machine more powerful. The workload is distributed across multiple machines, allowing them to work together. For example, instead of upgrading one server, we can add several servers and distribute the workload among them. Scale out = Add more machines. Big Data systems such as Hadoop mainly rely on horizontal scaling because very large datasets can be distributed across many machines, while there are practical limits to how powerful a single machine can become.

How Parallel Computing Leads to MapReduce

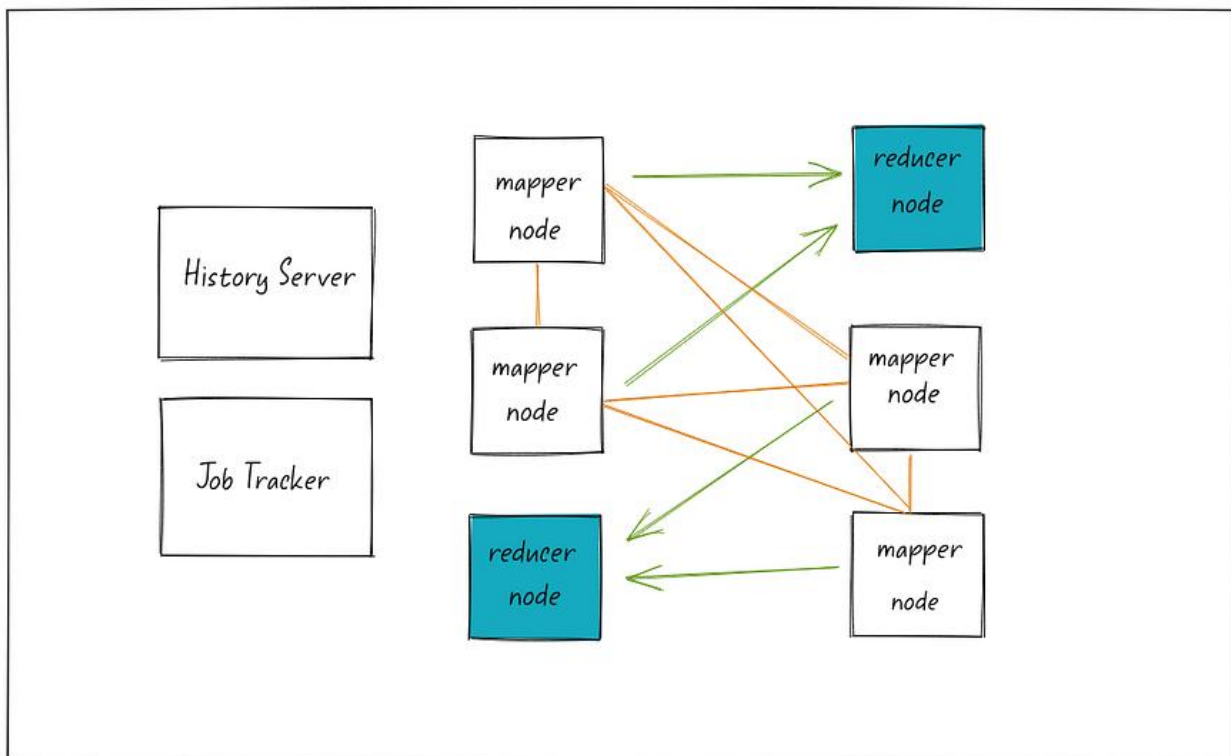
The idea behind MapReduce becomes clearer when we consider a huge dataset. If the dataset is too large or difficult for one machine to process, we can use multiple machines and divide the work among them. MapReduce is a programming model designed to process large datasets in a distributed and parallel manner. Instead of requiring the programmer to manually control every machine, the MapReduce framework manages much of the distributed processing.

The framework handles tasks such as distributing work, executing tasks in parallel, moving and grouping intermediate data, scheduling tasks, handling failures, and producing the final output. The programmer mainly defines two types of processing: **map()** and **reduce()**. The **map()** function processes the input data and produces intermediate results. The **reduce()** function then combines, summarizes, or aggregates those intermediate results to produce the final output.

MapReduce

***MapReduce** is a programming model and an associated implementation for processing and generating big data sets with a parallel, distributed algorithm on a cluster — Wikipedia*

This is the copy of article: <https://medium.com/@devparmar967/introduction-to-mapreduce-4d37712da879>



Modern applications generate extremely large datasets. A dataset may be too large to process efficiently on a single machine because a machine has limited CPU, memory, storage, and processing capacity. One approach is to make the machine more powerful, which is known as **vertical scaling**. However, vertical scaling has practical and physical limits.

Instead, large-scale systems commonly use **horizontal scaling**, where the workload is distributed across multiple machines.

Consider a simple data-processing task. Normally, we would load the data into memory, process it, and store the result on disk. This works well when the amount of data is relatively small. However, when dealing with big data, a single machine may not have enough memory, storage, or processing capacity to handle the entire dataset. In fact, this limitation is one of the main reasons distributed storage systems use multiple machines to store large datasets.

Once the data is distributed across multiple machines, another question arises: *How do we process that data efficiently?*

The basic idea is to do the same thing we did with storage: distribute the processing across multiple machines. Instead of processing the entire dataset on one machine, we divide the workload and allow multiple machines to process different portions of the data in parallel.

But this introduces new challenges. How do we decide which portion of the data should be processed by which machine? How do we coordinate the processing? How do we combine the results from different machines? And where should the final output be stored?

This is where MapReduce comes in.

MapReduce is a distributed programming model designed to process large datasets across multiple machines. It provides a structured way to divide a computation into smaller tasks, process those tasks in parallel, and combine their results.

It is worth noting that the original Hadoop MapReduce framework is no longer the first choice for many modern data-processing workloads. Frameworks such as Apache Spark and Apache Flink are often preferred because they provide more flexible and efficient processing for many workloads.

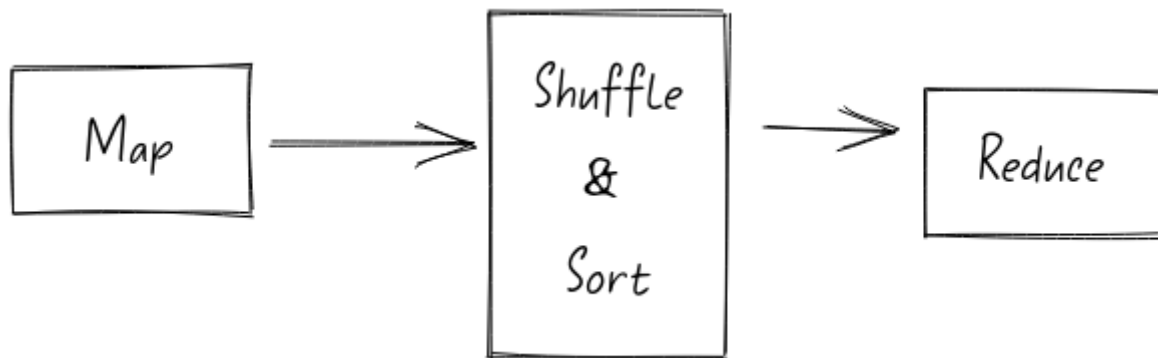
So, why should we still learn MapReduce?

1. **It provides fundamental concepts for distributed processing.** MapReduce introduced a simple and influential model for processing large datasets in parallel across a cluster.
2. **It helps build a foundation for understanding modern processing frameworks.** Many modern data-processing systems use concepts that are closely related to the ideas introduced by MapReduce, even when they do not directly use Hadoop MapReduce.
3. **Its limitations explain the evolution of modern frameworks.** Understanding the problems with MapReduce—such as repeated disk-based processing and limitations for iterative workloads—helps explain why frameworks such as Spark and Flink were developed.

With this background in mind, let us look at how MapReduce works and how it distributes data processing across a cluster.

MapReduce is a framework or library that comes with Hadoop. The MapReduce library contains classes that helps you write program that can run on multiple nodes, while hiding the details from the programmer. Details like, Resource management, Fault tolerance and other underlying functions. Thus MapReduce provides a layer of abstraction, so that the programmer can focus only on writing programs to run on a cluster ignoring how it all going to work out internally. A MapReduce application being executed is referred to as a MapReduce Job. A map reduce job splits the input data and process it through different phases by applying different function on the data and store the output. Storage nodes and processing nodes being used in MapReduce are same. Which means a MapReduce job executes on the same nodes HDFS use for storing data.

MapReduce on its core consists of 3 phases:



Each of these phase is called a task. Thus a MapReduce job is consist of multiple tasks.

Map:

As the name says, it's a function that takes input, performs some operation on the data to transform it, and produces output. The user has control, or in simple words user has to provide a custom function to be executed on data. In MapReduce all data exchanges has to be done in key-value pair. Except when taking input in Map Phase. Which means, it is necessary to produce a key-value pair as output in Map Phase. What about input data? well, input can be of any format HDFS or Hadoop supports like, Apache Avro, Parquet, CSV or JSON. It is not necessary to produce a output in a map task.

Press enter or click to view image in full size



There are two things to take in note:

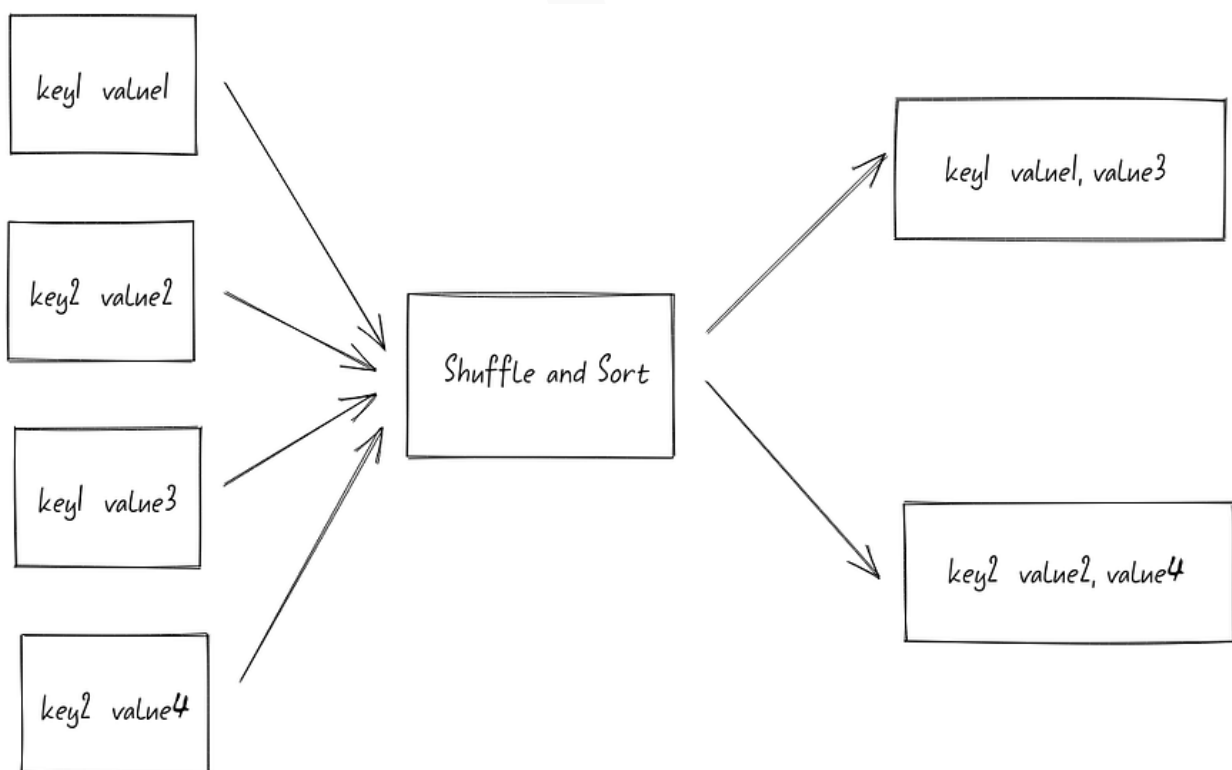
1. Generally, a map function is applied on each input block. So, if you are processing a single file that is divided in 3 blocks. 3 Map tasks would be spawned one for each block. Though you can control how many mappers you want to process the data. It is not a usual practice. for example, if you expect 10TB of input data and have a block size of 128MB, you'll end up with 82,000 maps, unless number of maps specified explicitly.

2. The produced output is stored on same node, until it's required for reduce phase, in that case, it would be moved.

Shuffle and Sort:

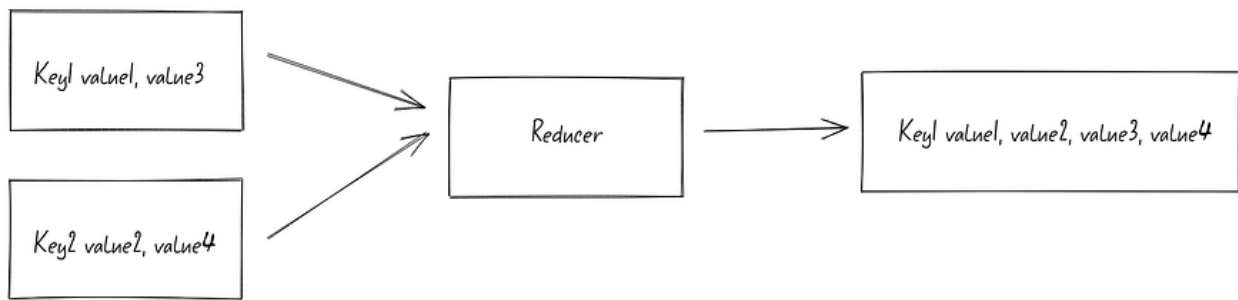
This is a phase internal to MapReduce framework. User has no control of it. It's the phase, where produced output from different mappers are collected, sorted, combined and given to reducers.

Shuffle & Sort is an internal framework stage, but certain components influencing it can be customized by the programmer.



Reduce:

This is the part where user has to write a custom function to reduce the mapped data into one or more final outcomes. You can think of reducer as an aggregate function whose role is to produce final output typically by combining or applying some function on data. You can specify zero or more reducer for a job.



NOTE: output of reducers are not sorted. And there can be multiple reducers.

The official page for Hadoop recommends, number of reducers to be 0.95 or 1.75 multiplied by (number of nodes * number of maximum containers per node).

With 0.95 all of the reduces can launch immediately and start transferring map outputs as the maps finish. With 1.75 the faster nodes will finish their first round of reduces and launch a second wave of reduces doing a much better job of load balancing. — hadoop's official docs

Other than these, there are also 2 internal tasks that are performed during a MapReduce job.

Comparator: Comparator controls how the data is sorted, during shuffle and sort phase. User can provide custom function for this task.

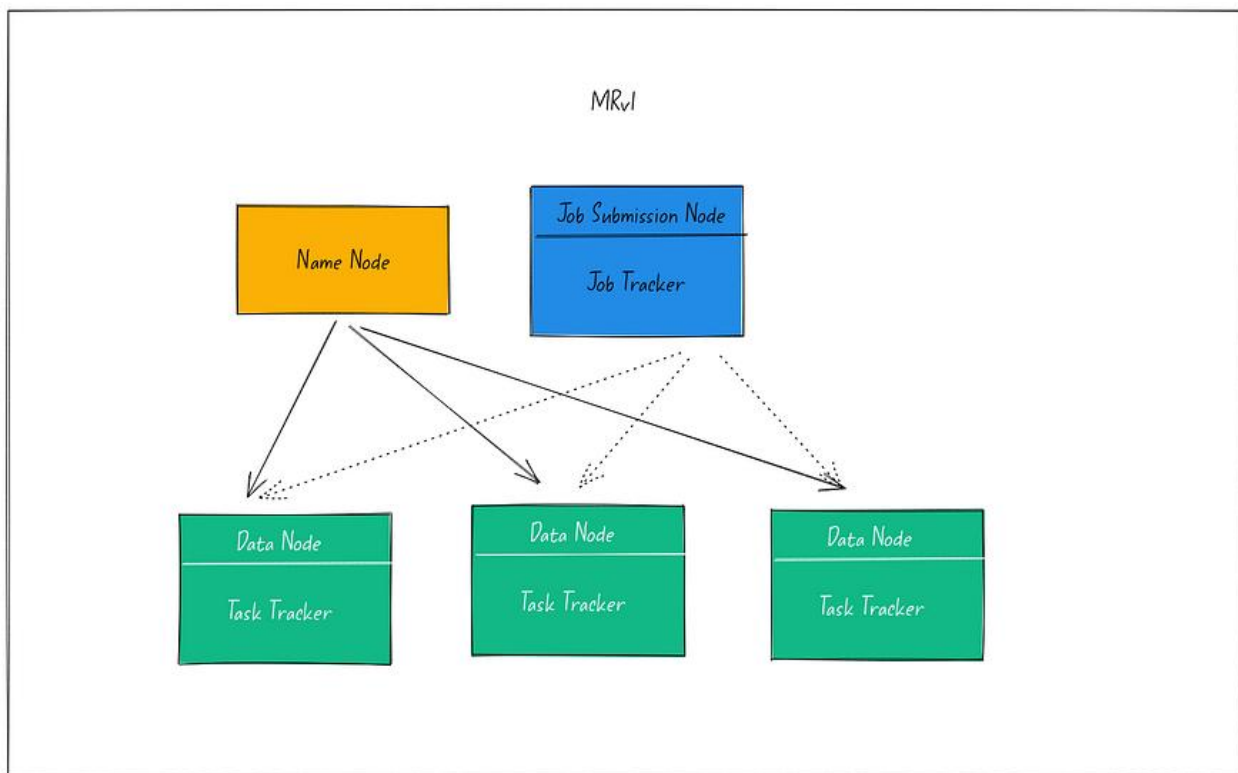
Partitioner: Partitioner controls how the key-value pairs produced after map and comparator task are partitioned to give to reducers. Also, this phase can be controlled by user by providing custom function.

There's one more thing you might encounter, that is counter.

Counter: counter is a facility for MapReduce applications to report statistics for a job. Basically, it's a counter variable which can be incremented to report progress of the job. Though it could be used for a lot more than this.

Architecture:

so far, we have understood what are components of a MapReduce application or Job. Now let's understand how phases or tasks are controlled and the program is executed.



The above diagram illustrates architecture of MapReduce version 1 or MRv1. As you can see there is one more node here, that is Job Submission Node. The Job Submission Node can be considered application master, as it is responsible for controlling the execute of whole job. Job Submission Node runs a daemon(process) Job Tracker that does all this. Just so you know, Job Tracker and Task Tacker are nothing but some processes generally referred as daemon. Usually a new node is set up for running Job Tracker, since there is supposed to be heavy load on it.

The Job tracker has following responsibilities:

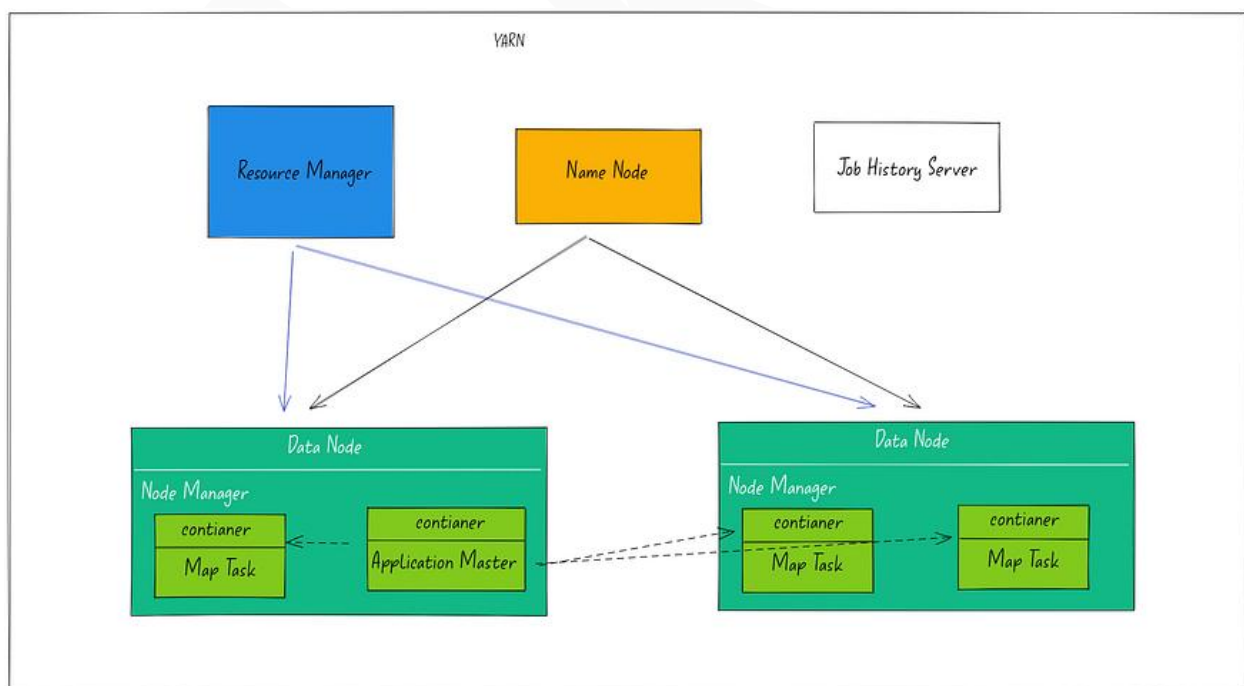
1. Initializing Map, Shuffle and sort or Reduce tasks on nodes.
2. Communicating with these nodes, for retrieving task progress.
3. Communicating with nodes for monitoring task progress, detecting failures, handling fault tolerance, and reassigning failed tasks.
4. accepting MapReduce jobs, dividing jobs into task. scheduling Map, assigning tasks to TaskTrackers, assigning failed tasks

Fault Tolerance:

Whenever a map task is failed, that is the process running the task dies due to reasons that were not caused by the program submitted by the user. Job Tracker launches another Map task, since the output of any number of map task done on same data would be same as well. Also, as you are aware in HDFS data is stored in replicated manner, the same task can be launched on another node containing same data block. The Job tracker does all that whenever a Map or Reduce task fails or dies.

YARN:

YARN or Yet Another Resource Negotiator is an evolved version of MapReduce. There map phase, reduce phase and other phases are still same but the architecture has changed to control the execution of applications. **YARN is Hadoop's resource-management and cluster-management layer that separates resource management from the MapReduce processing model.**



Resource Manager:

In Yarn Job Tracker is replaced with Resource Manager, which is responsible for allocating resources on the nodes. Whenever a worker node running any task requires more resources such as RAM it will contact Resource Manager for it.

Node Manager:

in Yarn Task Trackers are replaced by Node Managers, here as you can see all worker processes that is Map or Reduce tasks are run in a container. Thus, allowing us to create and run more than one map reduce tasks on a node. One more difference here is that the Application Master now runs in a container on any node. Which means that now not only we don't have to assign a different Node for Job Tracking but we can run Application master in distributed manner, by creating several instances of it on several nodes. YARN is pretty popular, and even these days it's still used extensively.

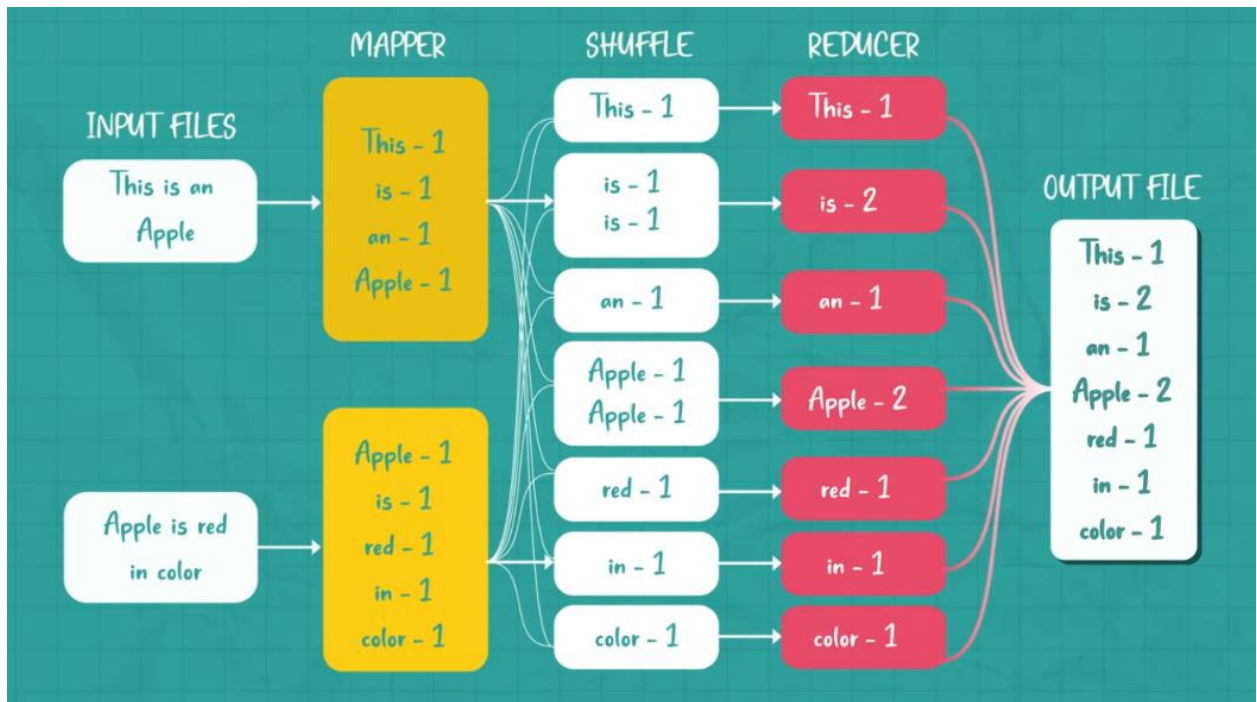
Programs:

MapReduce as Hadoop is written in Java. Though we can execute MapReduce applications in any language using a tool that comes built-in with Hadoop called **Hadoop Streaming**. More on Hadoop Streaming and writing MapReduce programs in future articles.

Summing up:

MapReduce was one of the earliest approach towards parallel computing but it has some limitations, such as:

1. IO operations were used extensively.
2. Storage was used frequently, for example when storing intermediate data. But with decreasing cost of RAM, it was cheaper to keep the intermediate data in memory. This will save us IO costs which appeared to be more than RAM cost.
3. MapReduce only works with data on HDFS. Which means you cannot load and process data from other sources such as Cloud.
4. Iterative operations on MapReduce were not cheap and hard to program and manage. And tasks such as Machine Learning consists of a lot of iterative operations.
5. MapReduce is notorious for its steep learning curve. MapReduce is written in java and apparently it requires a good knowledge of java to write efficient MapReduce jobs.



Optimizations in MapReduce Architecture

- **Speculative Execution:** If a task is running unusually slow, Hadoop launches a duplicate copy on another node. The task that finishes first is accepted, improving reliability and efficiency.
- **Fault Tolerance:** If a task or node fails, the Job Tracker reassigns the task to another node that has a replica of the data.
- **Load Balancing:** The Job Tracker distributes tasks across available Task Trackers to utilize the cluster efficiently.
- **Pipeline Execution:** Multiple tasks (Map and Reduce) run concurrently across the cluster, ensuring maximum throughput.

Advantages of MapReduce Architecture

- **Parallelism:** Tasks are executed simultaneously across hundreds or thousands of nodes.
- **Scalability:** Handles increasing data volumes by simply adding more nodes to the cluster.

- **Fault Tolerance:** Automatically recovers from task or node failures using replicated data.
- **Efficiency:** Reduces network traffic by processing data locally on the nodes where it resides.
- **Simplicit :** Developers only need to implement map() and reduce() functions; Hadoop handles distribution, fault tolerance and scheduling.

Applications of MapReduce

MapReduce is mainly used when very large amounts of data need to be processed across multiple machines. The input data is divided into smaller parts, Map tasks process those parts in parallel, and Reduce tasks combine or summarize the results. It is especially useful when the same type of operation needs to be performed on many pieces of data.

1. Word Count

```
Map:      (cloud,1) (cloud,1) (cloud,1) (cloud,1)
           ↓
Shuffle: cloud → [1,1,1,1]
           ↓
Reduce:  cloud → 4
```

Word counting is one of the simplest examples of MapReduce. Suppose we have millions of documents and want to find how many times each word occurs. The Map phase reads portions of the documents and produces (word, 1) for each word. During Shuffle and Sort, identical words are grouped together, and Reduce adds their values.

2. Log Analysis

```
Map → (HTTP 404, 1)
      ↓
Shuffle → HTTP 404 → [1,1,1,1,1]
      ↓
Reduce → HTTP 404 → 5
```

Large systems continuously generate logs containing information such as user activities, errors, requests, IP addresses, and response times. MapReduce can divide large log files

among different machines, allowing each Map task to analyze a portion of the logs. The Reduce phase then combines the results.

This can be used to find the number of errors, requests per day, frequently accessed pages, or average response times.

3. Web Search and Indexing

```
Map → (cloud, document1)
Map → (cloud, document5)
Map → (cloud, document9)
      ↓
Shuffle → cloud → [document1, document5, document9]
      ↓
Reduce → consolidated index
```

Search engines need to process enormous numbers of web pages. MapReduce can help create an index that connects words with the documents where they appear. For example, different Map tasks may produce (cloud, document1), (cloud, document5), and (cloud, document9). Shuffle groups these together, and Reduce creates a consolidated index.

4. Data Filtering and Selection

```
Record → Check condition → If likes > 1,000,000 → Output
```

MapReduce can process huge datasets to find records that satisfy a particular condition. For example, if a company has millions of video records and wants to find videos with more than one million likes, the Map task can check each record and output only the matching videos. Reduce can then collect or organize the selected records.

Record → Check condition → If likes > 1,000,000 → Output

5. Sorting Large Datasets

MapReduce can also be used to sort very large datasets distributed across multiple machines. Map tasks process portions of the data, while Shuffle and Sort organize the records according to their keys. The Reduce tasks then produce the final organized output. The main advantage is that the sorting work can be distributed across a cluster instead of being handled by one machine.

6. Counting and Aggregation

```
Map:
Kathmandu → 5000
Kathmandu → 3000
Pokhara → 4000

↓ Shuffle

Kathmandu → [5000, 3000]
Pokhara → [4000]

↓ Reduce

Kathmandu → 8000
Pokhara → 4000
```

MapReduce is useful for aggregation operations such as calculating total sales, number of customers, maximum or minimum values, number of transactions, or sales by product or region.

7. Recommendation Systems

MapReduce can process large amounts of user activity and transaction data to discover patterns. For example, if many customers purchase Bread and Milk together, MapReduce can identify this relationship from millions of transactions. These patterns can then be used for product recommendations and marketing.

8. Machine Learning and Data Mining

MapReduce can be used for certain large-scale machine-learning and data-mining tasks where data can be processed independently and the results can later be combined. However, traditional MapReduce is not ideal for highly iterative algorithms because intermediate results often need to be written to storage between iterations. Systems such as Spark are generally better suited for these workloads.

MapReduce is therefore especially useful for large-scale batch processing, counting, filtering, sorting, grouping, and aggregation.

Parallel Efficiency of MapReduce

Parallel efficiency describes how effectively MapReduce uses multiple machines to reduce the time required to process a large dataset. The basic goal is to perform more work simultaneously so that the total execution time decreases.

If one machine takes 100 minutes, ideally 10 machines could process the same work in about 10 minutes. In practice, the improvement is usually smaller because communication, data movement, synchronization, and other overheads consume time.

Sequential vs Parallel Processing

In sequential processing, one machine performs the entire task: With MapReduce, the dataset is divided into parts and processed by multiple Map tasks at the same time.

Speedup

Speedup tells us how much faster a parallel system is compared with a sequential system. $\text{Speedup} = T_1 / T_p$ where T_1 is the execution time using one machine and T_p is the execution time using p machines. For example, if one machine takes 100 seconds and four machines take 30 seconds:

$$\text{Speedup} = 100 / 30 = 3.33\times$$

So, four machines provide approximately 3.33× speedup, instead of the ideal 4×.

Parallel Efficiency

Parallel efficiency tells us how effectively the available machines are being used.

$$\text{Efficiency} = \text{Speedup} / p$$

Using the previous example:

$$\text{Efficiency} = 3.33 / 4 = 0.8325$$

Therefore:

$$\text{Efficiency} \approx 83.25\%$$

This means the four-machine system achieves about 83% parallel efficiency.

Ideal vs Actual Efficiency

Ideally, four machines would reduce 100 seconds to 25 seconds.

Ideal:

1 machine → 100 sec

4 machines → 25 sec

Speedup = 4×

Efficiency = 100%

In reality, four machines might take 30 seconds because some time is spent on communication, data transfer, Shuffle and Sort, scheduling, synchronization, disk I/O, task startup, and failure handling.

Real:

1 machine → 100 sec

4 machines → 30 sec

Speedup ≈ 3.33×

Efficiency ≈ 83.25%

Therefore, real parallel efficiency is usually less than 100%.

Data Locality

Data locality means processing data close to where the data is stored. In Hadoop, data is distributed across machines using HDFS. Instead of moving a huge amount of data to a processing machine, MapReduce tries to run the Map task on or near the machine containing the required data.

Shuffle and Sort Overhead

Shuffle and Sort connects the Map phase with the Reduce phase, but it requires data to be transferred, grouped, and sorted across machines. For example, several mappers may produce values for the same key, which then have to be brought together. This network

communication and reorganization consume time, so a large Shuffle phase can reduce parallel efficiency.

Load Balancing

Parallel processing works best when the workload is distributed evenly. If one Map task receives much more data than the others, most machines may finish early while one machine continues working. The entire phase may have to wait for the slowest task. Therefore, uneven workload distribution reduces parallel efficiency.

Stragglers

A straggler is a task that takes significantly longer than other tasks. For example, if three Map tasks finish but one is still running, the entire job may have to wait for that slow task. MapReduce can use speculative execution, where a duplicate of an unusually slow task is started on another machine. The result from whichever copy finishes first is used.

Fault Tolerance

Large clusters contain many machines, so machine or task failures are expected. A machine may crash, lose network connectivity, experience disk problems, or terminate a running task. MapReduce can detect failed tasks and re-execute them, while HDFS replication provides another copy of the required data. This makes distributed processing more reliable.

I/O Overhead

Traditional MapReduce relies heavily on disk-based processing. Intermediate Map output may be written to local disk, transferred during Shuffle, and the final Reduce output is eventually written to HDFS. These I/O operations can become a major performance bottleneck. This is one reason systems such as Spark try to keep more intermediate data in memory when appropriate.

Scalability

MapReduce can handle larger workloads by adding more machines. However, adding machines does not always produce proportional performance improvement. Network

communication, Shuffle overhead, I/O, coordination, and load imbalance can reduce the benefit. Therefore, MapReduce has good scalability, but scalability is not unlimited.

Amdahl's Law

Amdahl's Law explains why adding more machines cannot produce unlimited speedup. The basic idea is that some part of a program may not be parallelizable. That sequential portion limits the maximum possible speedup.

For example, if **20%** of a task is sequential and **80%** can be parallelized across four machines:

$$\text{Speedup} = 1 / (0.20 + 0.80/4) = 2.5\times$$

So even with four machines, the speedup is only **2.5×**, rather than the ideal 4×. The efficiency of MapReduce depends on how well the system can perform parallel work while minimizing overhead.

A good MapReduce job generally has:

- Enough work to process in parallel
- Good data locality
- Balanced workloads
- Low communication and I/O overhead
- Few stragglers
- Efficient Shuffle and Sort
- Appropriate numbers of Map and Reduce tasks
- Good fault handling

Cloud Platforms and Applications

Web Services

Web services are software components that allow different applications to communicate and exchange data over a network using standard web technologies. They provide a standard interface through which one application can request services or data from another application, regardless of the programming language or operating system used.

The most important characteristic of web services is interoperability, which means applications developed using different technologies can communicate with each other. They also provide loose coupling, so applications do not need to know the internal implementation of the service. Web services are based on standard technologies such as HTTP/HTTPS, XML, JSON, SOAP, and REST, making them reusable across different applications.

Types of Web Services

1. SOAP (Simple Object Access Protocol)

SOAP is a protocol for exchanging structured information between applications. It primarily uses XML for communication and follows a formal message structure. SOAP is commonly used where strict standards, reliability, and security are important.

2. REST (Representational State Transfer)

REST is an architectural style that uses standard HTTP methods such as GET, POST, PUT, and DELETE to access and manipulate resources. REST commonly uses JSON because it is lightweight and easy to process. It is widely used for modern web applications and cloud APIs.

Web services provide a common way for cloud applications, users, and resources to communicate. Cloud providers expose many resources through APIs and web services, allowing applications to access computing resources, storage, databases, and other services over the Internet.

AppEngine

Google App Engine (AppEngine) is a Platform as a Service (PaaS) provided by Google that allows developers to build, deploy, and manage web applications without directly managing the underlying servers and infrastructure.

AppEngine provides automatic scaling, meaning computing resources can increase or decrease according to application demand. It also provides a managed infrastructure, where Google handles much of the server, networking, and infrastructure management. Developers can therefore concentrate mainly on developing and deploying their applications rather than maintaining physical servers.

AppEngine can also integrate with other Google Cloud services, making it suitable for developing complete cloud-based applications.

Advantages: The main advantage of AppEngine is that it reduces infrastructure management and simplifies application deployment. It can automatically handle changes in workload, making it suitable for applications whose resource requirements vary over time.

Developer → AppEngine → Cloud Infrastructure → Users

The developer deploys the application to AppEngine. AppEngine manages the required infrastructure and scaling, while users access the application through the Internet.

Azure Platform

Microsoft Azure is a cloud computing platform that provides a wide range of services for computing, storage, networking, databases, application development, and other cloud-based requirements.

Azure provides compute services for running applications and virtual machines, storage services for storing different types of data, and database services for managing structured and unstructured data. It also provides networking services for communication between cloud resources and application services for developing and deploying cloud applications.

Features: Azure provides scalability and elasticity, allowing resources to be increased or decreased according to workload. It also provides high availability, global infrastructure, and a pay-as-you-go pricing model. Azure supports integration with various development tools and enterprise systems.

Advantages: Azure allows organizations to use computing resources without maintaining all the required physical infrastructure themselves. It supports applications ranging from small web applications to large enterprise systems and allows resources to be scaled as requirements change.

Aneka

Aneka is a cloud application platform that provides a framework for developing, deploying, and managing **distributed applications** in cloud environments. It can use physical and virtual computing resources to execute applications.

Aneka can be understood through three main layers:

- 1. Application Layer:** Contains the applications that are developed and executed using Aneka.
- 2. Service Layer:** Provides services such as application execution, resource management, scheduling, monitoring, and other cloud-related functions.

3. Infrastructure Layer: Contains the underlying physical or virtual computing resources on which applications are executed.

Features: Aneka supports different programming models for distributed applications and provides resource management and scheduling. It can work with heterogeneous resources, allowing different types of computing resources to participate in application execution.

Applications: Aneka can be used for scientific computing, engineering applications, data processing, and business applications, particularly where large computational tasks need to be distributed across multiple resources.

Open Challenges

Although cloud platforms provide scalability, flexibility, and cost benefits, several challenges must be addressed.

- **Security and Privacy**

Cloud applications often store sensitive data on shared infrastructure. Therefore, organizations must protect data and identities against unauthorized access, attacks, and privacy violations.

- **Performance and Availability**

Cloud applications are expected to remain available and provide acceptable performance even when workloads change or infrastructure failures occur. Maintaining reliable performance across distributed cloud resources can be challenging.

- **Interoperability and Portability**

Different cloud providers may use different technologies, APIs, and services. This can make it difficult to move an application or data from one cloud platform to another.

- **Data Management**

Cloud environments may handle very large amounts of data. Managing data storage, transfer, consistency, backup, and availability becomes increasingly difficult as the amount of data grows.

- **Vendor Lock-in**

An organization may become highly dependent on the services and technologies of a particular cloud provider. Moving applications and data to another provider can then become technically difficult or expensive.

Scientific Applications

Scientific applications use cloud computing to perform complex calculations, simulations, experiments, and analysis of large datasets. These applications often require significant computing and storage resources. Scientific applications include weather and climate modeling, genomics, astronomy, physics simulations, and engineering simulations.

Cloud computing is useful for these applications because researchers can obtain large amounts of computing power when required without maintaining expensive infrastructure permanently. Cloud resources can also be scaled according to the computational requirements of the scientific task.

Example: A weather research organization can use cloud computing to run large-scale weather and climate simulations, which may require significant processing power and storage.

Business and Consumer Applications

- **Business Applications**

Cloud computing is widely used in business for applications such as Customer Relationship Management (CRM), enterprise applications, online collaboration, data analytics, and backup and disaster recovery.

The main advantage is that organizations can access applications and resources without maintaining all the underlying infrastructure themselves. Cloud services also provide scalability, remote access, easier collaboration, and faster deployment.

- **Consumer Applications**

Cloud computing is also part of many everyday consumer services. Examples include online file storage, web-based email, photo and video storage, video and music streaming, and online gaming.

These services allow users to access their data and applications from different devices. Cloud storage also enables synchronization, so changes made on one device can be reflected on other devices.

References

Websites



- IBM – <https://www.ibm.com>



- GeeksforGeeks – <https://www.geeksforgeeks.org>



- Medium – <https://medium.com>

YouTube Channels



Gate Smashers ✓

@GateSmashers • 2.83M subscribers • 2.2K videos

Welcome to Gate Smashers, one of the fastest-growing EdTech communities with 2.6 M+ ...more

🔗 Facebook and 1 more link

🔔 Subscribed ▼

👤 Community

- Gate Smasher – <https://www.youtube.com/@GateSmashers>



ByteMonk ✓

@ByteMonk • 398K subscribers • 553 videos

ByteMonk - Where Every Byte Sparks Insight! ...more

🔗 academy.bytemonk.io/courses and 3 more links

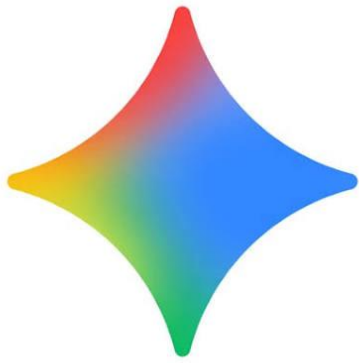
🔔 Subscribed ▼

- ByteMonk – <https://www.youtube.com/@ByteMonk>

AI Tools Used



- ChatGPT - <https://chatgpt.com>



- Gemini - <https://gemini.google.com/app>

Drawing tool:



apperaser.io - <https://app.eraser.io/>

Book:

